الفصل 1

اختبار

تعريفات

- **المصفوفة** (جدول ثنائي البعد): جدول من الأسطر والأعمدة، نصل إلى كل خانة بفهرسين `matrix[row][col]`.
- **التصريح**: `int sales[3][4];` ثلاثة أسطر وأربعة أعمدة (12 خانة).
- **التخزين**: الأسطر متتالية في الذاكرة (سطر بعد سطر).
- **المرور**: حلقتان متداخلتان، الخارجية على الأسطر والداخلية على الأعمدة (أو العكس لجميع الأعمدة).

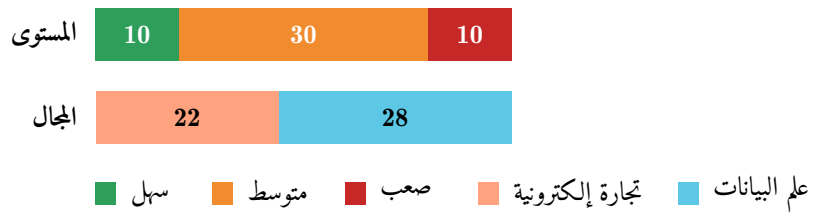
	day 3	day 2	day 1	day 0	
	180	140	95	120	store 0
	90	75	60	80	store 1
sales[2][...] →	250	190	210	200	store 2

العملية	الفكرة
مجموع سطر	الحلقة الداخلية على الأعمدة، والمجمّع يُهيأ لكل سطر
مجموع عمود	نبدل الحلقتين: الخارجية على الأعمدة
المنقول	`transposed[col][row] = matrix[row][col]`
الجداء	`product[row][col] = ∑ A[row][k] * B[k][col]` (ثلاث حلقات)
البرمجة الديناميكية	كل خانة تُحسب من خانات سبق حسابها (فوق، يسار)

ترتيب الفهارس

`matrix[row][col]`: السطر أولًا ثم العمود. الخلط بينهما يقرأ خانات خاطئة أو يتجاوز حدود المصفوفة.

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 5.1 جدول مبيعات المتاجر

تجارة إلكترونية سهل

مبيعات 3 متاجر خلال 4 أيام مخزنة في مصفوفة `sales[3][4]` (السطر = المتجر، العمود = اليوم). اكتب برنامجًا يعرضها على شكل جدول منسق.

خطوات الحل

1) نهيئ المصفوفة بأقواس متداخلة: سطر لكل متجر.
2) حلقة على الأسطر، وداخلها حلقة على الأعمدة.
3) نطبع سطرًا جديدًا بعد كل متجر.

البرنامج بلغة C

```c
#include <stdio.h>

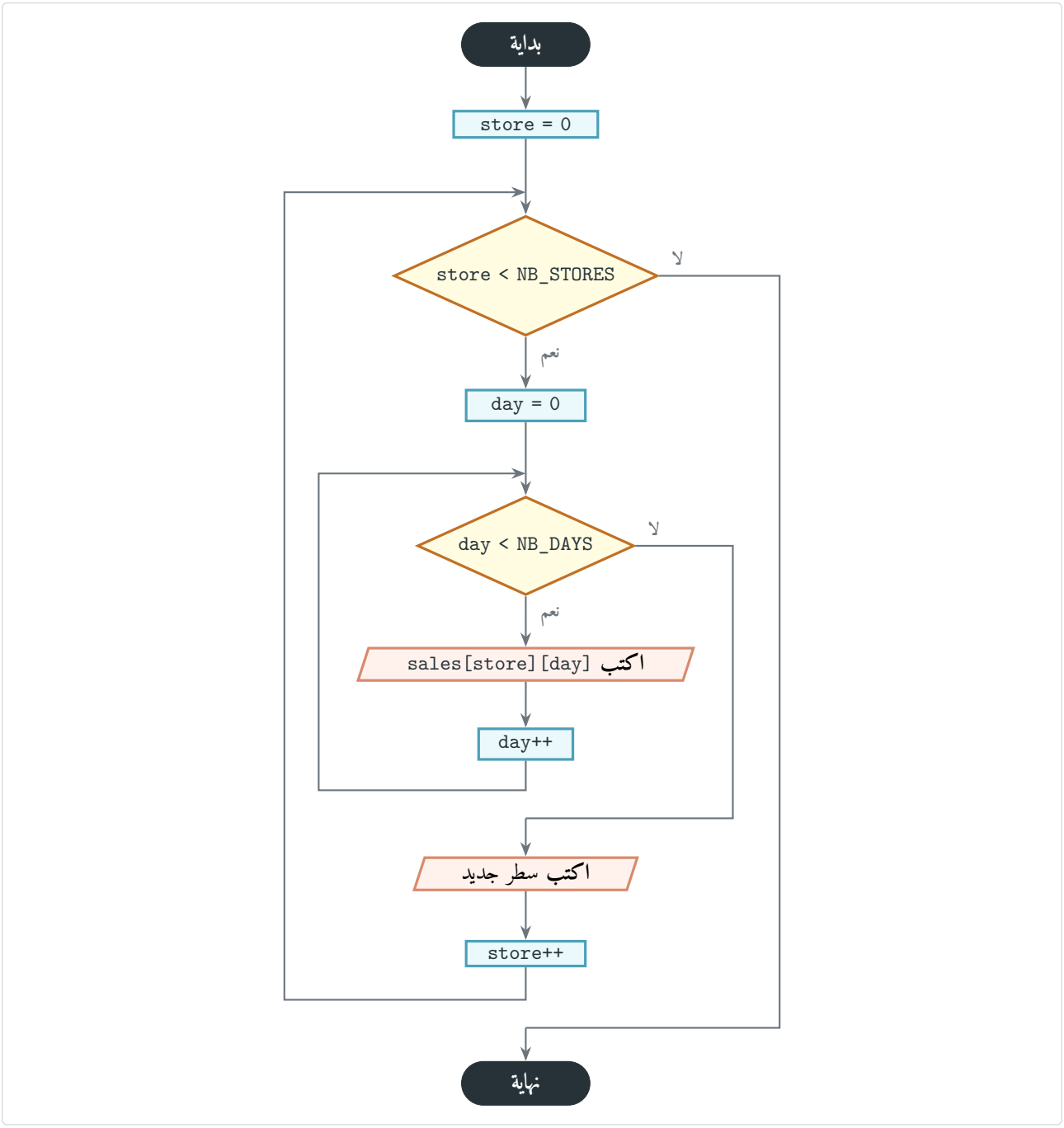
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {
        {120, 95, 140, 180},          // المتجر 0
        { 80, 60,  75,  90},          // المتجر 1
        {200, 210, 190, 250}          // المتجر 2
    };
    int store, day;
    printf("Store | D1  D2  D3  D4\n");
    for (store = 0; store < NB_STORES; store++) {      // الأسطر
        printf("%5d |", store);
        for (day = 0; day < NB_DAYS; day++)           // الأعمدة
            printf(" %3d", sales[store][day]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Store | D1  D2  D3  D4
    0 | 120  95 140 180
    1 |  80  60  75  90
    2 | 200 210 190 250
```

المخطط الإنسيابي (Organigramme)



تمرين 5.2 المجموع الكلي للمبيعات

علم البيانات

سهل

اكتب برنامجًا يحسب مجموع كل خانات مصفوفة المبيعات (3 متاجر × 4 أيام) والمتوسط لكل خانة.

خطوات الحل

1) مجمّع واحد `total`.
2) حلقتان متداخلتان تضيفان كل خانة.
3) المتوسط: `total / (NB_STORES * NB_DAYS)`.

البرنامج بلغة C

```c
#include <stdio.h>

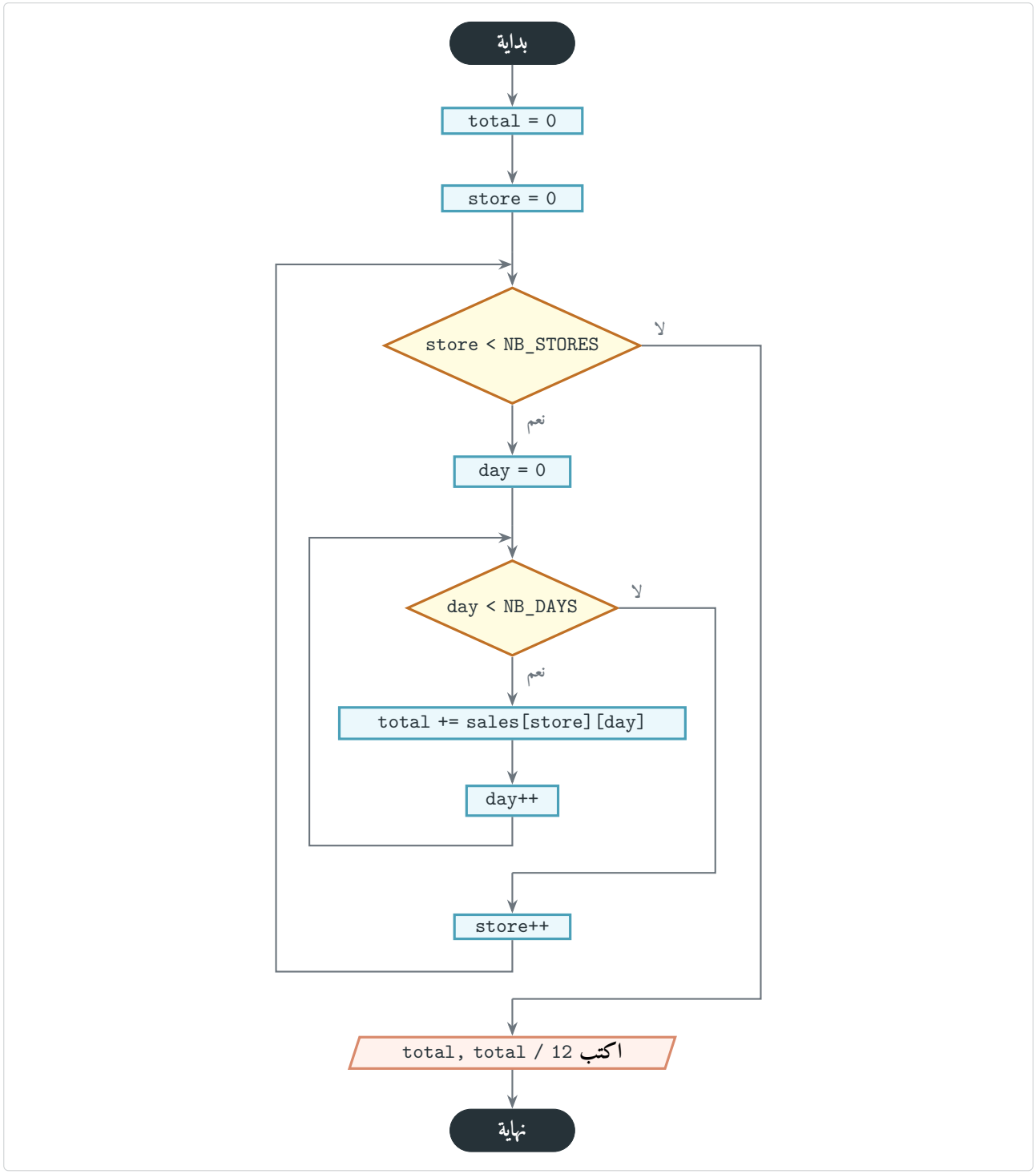
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, total = 0;
    for (store = 0; store < NB_STORES; store++)
        for (day = 0; day < NB_DAYS; day++)
            total += sales[store][day];   // كل خانة مرة واحدة
    printf("Total = %d\n", total);
    printf("Average per cell = %.2f\n", (double)total / (NB_STORES * NB_DAYS));
    return 0;
}
```

نتيجة التنفيذ

```
Total = 1690
Average per cell = 140.83
```

المخطط الإنسيابي (Organigramme)



تمرين 5.3 مجموع كل متجر

تجارة إلكترونية | سهل

اكتب برنامجًا يحسب مجموع مبيعات **كل متجر** (مجموع كل سطر) ويخزنها في جدول `storeTotal`.

خطوات الحل

1) لكل سطر: نهيئ `storeTotal[store] = 0` ثم نجمع أعمدته.

البرنامج بلغة C

```c
#include <stdio.h>

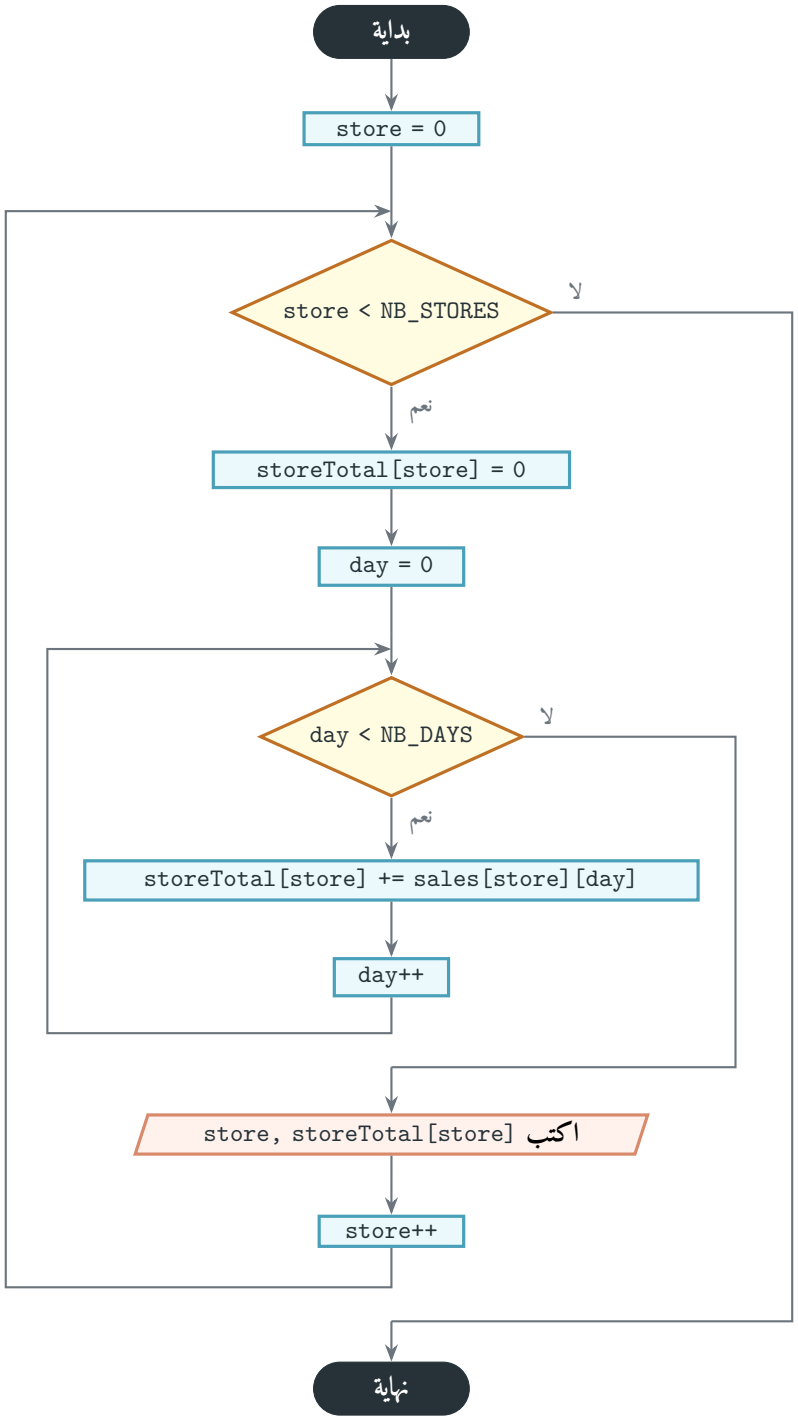
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int storeTotal[NB_STORES];
    int store, day;
    for (store = 0; store < NB_STORES; store++) {
        storeTotal[store] = 0;            // مجمّع خاص بكل سطر
        for (day = 0; day < NB_DAYS; day++)
            storeTotal[store] += sales[store][day];
        printf("Store %d: %d\n", store, storeTotal[store]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Store 0: 535
Store 1: 305
Store 2: 850
```

المخطط الإنسيابي (Organigramme)



تمرين 5.4 مجموع كل يوم

تجارة إلكترونية | سهل

اكتب برنامجًا يحسب مجموع مبيعات **كل يوم** في كل المتاجر (مجموع كل عمود). لاحظ أن الحلقة الخارجية تصبح على الأعمدة.

خطوات الحل

1) الحلقة الخارجية على `day`، والداخلية على `store`.
2) `dayTotal[day] += sales[store][day]`.

البرنامج بلغة C

```c
#include <stdio.h>

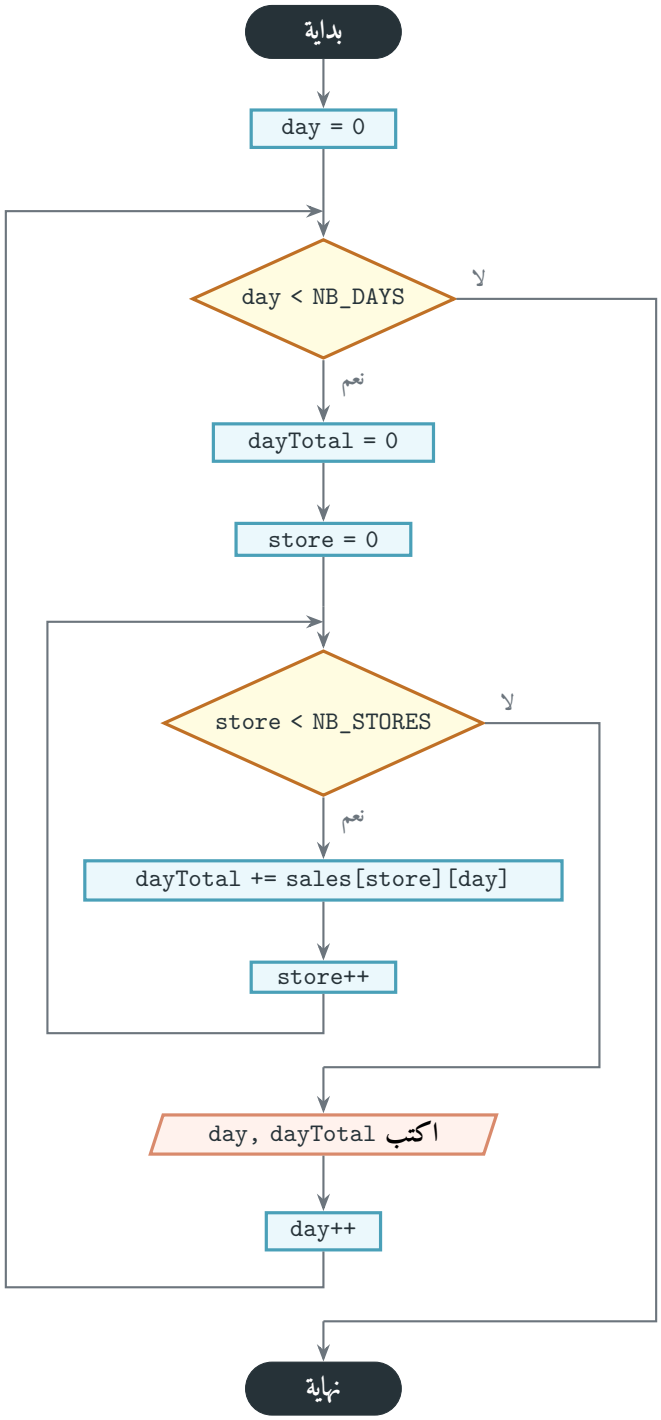
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, dayTotal;
    for (day = 0; day < NB_DAYS; day++) {          // الأعمدة في الخارج
        dayTotal = 0;
        for (store = 0; store < NB_STORES; store++)
            dayTotal += sales[store][day];
        printf("Day %d: %d\n", day + 1, dayTotal);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Day 1: 400
Day 2: 365
Day 3: 405
Day 4: 520
```

المخطط الإنسيابي (Organigramme)



تمرين 5.5 أفضل خانة في المصفوفة

علم البيانات ⭐ سهل

اكتب برنامجًا يجد أكبر قيمة مبيعات في المصفوفة مع المتجر واليوم الموافقين.

خطوات الحل

1) نحفظ موضع الأكبر `bestStore` و`bestDay` (نبدأ بالخانة `[0][0]`).
2) نقارن كل خانة بـ `sales[bestStore][bestDay]`.

البرنامج بلغة C

```c
#include <stdio.h>

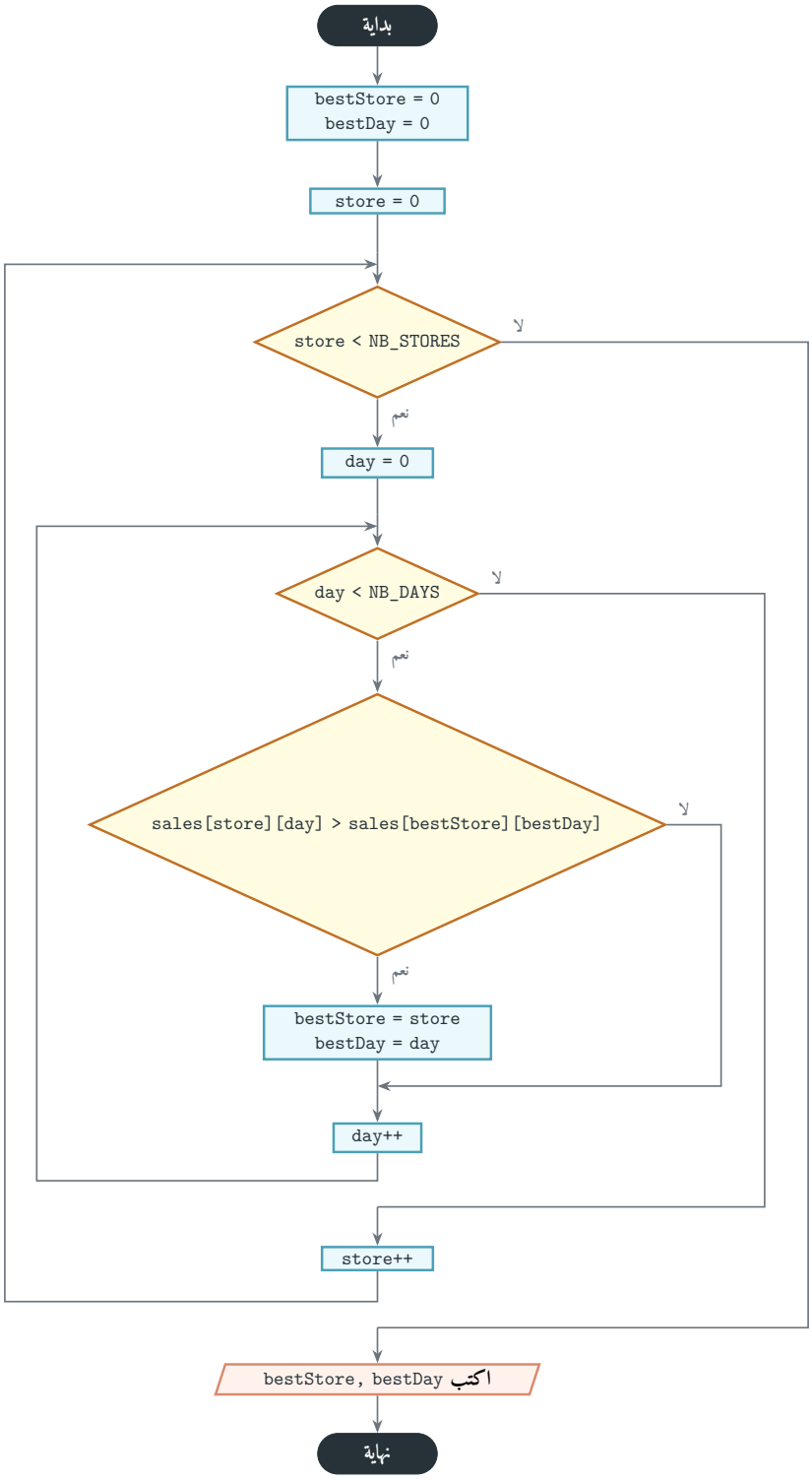
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, bestStore = 0, bestDay = 0;
    for (store = 0; store < NB_STORES; store++)
        for (day = 0; day < NB_DAYS; day++)
            if (sales[store][day] > sales[bestStore][bestDay]) {
                bestStore = store;        // نحفظ الإحداثيتين
                bestDay = day;
            }
    printf("Best: store %d, day %d -> %d\n", bestStore, bestDay + 1,
           sales[bestStore][bestDay]);
    return 0;
}
```

نتيجة التنفيذ

```
Best: store 2, day 4 -> 250
```

المخطط الإنسيابي (Organigramme)



تمرين 5.6 الرفوف الفارغة في المستودع

تجارة إلكترونية | سهل

مصفوفة `stock[4][5]` تمثل مخزون المنتجات (الأعمدة) في 4 مستودعات (الأسطر). اكتب برنامجًا يعرض إحداثيات كل خانة مخزونها 0 ويعد عددها.

خطوات الحل

1) حلقتان متداخلتان.
2) إذا كان `stock[depot][product] == 0` نعرض الإحداثيتين ونزيد العداد.

البرنامج بلغة C

```c
#include <stdio.h>

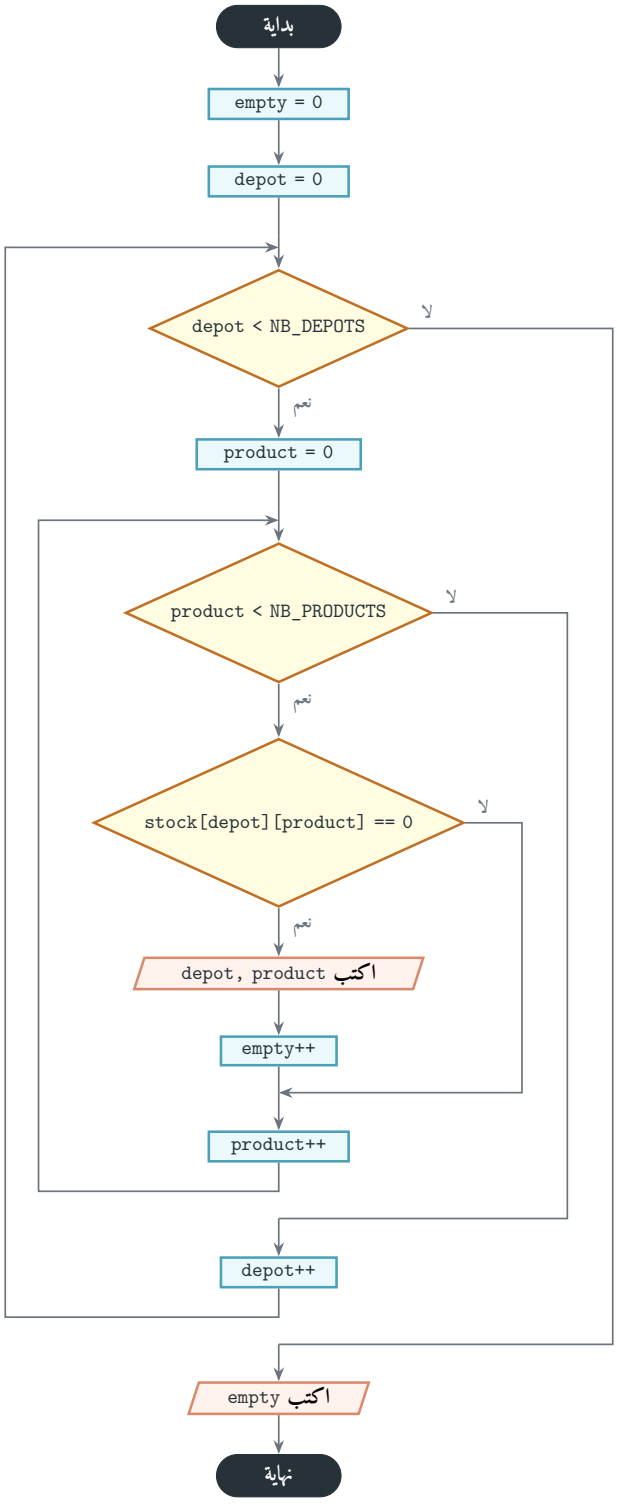
#define NB_DEPOTS 4
#define NB_PRODUCTS 5

int main(void) {
    int stock[NB_DEPOTS][NB_PRODUCTS] = {{5, 0, 12, 3, 8},
                                         {0, 4,  0, 9, 1},
                                         {7, 7,  2, 0, 6},
                                         {3, 0,  5, 5, 0}};
    int depot, product, empty = 0;
    for (depot = 0; depot < NB_DEPOTS; depot++)
        for (product = 0; product < NB_PRODUCTS; product++)
            if (stock[depot][product] == 0) {
                printf("Depot %d, product %d is empty\n", depot, product);
                empty++;
            }
    printf("%d empty slots\n", empty);
    return 0;
}
```

نتيجة التنفيذ

```
Depot 0, product 1 is empty
Depot 1, product 0 is empty
Depot 1, product 2 is empty
Depot 2, product 3 is empty
Depot 3, product 1 is empty
Depot 3, product 4 is empty
6 empty slots
```

المخطط الإنسيابي (Organigramme)



تمرين 5.7 تحديل كل الأسعار في المصفوفة

تجارة إلكترونية سهل

مصفوفة `prices[3][3]` تحتوي أسعار 3 منتجات (أسطر) بثلاثة أحجام (أعمدة). اكتب برنامجًا يطبق تخفيضًا `percent`% على كل الأسعار في مكانها ثم يعرضها.

خطوات الحل

1) نقرأ النسبة.

2) `prices[product][size] *= (1 - percent / 100)`

لكل خانة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double prices[3][3] = {{1200, 1500, 1800},     // صغير، متوسط، كبير
                           { 450,  600,  750},
                           {3000, 3600, 4200}};
    double percent;
    int product, size;
    scanf("%lf", &percent);
    for (product = 0; product < 3; product++) {
        for (size = 0; size < 3; size++) {
            prices[product][size] *= 1 - percent / 100;    // التعديل في المكان
            printf("%8.2f", prices[product][size]);
        }
        printf("\n");
    }
    return 0;
}
```

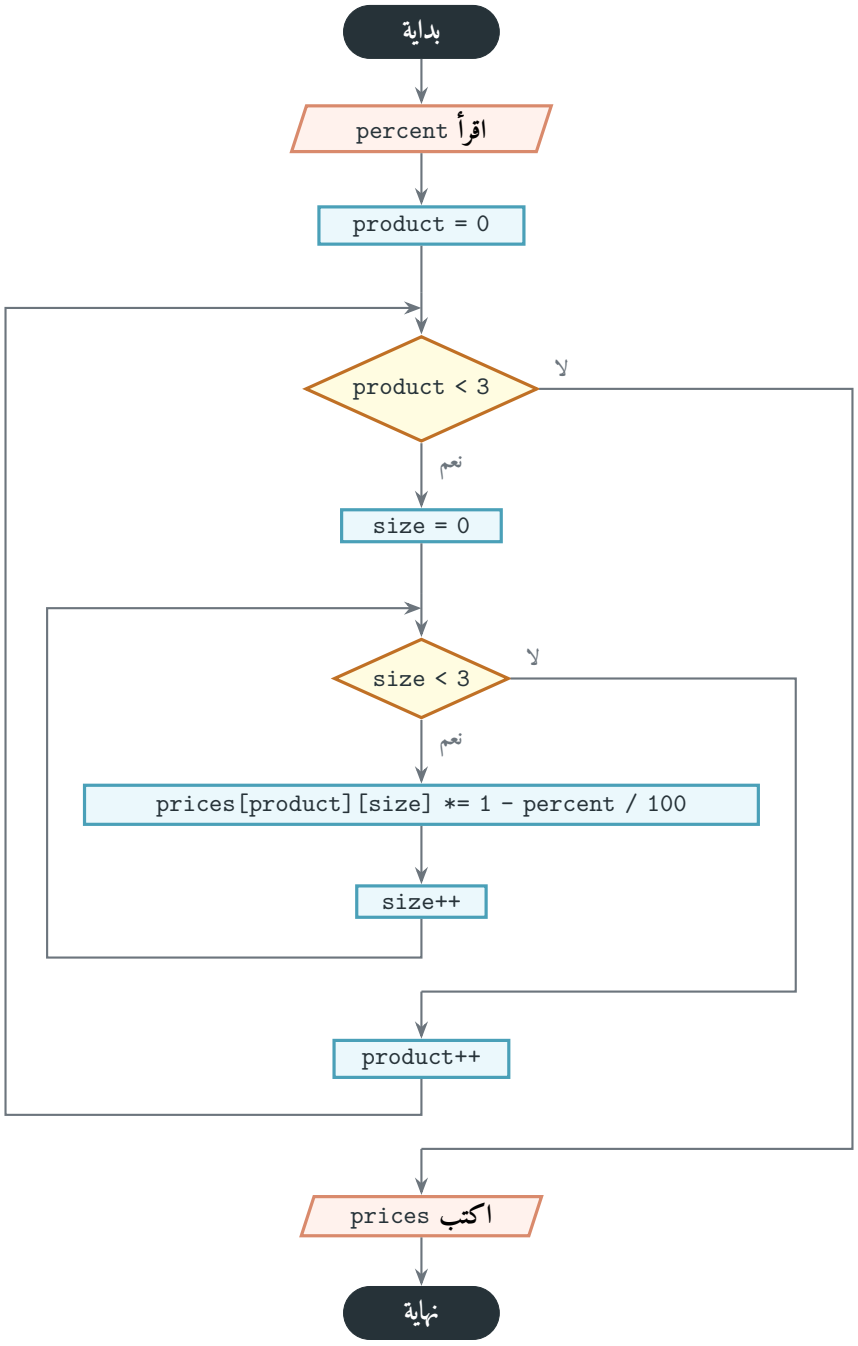
المدخلات

```
15
```

نتيجة التنفيذ

```
1020.00 1275.00 1530.00
 382.50  510.00  637.50
2550.00 3060.00 3570.00
```

المخطط الإنسيابي (Organigramme)



تمرين 5.8 قراءة مصفوفة تقييمات

علم البيانات ⭐ سهل

اكتب برنامجًا يقرأ عدد الزبائن `nbUsers` وعدد المنتجات `nbItems` (لا يتجاوزان 10) ثم مصفوفة التقييمات، ويعرضها مع متوسط كل زبون.

خطوات الحل

1) نصرح بمصفوفة بالسعة القصوى `ratings[10][10]` ونستعمل جزءًا منها.
2) القراءة بحلقتين متداخلتين.
3) متوسط كل سطر عند العرض.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 10

int main(void) {
    int ratings[MAX][MAX];
    int nbUsers, nbItems, user, item, rowSum;
    scanf("%d %d", &nbUsers, &nbItems);
    for (user = 0; user < nbUsers; user++)
        for (item = 0; item < nbItems; item++)
            scanf("%d", &ratings[user][item]);     // عنوان الخانة
    for (user = 0; user < nbUsers; user++) {
        rowSum = 0;
        printf("User %d:", user);
        for (item = 0; item < nbItems; item++) {
            printf(" %d", ratings[user][item]);
            rowSum += ratings[user][item];
        }
        printf("  avg %.2f\n", (double)rowSum / nbItems);
    }
    return 0;
}
```

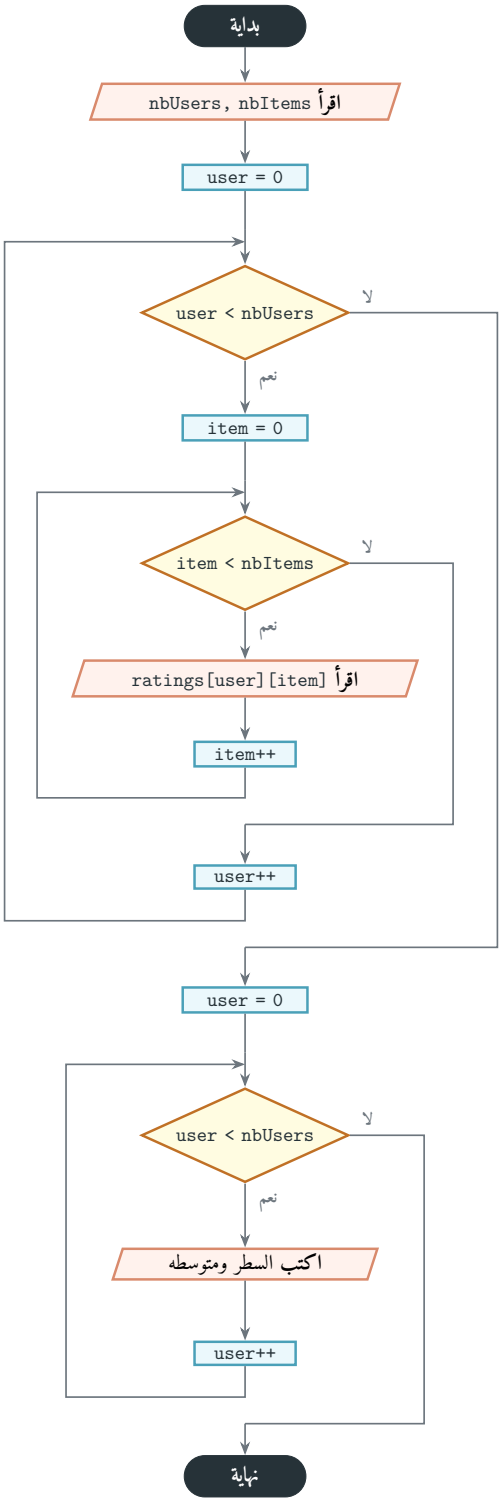
المدخلات

```
3 4
5 3 0 4
4 0 4 5
1 2 5 0
```

نتيجة التنفيذ

```
User 0: 5 3 0 4  avg 3.00
User 1: 4 0 4 5  avg 3.25
User 2: 1 2 5 0  avg 2.00
```

المخطط الإنسيابي (Organigramme)



تمرين 5.9 القطر الرئيسي لمصفوفة التحويلات

تجارة إلكترونية
سهل

مصفوفة مربعة `transfers[4][4]` تعطي عدد الطرود المرسلة من مستودع (سطر) إلى مستودع (عمود). القطر الرئيسي يمثل الطرود التي بقيت في نفس المستودع. اكتب برنامجًا يعرض القطر ومجموعه (الأثر) ومجموع الطرود المنقولة فعلًا.

خطوات الحل

1) عناصر القطر هي `transfers[depot][depot]`: حلقة واحدة تكفي.

2) المنقولة = المجموع الكلي − الأثر.

البرنامج بلغة C

```c
#include <stdio.h>

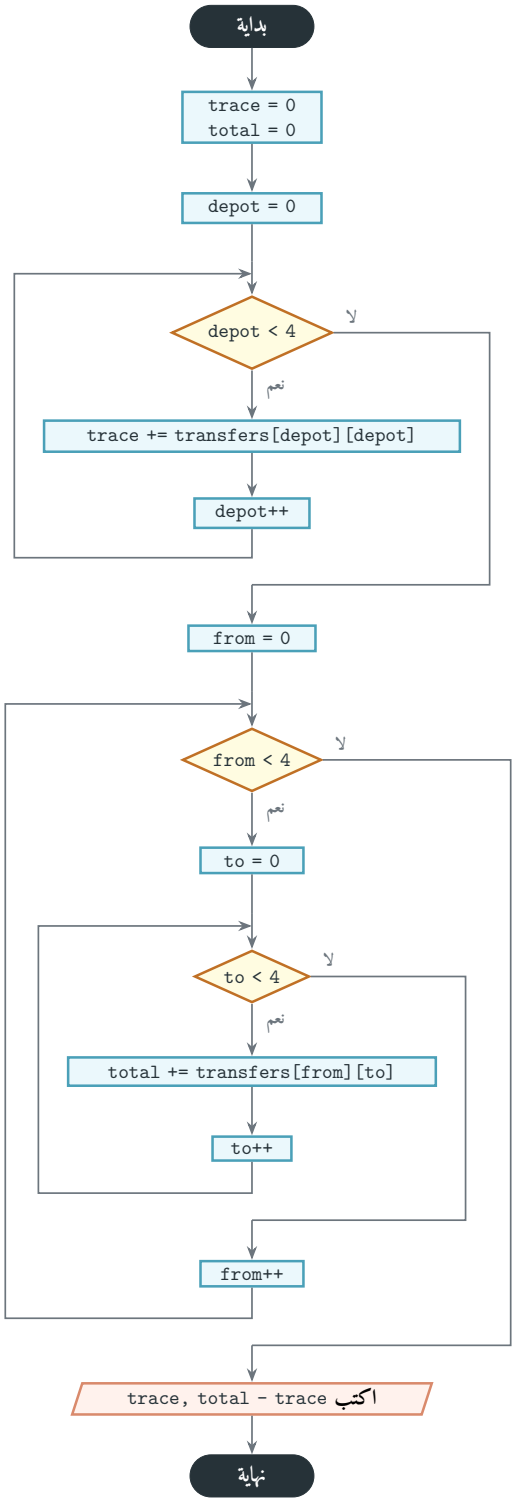
#define NB 4

int main(void) {
    int transfers[NB][NB] = {{30,  5,  2, 0},
                             { 4, 25,  6, 1},
                             { 0,  7, 40, 3},
                             { 2,  1,  5, 18}};
    int depot, from, to, trace = 0, total = 0;
    printf("Diagonal:");
    for (depot = 0; depot < NB; depot++) {
        printf(" %d", transfers[depot][depot]);   // السطر = العمود
        trace += transfers[depot][depot];
    }
    for (from = 0; from < NB; from++)
        for (to = 0; to < NB; to++)
            total += transfers[from][to];
    printf("\nKept = %d, moved = %d\n", trace, total - trace);
    return 0;
}
```

نتيجة التنفيذ

```
Diagonal: 30 25 40 18
Kept = 113, moved = 36
```

المخطط الإنسيابي (Organigramme)



تمرين 5.10 البحث عن منتج في الرفوف

تجارة إلكترونية | سهل

مصفوفة `shelves[3][4]` تحتوي رقم المنتج الموضوع في كل رف. اكتب برنامجًا يبحث عن منتج ويعرض **رقم الممر والرف** أو "غير موجود"، ويتوقف عند أول تطابق.

خطوات الحل

1) `found = 0`، الحلقتان تستمران ما دام `!found`.
2) عند التطابق نحفظ الموضع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int shelves[3][4] = {{101, 204, 330, 415},
                         {512, 618, 777, 801},
                         {905, 999, 120, 343}};
    int target, aisle, slot, found = 0, foundAisle = 0, foundSlot = 0;
    scanf("%d", &target);
    for (aisle = 0; aisle < 3 && !found; aisle++)          // نخرج من الحلقتين
        for (slot = 0; slot < 4 && !found; slot++)
            if (shelves[aisle][slot] == target) {
                found = 1;
                foundAisle = aisle;
                foundSlot = slot;
            }
    if (found)
        printf("Product %d: aisle %d, slot %d\n", target, foundAisle, foundSlot);
    else
        printf("Product %d not found\n", target);
    return 0;
}
```

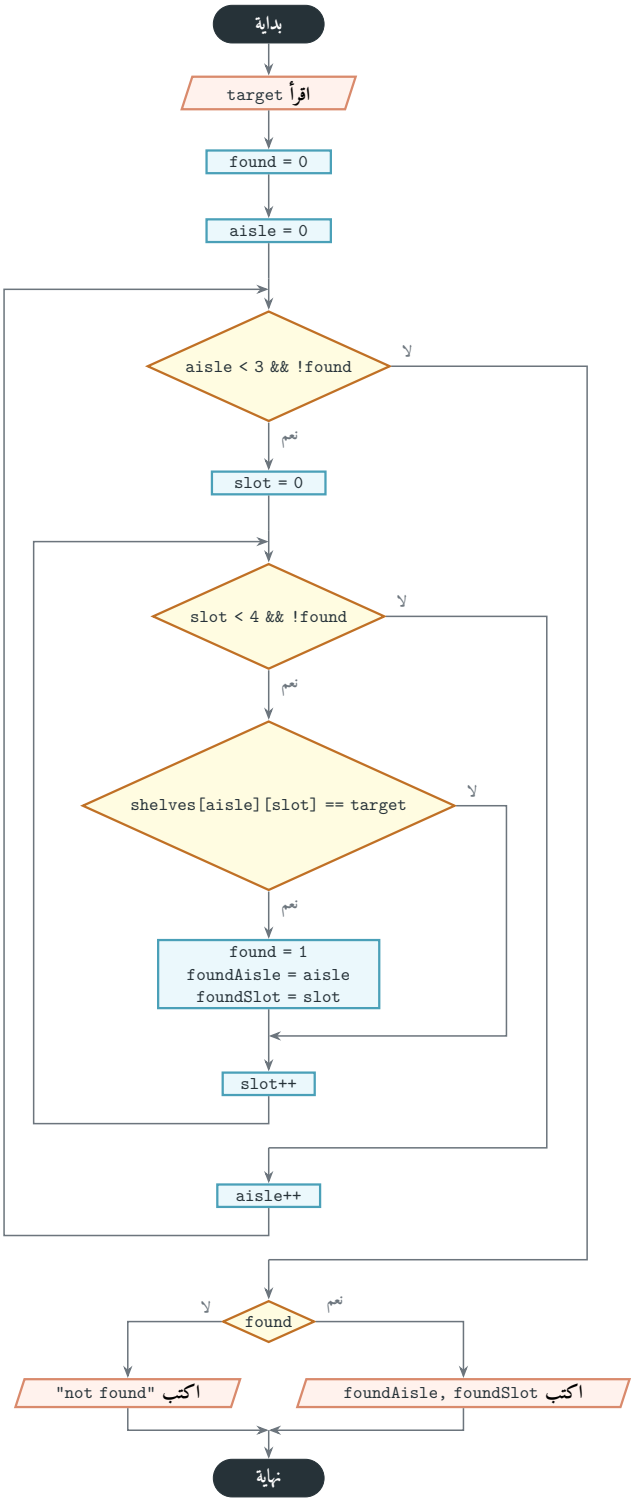
المدخلات

```
512
```

نتيجة التنفيذ

```
Product 512: aisle 1, slot 0
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 5.11 منقول المصفوفة

متوسط ⭐⭐ | علم البيانات

مصفوفة `purchases[3][5]` (3 زبائن × 5 منتجات). يحتاج المحلل النظرة المعاكسة: منتجات × زبائن. اكتب برنامجًا يحسب المنقول `transposed[5][3]` ويعرضه.

خطوات الحل

1) `transposed[item][user] = purchases[user][item]`

.

2) أبعاد المنقول معكوسة.

البرنامج بلغة C

```c
#include <stdio.h>

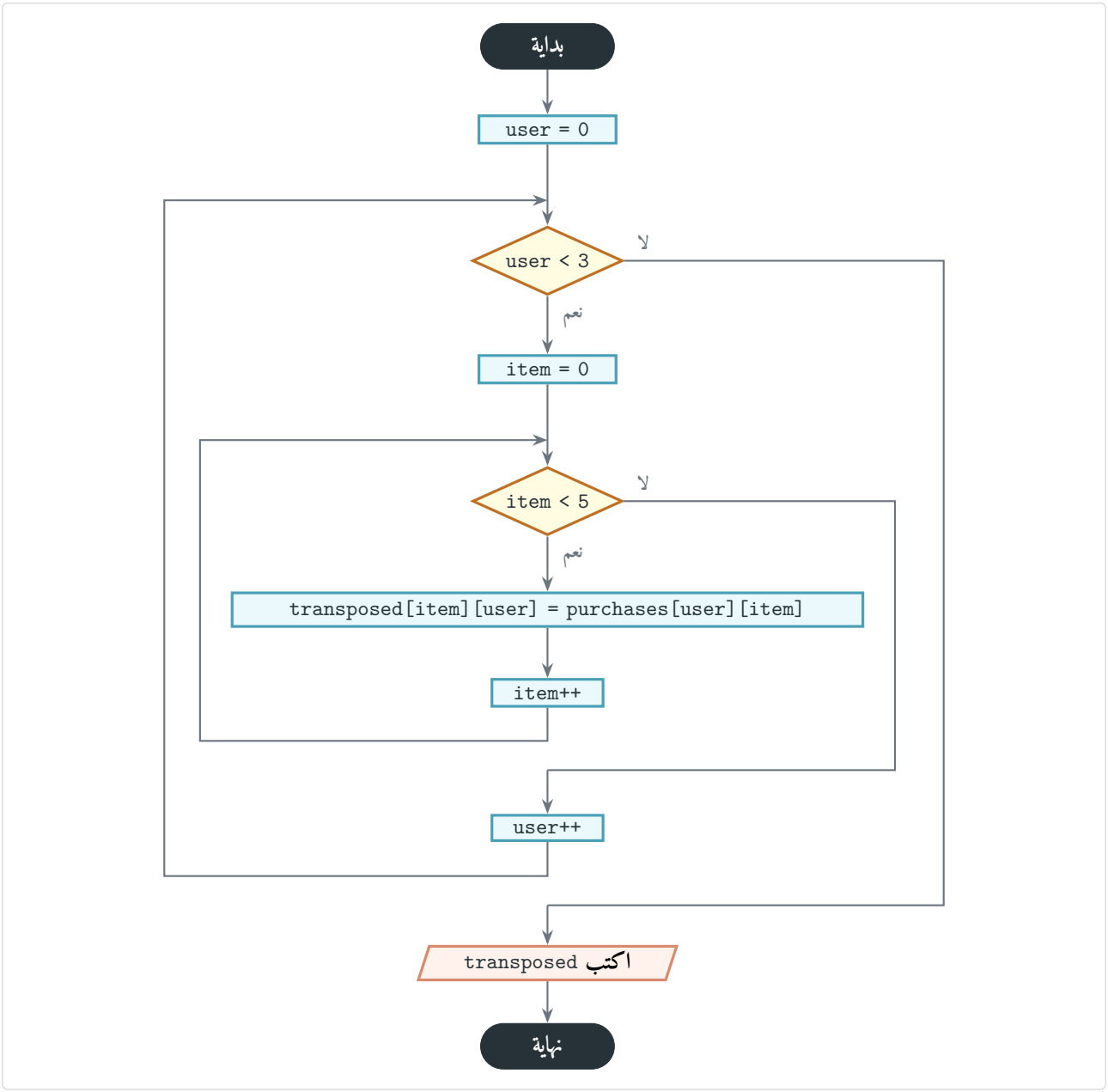
#define NB_USERS 3
#define NB_ITEMS 5

int main(void) {
    int purchases[NB_USERS][NB_ITEMS] = {{1, 0, 2, 0, 1},
                                         {0, 3, 0, 1, 0},
                                         {2, 1, 0, 0, 4}};
    int transposed[NB_ITEMS][NB_USERS];
    int user, item;
    for (user = 0; user < NB_USERS; user++)
        for (item = 0; item < NB_ITEMS; item++)
            transposed[item][user] = purchases[user][item];    // تبديل الفهرسين
    for (item = 0; item < NB_ITEMS; item++) {
        printf("Item %d:", item);
        for (user = 0; user < NB_USERS; user++)
            printf(" %d", transposed[item][user]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 1 0 2
Item 1: 0 3 1
Item 2: 2 0 0
Item 3: 0 1 0
Item 4: 1 0 4
```

المخطط الإنسيابي (Organigramme)



تمرين 5.12 جمع مبيعات شهرين

تجارة إلكترونية ★★ متوسط

لدينا مصفوفتا مبيعات (منتجات × متاجر) لشهري جانفي وفيفري. اكتب برنامجًا يحسب **مجموعهما** خانة بخانة، ثم **نسبة النمو** لكل خانة من الشهر الأول إلى الثاني.

خطوات الحل

1) `total[row][col] = jan[row][col] + feb[row][col]`

.

2) `growth = (feb - jan) / jan * 100` إذا كان `jan > 0`.

البرنامج بلغة C

```c
#include <stdio.h>

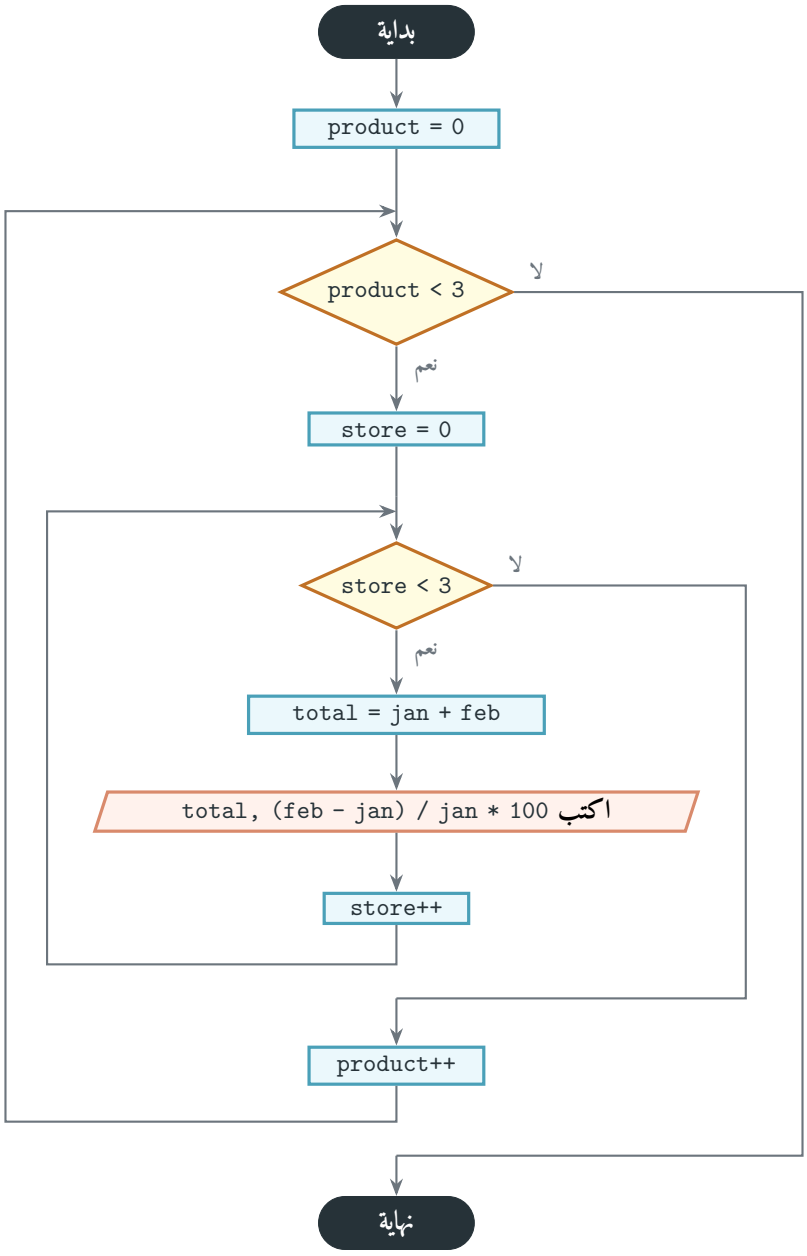
#define NB 3

int main(void) {
    int jan[NB][NB] = {{120, 80, 60}, {40, 55, 70}, {300, 210, 150}};
    int feb[NB][NB] = {{132, 70, 66}, {52, 55, 63}, {330, 240, 150}};
    int total[NB][NB], product, store;
    for (product = 0; product < NB; product++) {
        for (store = 0; store < NB; store++) {
            total[product][store] = jan[product][store] + feb[product][store];
            printf("%4d (%+5.1f%%)", total[product][store],
                   (feb[product][store] - jan[product][store]) * 100.0 / jan[product][store]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
252 (+10.0%) 150 (-12.5%) 126 (+10.0%)
 92 (+30.0%) 110 ( +0.0%) 133 (-10.0%)
630 (+10.0%) 450 (+14.3%) 300 ( +0.0%)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.13 جداء المصفوفات: كلفة الطلبيات عند الموردين

تجارة إلكترونية ★★ متوسط

`quantities[2][3]`: كمية كل منتج (3 منتجات) في كل طلبية (2 طلبيات). `unitPrice[3][2]`: سعر كل منتج عند كل مورد (2 موردين). اكتب برنامجًا يحسب **جداء المصفوفتين** `cost[2][2]`: كلفة كل طلبية عند كل مورد، ويحدد المورد الأرخص لكل طلبية.

خطوات الحل

1) `cost[order][supplier] = Σ quantities[order][prod] * unitPrice[prod][supplier]`

.

2) ثلاث حلقات متداخلة: السطر، العمود، ثم المجموع.

3) الأرخص: مقارنة عمودي كل سطر.

البرنامج بلغة C

```c
#include <stdio.h>

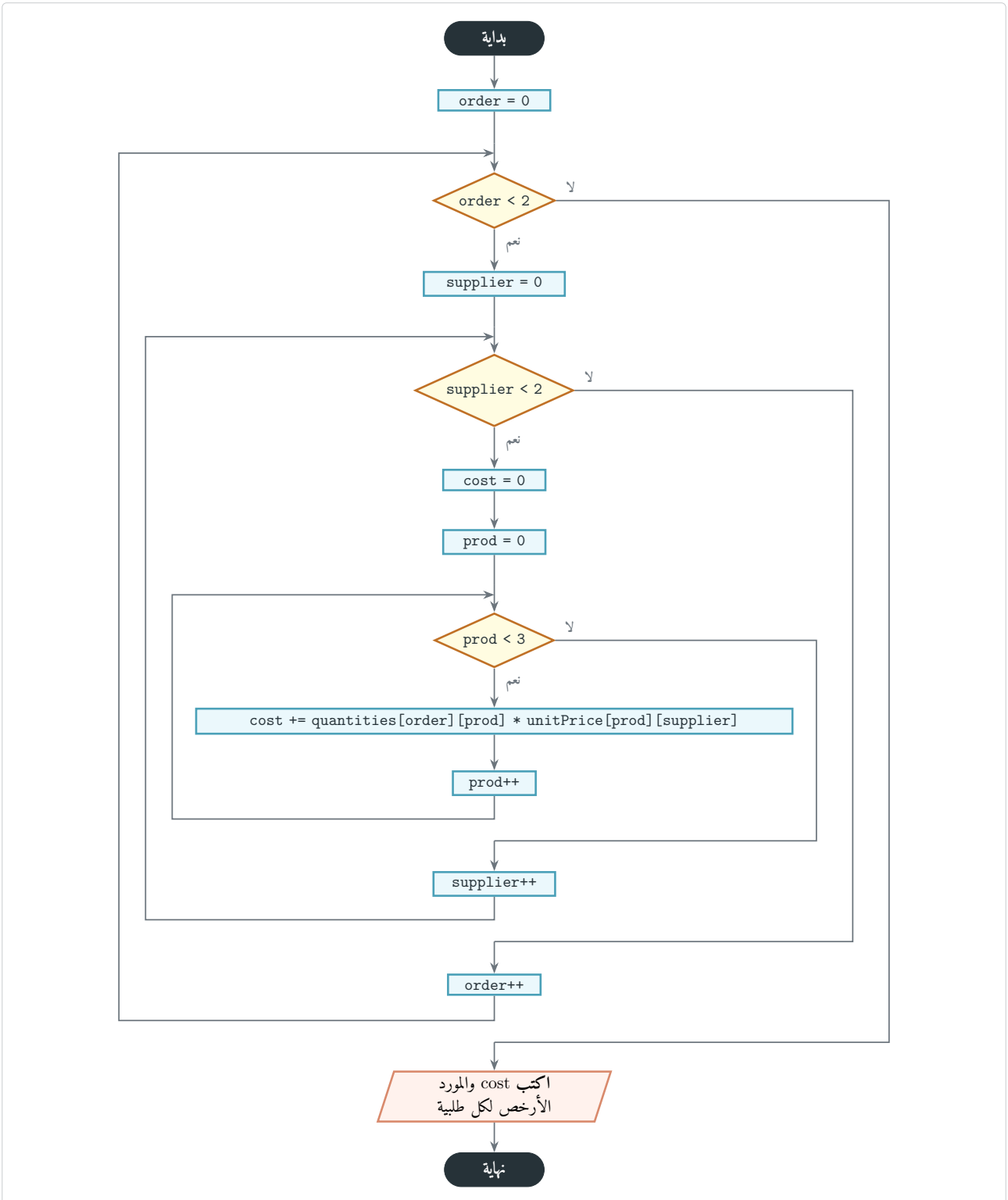
#define NB_ORDERS 2
#define NB_PRODUCTS 3
#define NB_SUPPLIERS 2

int main(void) {
    int quantities[NB_ORDERS][NB_PRODUCTS] = {{10, 5, 2}, {0, 20, 8}};
    int unitPrice[NB_PRODUCTS][NB_SUPPLIERS] = {{100, 95}, {40, 48}, {250, 230}};
    int cost[NB_ORDERS][NB_SUPPLIERS];
    int order, supplier, prod;
    for (order = 0; order < NB_ORDERS; order++) {
        for (supplier = 0; supplier < NB_SUPPLIERS; supplier++) {
            cost[order][supplier] = 0;
            for (prod = 0; prod < NB_PRODUCTS; prod++)   // سطر × عمود
                cost[order][supplier] += quantities[order][prod] * unitPrice[prod][supplier];
        }
        printf("Order %d: supplier A = %d, supplier B = %d -> choose %c\n", order,
               cost[order][0], cost[order][1], cost[order][0] <= cost[order][1] ? 'A' : 'B');
    }
    return 0;
}
```

نتيجة التنفيذ

```
Order 0: supplier A = 1700, supplier B = 1650 -> choose B
Order 1: supplier A = 2800, supplier B = 2800 -> choose A
```

المخطط الإنسيابي (Organigramme)



تمرين 5.14 فاتورة كل زبونٌ بجداء مصفوفة في شعاع

تجارة إلكترونية ★★ متوسط

مصفوفة `cart[4][5]`: كمية كل منتج في سلة كل زبون. جدول `price[5]`: أسعار المنتجات. اكتب برنامجًا يحسب **جداء المصفوفة في الشعاع**: فاتورة كل زبون، والزبون الأكبر إنفاقًا.

خطوات الحل

1) `bill[customer] = Σ cart[customer][item] * price[item]`

.

2) نحتفظ بأكبر فاتورة.

البرنامج بلغة C

```c
#include <stdio.h>

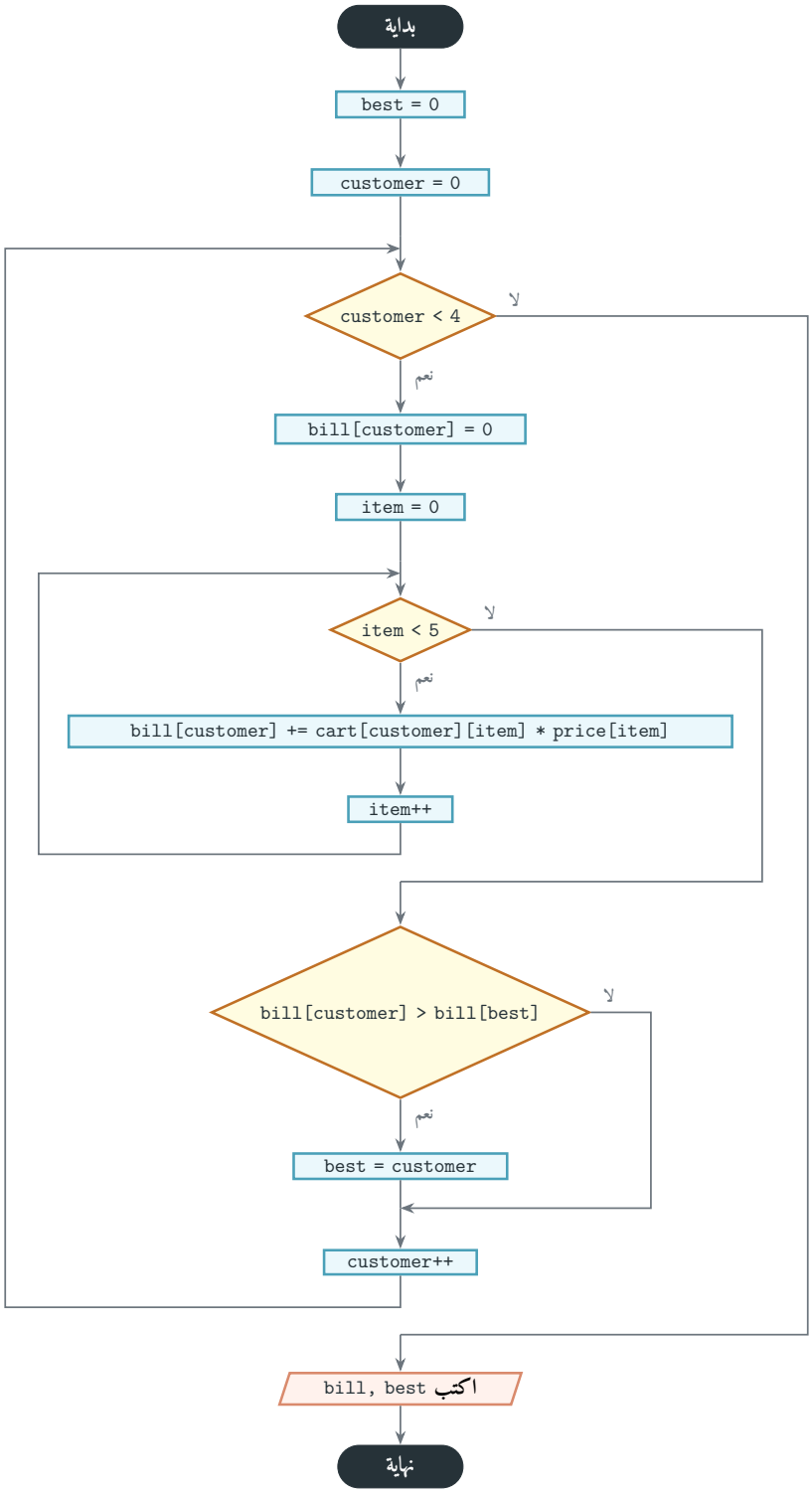
#define NB_CUST 4
#define NB_ITEMS 5

int main(void) {
    int cart[NB_CUST][NB_ITEMS] = {{1, 0, 2, 0, 1},
                                   {0, 3, 0, 1, 0},
                                   {2, 1, 0, 0, 4},
                                   {0, 0, 1, 2, 2}};
    double price[NB_ITEMS] = {1500, 350, 2200, 999, 120};
    double bill[NB_CUST];
    int customer, item, best = 0;
    for (customer = 0; customer < NB_CUST; customer++) {
        bill[customer] = 0;
        for (item = 0; item < NB_ITEMS; item++)
            bill[customer] += cart[customer][item] * price[item];  // كمية × سعر
        printf("Customer %d: %.2f DA\n", customer, bill[customer]);
        if (bill[customer] > bill[best])
            best = customer;
    }
    printf("Top spender: customer %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Customer 0: 6020.00 DA
Customer 1: 2049.00 DA
Customer 2: 3830.00 DA
Customer 3: 4438.00 DA
Top spender: customer 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.15 أفضل متجر في كل يوم

علم البيانات ★★ متوسط

اكتب برنامجًا يحدد لكل يوم (عمود) **المتجر الأفضل** (السطر ذو الأكبر قيمة)، ثم يعد عدد الأيام التي فاز بها كل متجر.

خطوات الحل

1) لكل عمود: بحث عن فهرس السطر الأكبر `winner`.
2) `wins[winner]++`.

البرنامج بلغة C

```c
#include <stdio.h>

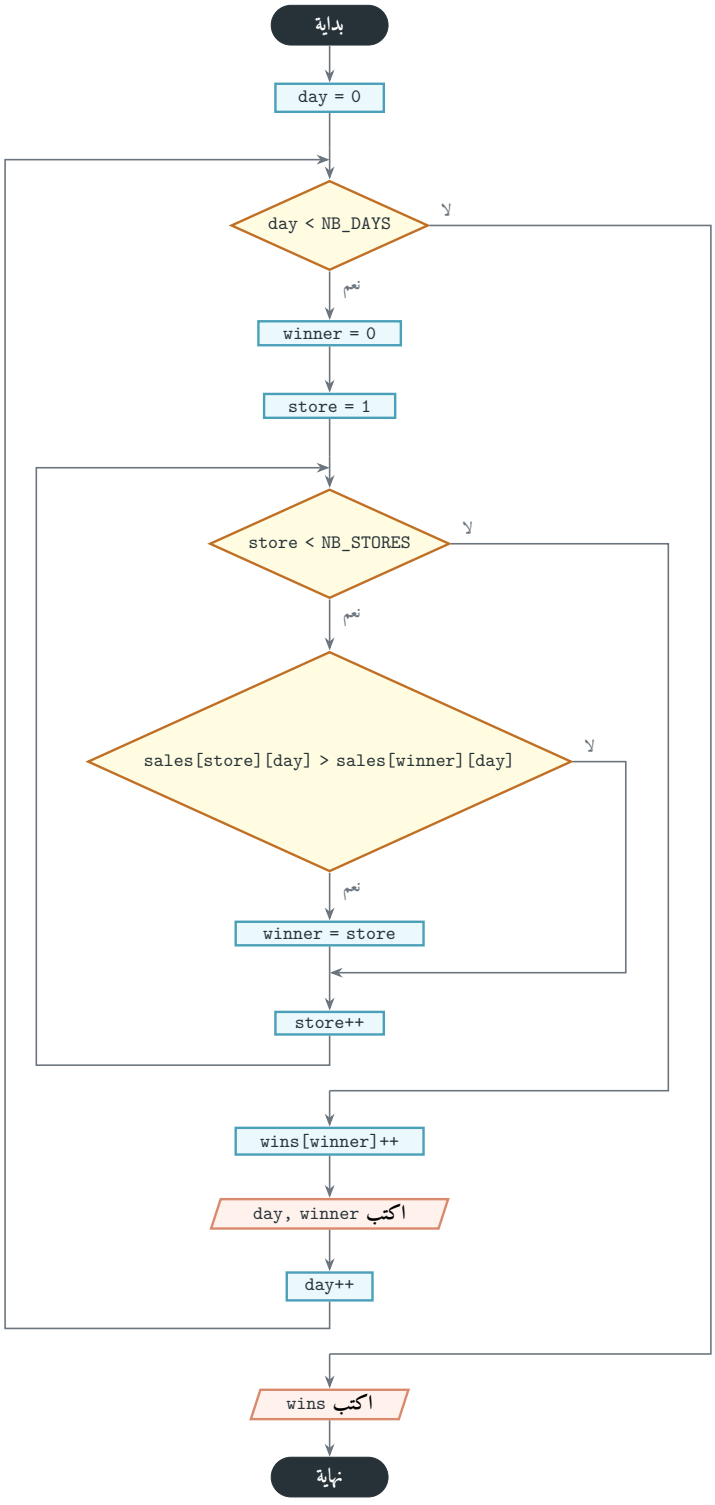
#define NB_STORES 3
#define NB_DAYS 6

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180, 90, 210},
                                     {130, 60, 175,  90, 95, 100},
                                     {100, 99, 150, 170, 80, 240}};
    int wins[NB_STORES] = {0};
    int day, store, winner;
    for (day = 0; day < NB_DAYS; day++) {
        winner = 0;                          // نبحث في العمود day
        for (store = 1; store < NB_STORES; store++)
            if (sales[store][day] > sales[winner][day])
                winner = store;
        wins[winner]++;
        printf("Day %d: store %d\n", day + 1, winner);
    }
    for (store = 0; store < NB_STORES; store++)
        printf("Store %d won %d day(s)\n", store, wins[store]);
    return 0;
}
```

نتيجة التنفيذ

```
Day 1: store 1
Day 2: store 2
Day 3: store 1
Day 4: store 0
Day 5: store 1
Day 6: store 2
Store 0 won 1 day(s)
Store 1 won 3 day(s)
Store 2 won 2 day(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.16 متوسط تقييم كل منتج دون الخانات الفارغة

علم البيانات ★★ متوسط

في مصفوفة التقييمات (زبائن × منتجات) القيمة **0** تعني "لم يقيّم". اكتب برنامجًا يحسب متوسط كل منتج على التقييمات الموجودة فقط، وعدد مقيّميه، ثم المنتج الأعلى تقييمًا.

خطوات الحل

1) لكل عمود: نجمع الخانات غير الصفرية ونعدها.
2) المتوسط `sum / count` إذا كان `count > 0`.

البرنامج بلغة C

```c
#include <stdio.h>

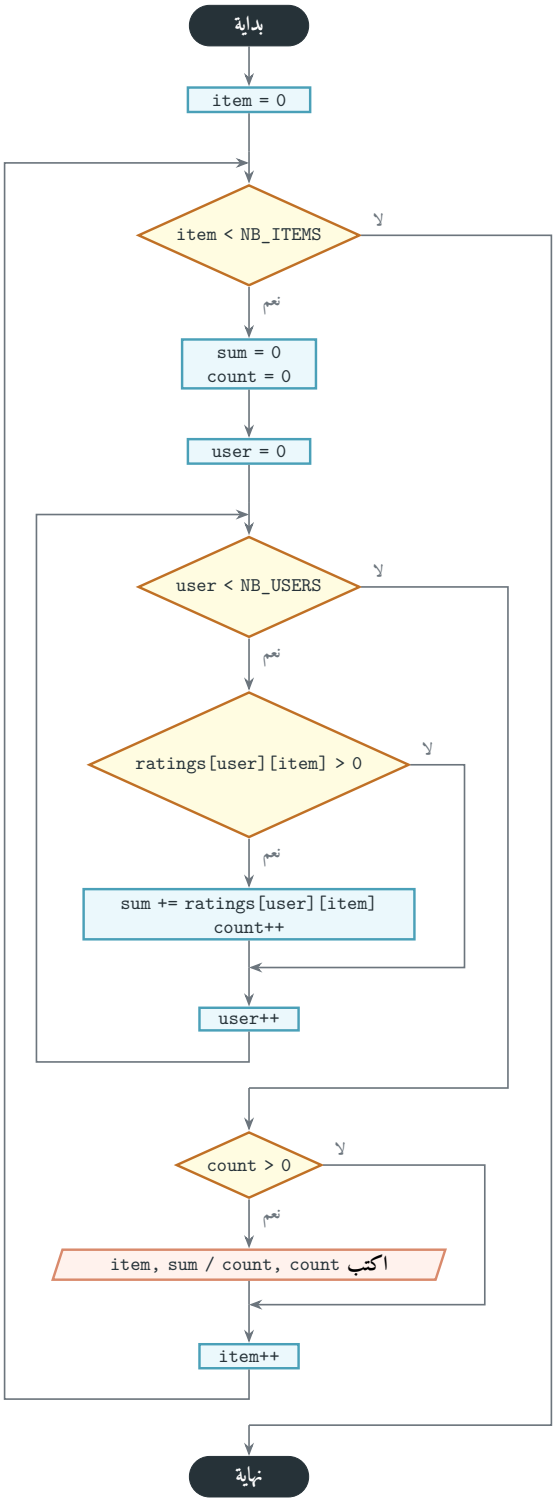
#define NB_USERS 5
#define NB_ITEMS 4

int main(void) {
    int ratings[NB_USERS][NB_ITEMS] = {{5, 3, 0, 4},
                                       {4, 0, 0, 5},
                                       {1, 2, 5, 0},
                                       {0, 3, 4, 4},
                                       {5, 0, 3, 0}};
    int user, item, sum, count, best = -1;
    double avg, bestAvg = 0;
    for (item = 0; item < NB_ITEMS; item++) {
        sum = 0;
        count = 0;
        for (user = 0; user < NB_USERS; user++)
            if (ratings[user][item] > 0) {    // نتجاهل الخانات الفارغة
                sum += ratings[user][item];
                count++;
            }
        if (count > 0) {
            avg = (double)sum / count;
            printf("Item %d: %.2f (%d ratings)\n", item, avg, count);
            if (avg > bestAvg) { bestAvg = avg; best = item; }
        }
    }
    printf("Best rated: item %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 3.75 (4 ratings)
Item 1: 2.67 (3 ratings)
Item 2: 4.00 (3 ratings)
Item 3: 4.33 (3 ratings)
Best rated: item 3
```

المخطط الإنسيابي (Organigramme)



تمرين 5.17 تطبيع أعمدة جدول الخصائص

علم البيانات ★★ متوسط

مصفوفة `features[6][3]`: لكل منتج (سطر) ثلاث خصائص (السعر، الوزن، المبيعات) بوحدات مختلفة. اكتب برنامجًا **يطبع** كل **عمود** على حدة إلى `[0, 1]` (Min-Max)، لأن نماذج التعلم الآلي حساسة لاختلاف المقاييس.

خطوات الحل

1) لكل عمود: نجد `colMin` و`colMax`.
2) ثم نحول كل خانة في العمود.

البرنامج بلغة C

```c
#include <stdio.h>

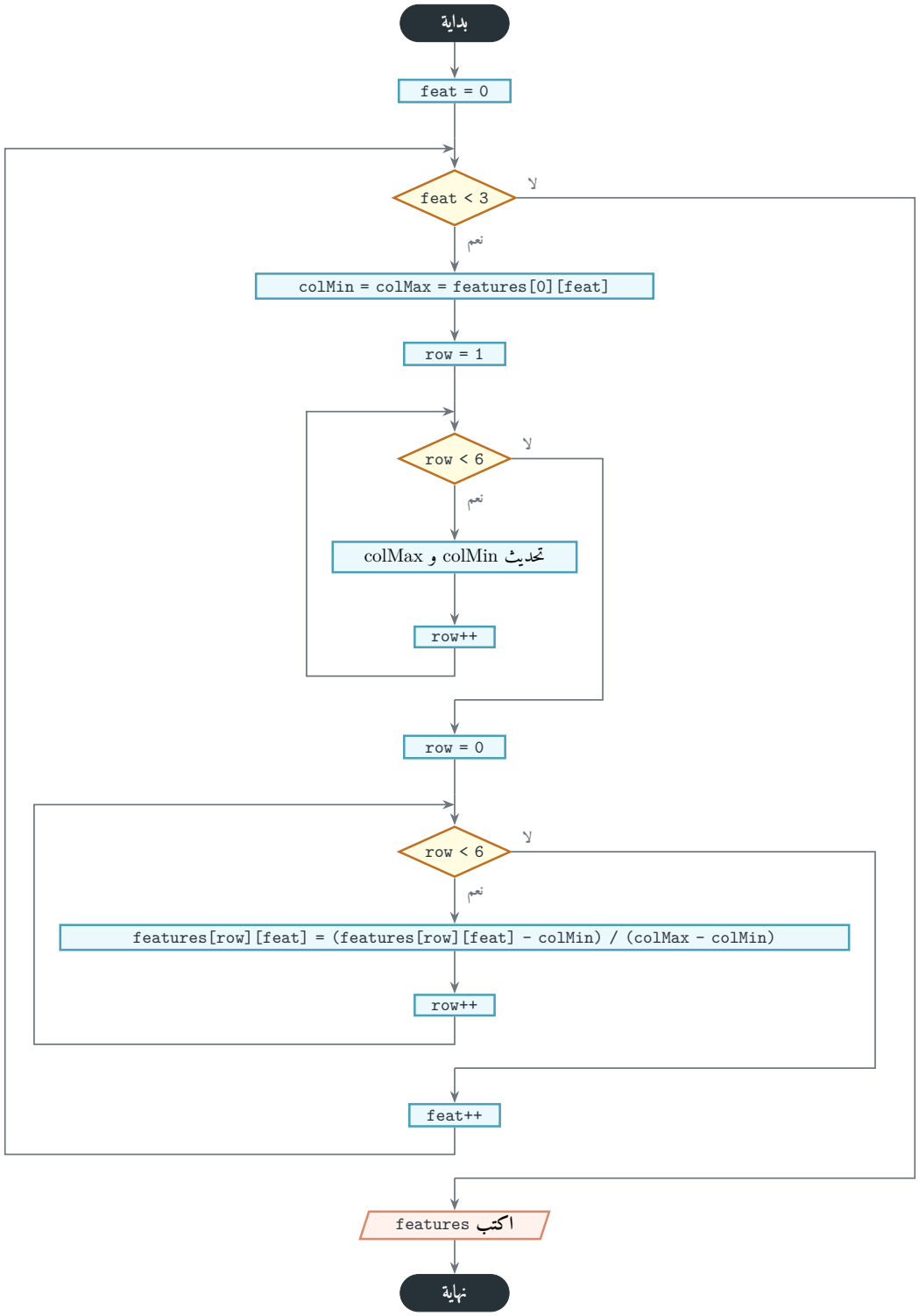
#define NB_ROWS 6
#define NB_FEAT 3

int main(void) {
    double features[NB_ROWS][NB_FEAT] = {{1200, 0.5, 340},
                                         {4500, 2.0,  80},
                                         { 300, 0.2, 900},
                                         {8000, 3.5,  45},
                                         {2500, 1.1, 210},
                                         { 950, 0.8, 500}};
    double colMin, colMax;
    int feat, row;
    for (feat = 0; feat < NB_FEAT; feat++) {          // عمود بعد عمود
        colMin = colMax = features[0][feat];
        for (row = 1; row < NB_ROWS; row++) {
            if (features[row][feat] < colMin) colMin = features[row][feat];
            if (features[row][feat] > colMax) colMax = features[row][feat];
        }
        for (row = 0; row < NB_ROWS; row++)
            features[row][feat] = (features[row][feat] - colMin) / (colMax - colMin);
    }
    printf(" price weight sales\n");
    for (row = 0; row < NB_ROWS; row++)
        printf(" %.2f  %.2f   %.2f\n", features[row][0], features[row][1], features[row][2]);
    return 0;
}
```

نتيجة التنفيذ

```
price weight sales
0.12  0.09   0.35
0.55  0.55   0.04
0.00  0.00   1.00
1.00  1.00   0.00
0.29  0.27   0.19
0.08  0.18   0.53
```

المخطط الإنسيابي (Organigramme)



تمرين 5.18 مصفوفة الإرتباط

علم البيانات ★★ متوسط

لثلاث خصائص لكل زبون (الزيارات، الدقائق، المبلغ) اكتب برنامجًا يحسب **مصفوفة الارتباط** `corr[3][3]` حيث `corr[a][b]` معامل بيرسون بين الخاصيتين، باستعمال الجذر بطريقة نيوتن.

خطوات الحل

1) نحسب متوسط كل عمود.

2) `cov[first][second] = Σ (x - mean)(y - mean)`

.

3) `corr = cov[first][second] / cov[first][first])  * cov[second][second])`

.

البرنامج بلغة C

```c
#include <stdio.h>

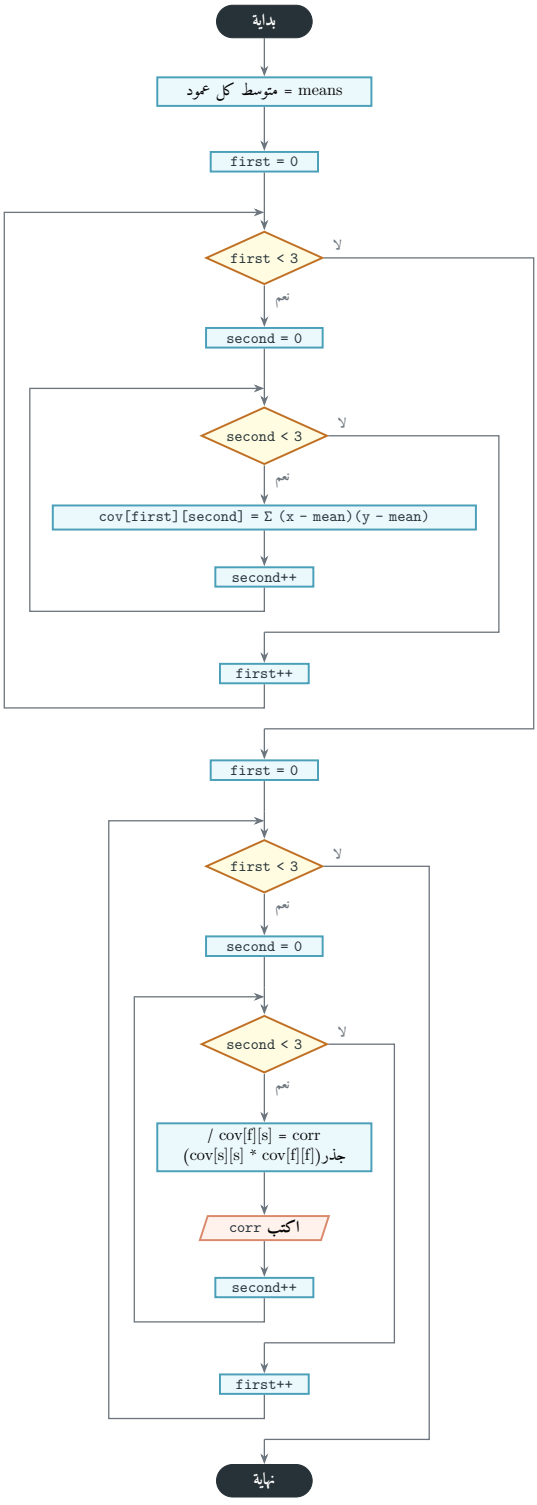
#define NB 6
#define NB_FEAT 3

int main(void) {
    double data[NB][NB_FEAT] = {{3, 12, 1500}, {8, 40, 5200}, {5, 22, 2400},
                                {1,  4,  300}, {9, 50, 6100}, {4, 25, 1900}};
    double mean[NB_FEAT] = {0}, cov[NB_FEAT][NB_FEAT], product, root;
    int row, first, second, step;
    for (row = 0; row < NB; row++)
        for (first = 0; first < NB_FEAT; first++)
            mean[first] += data[row][first] / NB;
    for (first = 0; first < NB_FEAT; first++)
        for (second = 0; second < NB_FEAT; second++) {
            cov[first][second] = 0;           // التغاير (دون القسمة على n)
            for (row = 0; row < NB; row++)
                cov[first][second] += (data[row][first] - mean[first])
                                    * (data[row][second] - mean[second]);
        }
    for (first = 0; first < NB_FEAT; first++) {
        for (second = 0; second < NB_FEAT; second++) {
            product = cov[first][first] * cov[second][second];
            root = product;                   // الجذر بطريقة نيوتن
            for (step = 0; step < 60; step++)
                root = (root + product / root) / 2;
            printf(" %6.3f", cov[first][second] / root);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
 1.000  0.981  0.990
 0.981  1.000  0.976
 0.990  0.976  1.000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.19 مصفوفة المسافات بين الزبائن

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب **مصفوفة المسافات** `dist[5][5]` (مربع المسافة الإقليدية) بين 5 زبائن ممثلين بخاصيتين، مستغلًا **التناظر** `dist[a][b] = dist[b][a]` لحساب نصف الخانات فقط، ثم يجد أقرب زوج.

خطوات الحل

1) القطر أصفار.
2) لكل first < second: نحسب المسافة ونكتبها في الخانتين.
3) نحتفظ بأصغر مسافة.

البرنامج بلغة C

```c
#include <stdio.h>

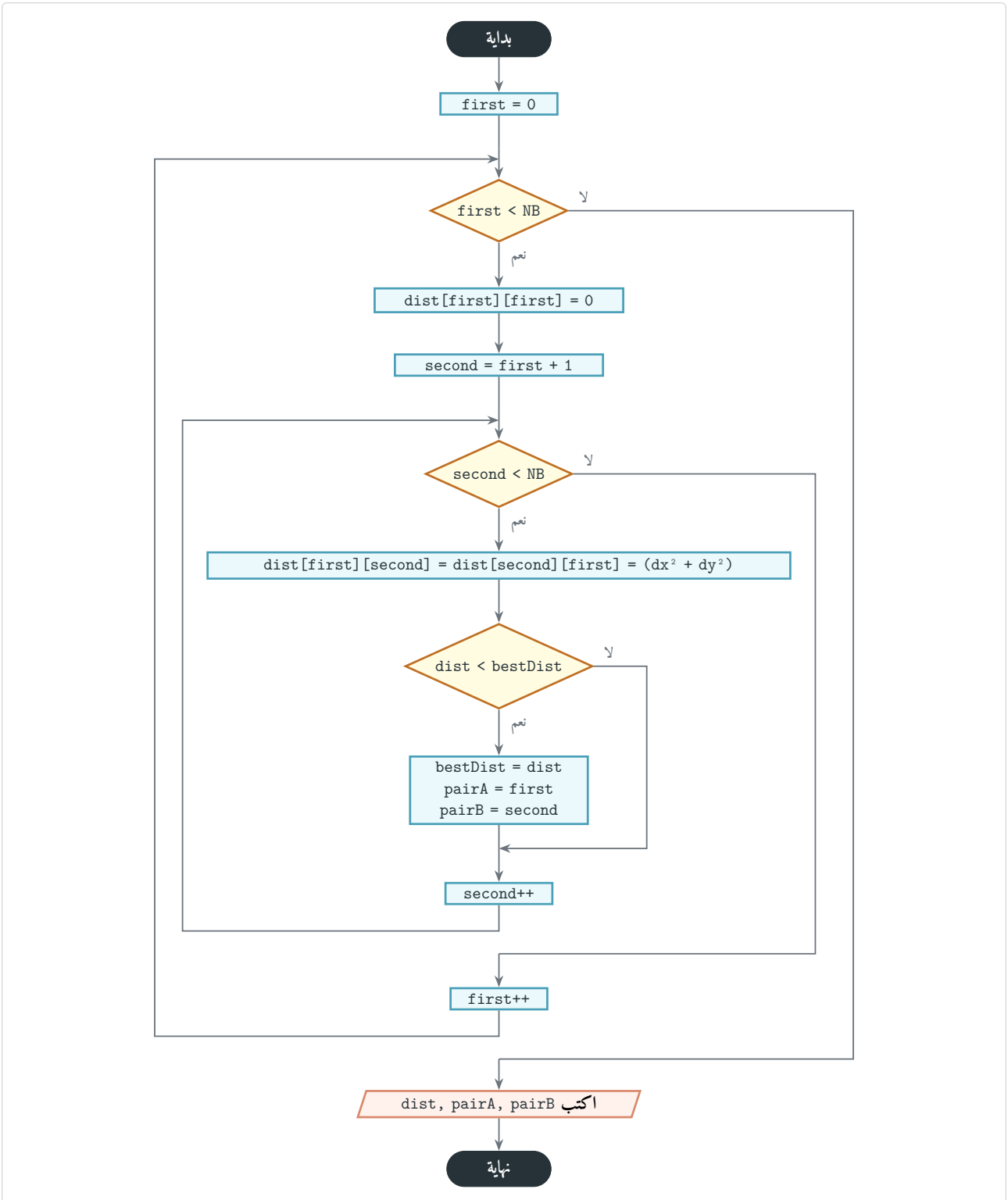
#define NB 5

int main(void) {
    double points[NB][2] = {{2, 1.5}, {8, 6}, {3, 2}, {9, 7.5}, {5, 3}};
    double dist[NB][NB], dx, dy, bestDist = -1;
    int first, second, pairA = 0, pairB = 0;
    for (first = 0; first < NB; first++) {
        dist[first][first] = 0;
        for (second = first + 1; second < NB; second++) {   // النصف العلوي فقط
            dx = points[first][0] - points[second][0];
            dy = points[first][1] - points[second][1];
            dist[first][second] = dist[second][first] = dx * dx + dy * dy;
            if (bestDist < 0 || dist[first][second] < bestDist) {
                bestDist = dist[first][second];
                pairA = first;
                pairB = second;
            }
        }
    }
    for (first = 0; first < NB; first++) {
        for (second = 0; second < NB; second++)
            printf("%7.2f", dist[first][second]);
        printf("\n");
    }
    printf("Closest customers: %d and %d\n", pairA, pairB);
    return 0;
}
```

نتيجة التنفيذ

```
   0.00  56.25   1.25  85.00  11.25
  56.25   0.00  41.00   3.25  18.00
   1.25  41.00   0.00  66.25   5.00
  85.00   3.25  66.25   0.00  36.25
  11.25  18.00   5.00  36.25   0.00
Closest customers: 0 and 2
```

المخطط الإنسيابي (Organigramme)



تمرين 5.20 مصفوفة الإلتباس والدقة

علم البيانات ★★ متوسط

نموذج يصنف التعليقات إلى 3 أصناف (0 سلبي، 1 محايد، 2 إيجابي). لدينا جدولا القيم الحقيقية والمتنبأ بها لـ 15 تعليقًا. اكتب برنامجًا يبني **مصفوفة الالتباس** `confusion[actual][predicted]` ويحسب **الدقة** (مجموع القطر / الكل).

خطوات الحل

1) نهيئ المصفوفة بأصفار.
2) لكل تعليق:

```
confusion[actual[index]][predicted[index]]++
```

.
3) الدقة = الأثر / عدد التعليقات.

البرنامج بلغة C

```c
#include <stdio.h>

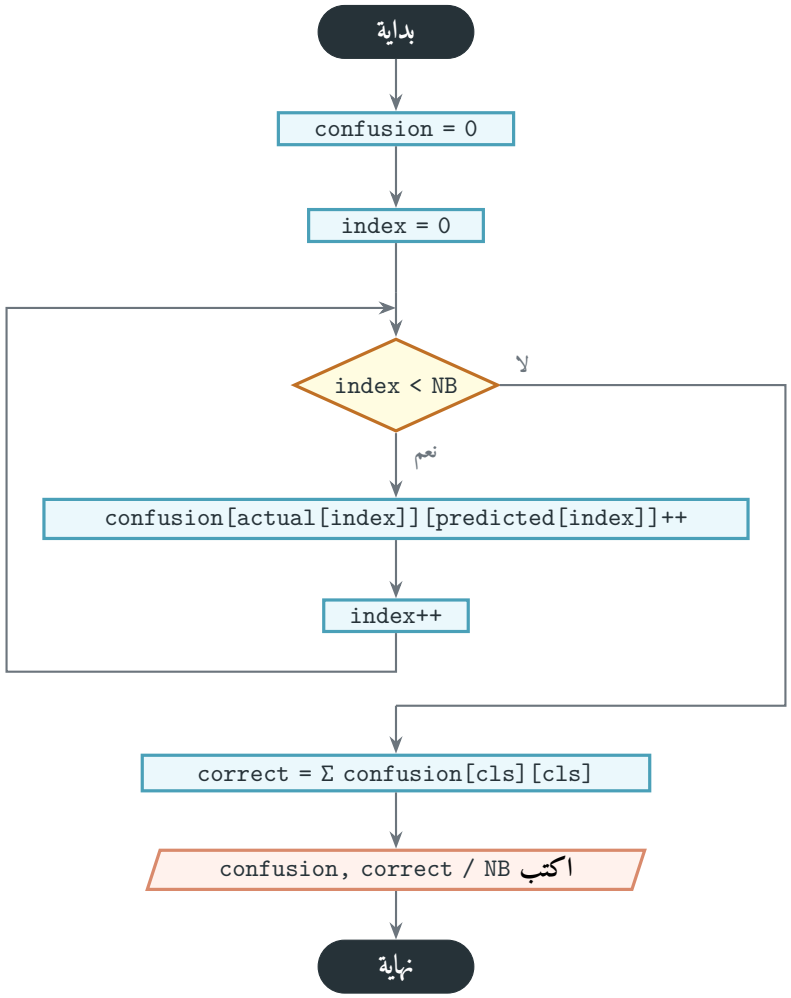
#define NB 15
#define NB_CLASSES 3

int main(void) {
    int actual[NB]    = {2, 0, 1, 2, 2, 0, 1, 1, 2, 0, 2, 1, 0, 2, 1};
    int predicted[NB] = {2, 0, 2, 2, 1, 0, 1, 0, 2, 0, 2, 1, 1, 2, 1};
    int confusion[NB_CLASSES][NB_CLASSES] = {{0}};
    int index, row, col, correct = 0;
    for (index = 0; index < NB; index++)
        confusion[actual[index]][predicted[index]]++;   // الفهرسان من الجدولين
    printf("actual\\pred  neg neu pos\n");
    for (row = 0; row < NB_CLASSES; row++) {
        printf("%11d", row);
        for (col = 0; col < NB_CLASSES; col++)
            printf(" %3d", confusion[row][col]);
        printf("\n");
        correct += confusion[row][row];   // القطر = التنبؤات الصحيحة
    }
    printf("Accuracy = %d/%d = %.2f\n", correct, NB, (double)correct / NB);
    return 0;
}
```

نتيجة التنفيذ

```
actual\pred  neg neu pos
          0   3   1   0
          1   1   3   1
          2   0   1   5
Accuracy = 11/15 = 0.73
```

المخطط الإنسيابي (Organigramme)



تمرين 5.21 الدقة والإستدعاء لكل صنف

علم البيانات ★★ متوسط

انطلاقًا من مصفوفة التباس جاهزة (3 أصناف)، اكتب برنامجًا يحسب لكل صنف **الدقة** `precision = TP /` والاستدعاء `recall = TP /` و**F1** `= 2PR / (P + R)`.

خطوات الحل

1) `TP = confusion[cls][cls]`.
2) مجموع العمود `cls`: كل التنبؤات بهذا الصنف.
3) مجموع السطر `cls`: كل العناصر التي صنفها الحقيقي `cls`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_CLASSES 3

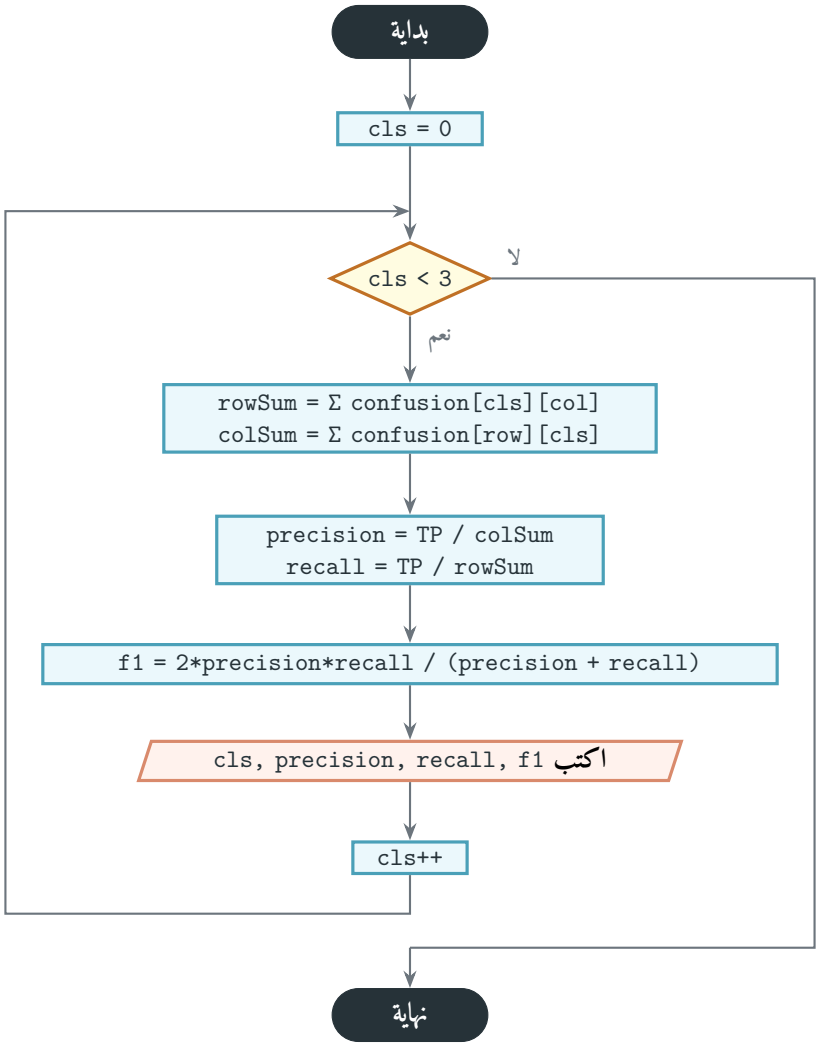
int main(void) {
    int confusion[NB_CLASSES][NB_CLASSES] = {{50,  8,  2},
                                             { 6, 30, 14},
```

```
                                                { 1,  9, 80}};
    int cls, other, rowSum, colSum, truePos;
    double precision, recall, f1;
    for (cls = 0; cls < NB_CLASSES; cls++) {
        rowSum = colSum = 0;
        for (other = 0; other < NB_CLASSES; other++) {
            rowSum += confusion[cls][other];  // cls = الحقيقي
            colSum += confusion[other][cls];  // cls = المتنبأ
        }
        truePos = confusion[cls][cls];
        precision = (double)truePos / colSum;
        recall = (double)truePos / rowSum;
        f1 = 2 * precision * recall / (precision + recall);
        printf("Class %d: precision %.3f, recall %.3f, F1 %.3f\n",
               cls, precision, recall, f1);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Class 0: precision 0.877, recall 0.833, F1 0.855
Class 1: precision 0.638, recall 0.600, F1 0.619
Class 2: precision 0.833, recall 0.889, F1 0.860
```

المخطط الإنسيابي (Organigramme)



تمرين 5.22 تناظر مصفوفة الشراء المشترك

علم البيانات ★★ متوسط

`together[a][b]` عدد المرات التي اشتُري فيها المنتجان `a` و`b` معًا؛ يجب أن تكون المصفوفة **متناظرة**. اكتب برنامجًا يتحقق من ذلك ويعرض الخانات غير المتطابقة (أخطاء تسجيل).

خطوات الحل

1) نقارن فقط النصف العلوي `row < col` بالنصف السفلي.
2) كل اختلاف خطأ نعرضه.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

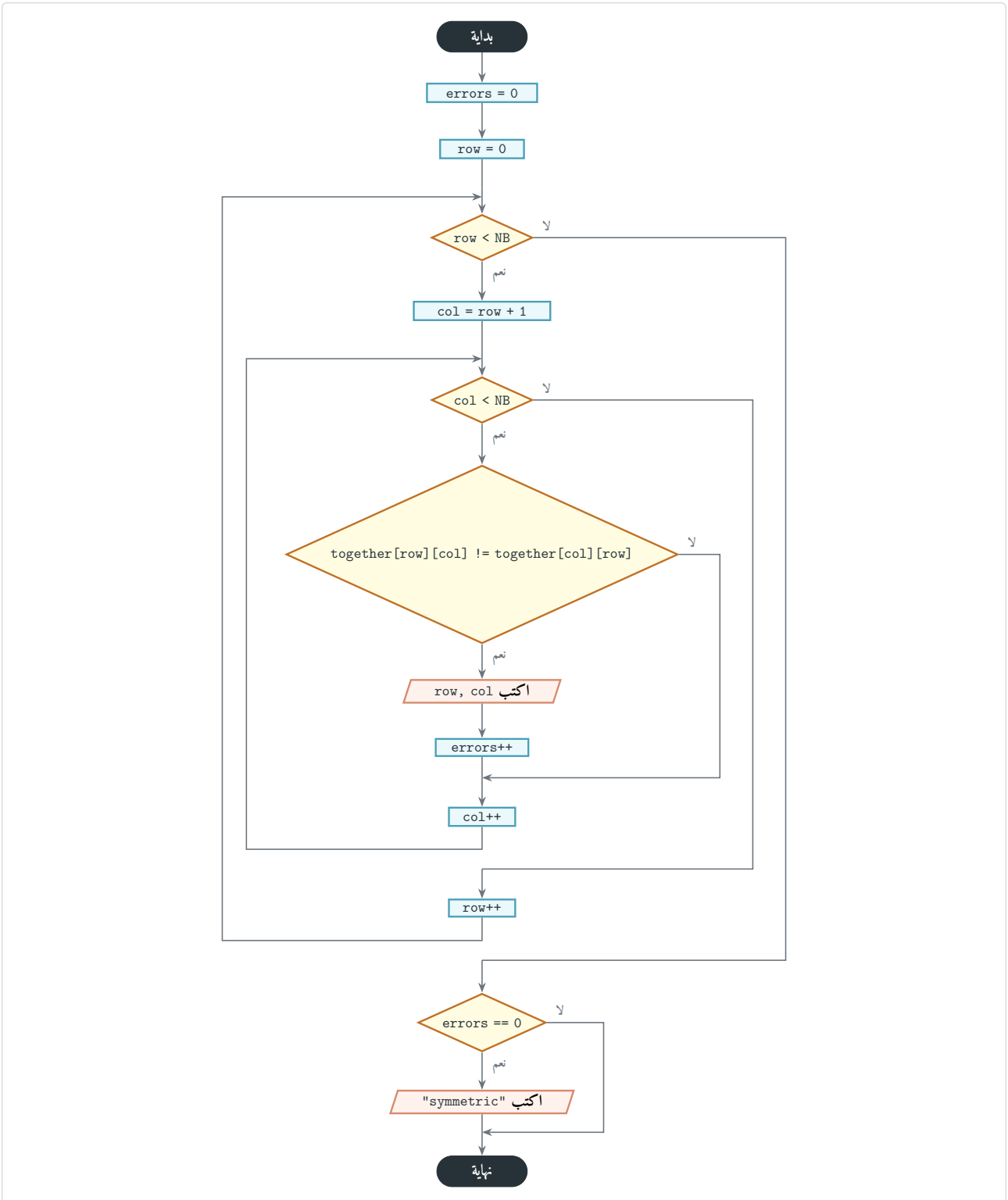
int main(void) {
    int together[NB][NB] = {{ 0, 12,  5,  3},
                            {12,  0,  7,  9},
```

```
                            { 5,  8,  0,  2},
                            { 3,  9,  2,  0}};
    int row, col, errors = 0;
    for (row = 0; row < NB; row++)
        for (col = row + 1; col < NB; col++)      // نصف المصفوفة يكفي
            if (together[row][col] != together[col][row]) {
                printf("Mismatch (%d,%d): %d vs %d\n", row, col,
                       together[row][col], together[col][row]);
                errors++;
            }
    if (errors == 0)
        printf("Matrix is symmetric\n");
    else
        printf("%d error(s) found\n", errors);
    return 0;
}
```

نتيجة التنفيذ

```
Mismatch (1,2): 7 vs 8
1 error(s) found
```

المخطط الإنسيابي (Organigramme)



تمرين 5.23 حساب الشراء المشترك من السلات

علم البيانات ★★ متوسط

مصفوفة `baskets[6][4]` تحتوي 1 إذا احتوت السلة (سطر) المنتج (عمود). اكتب برنامجًا يحسب مصفوفة **الشراء المشترك** `co = B × B`، حيث `co[a][b]` عدد السلات التي فيها `a` و`b` معًا، والقطر عدد السلات التي فيها كل منتج.

خطوات الحل

1) `co[first][second] = Σ baskets[basket][first] * baskets[basket][second]`

.

2) الجداء يساوي 1 فقط إذا كان المنتجان موجودين.

البرنامج بلغة C

```c
#include <stdio.h>

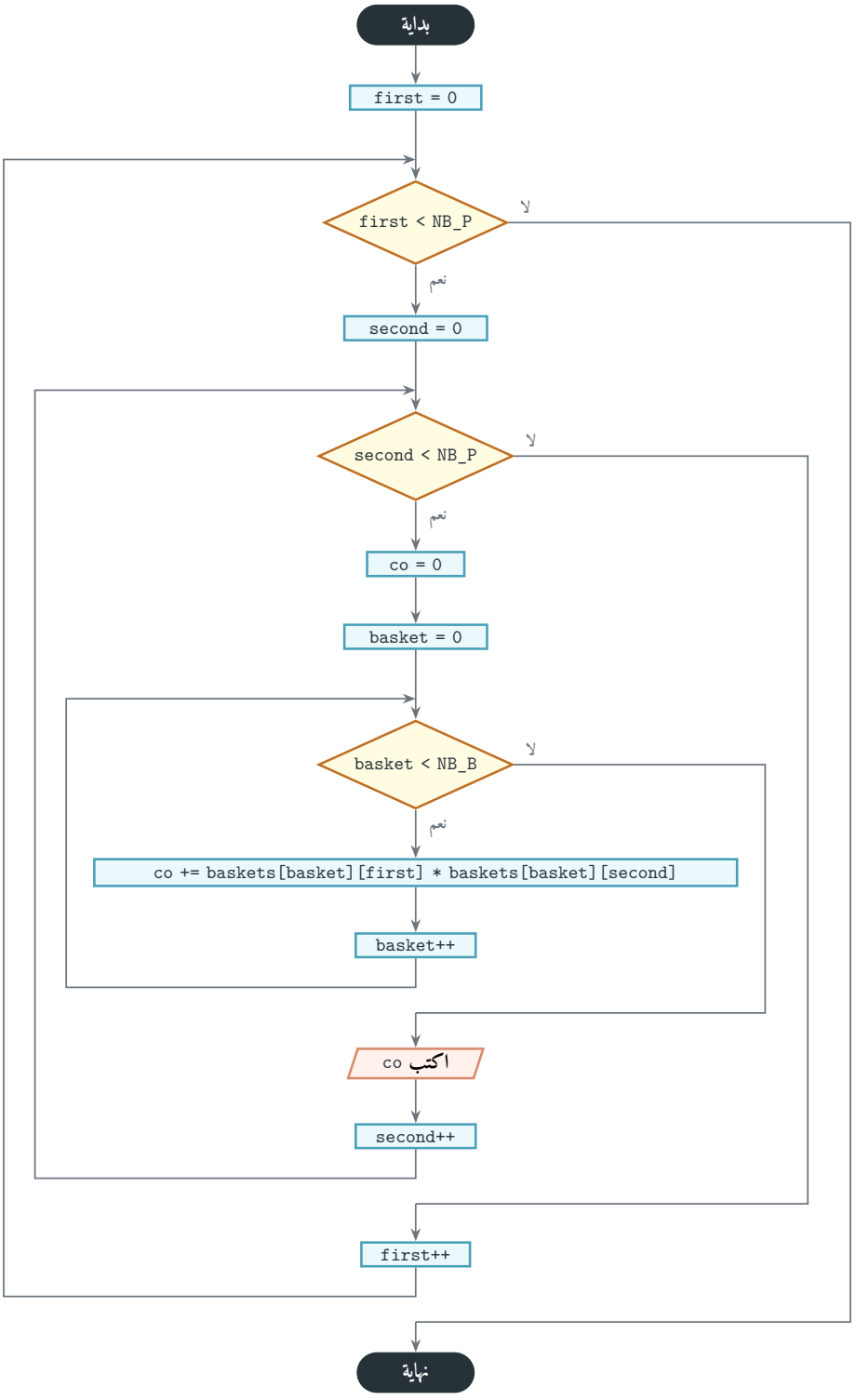
#define NB_B 6
#define NB_P 4

int main(void) {
    int baskets[NB_B][NB_P] = {{1, 1, 0, 0},
                               {1, 1, 1, 0},
                               {0, 1, 1, 0},
                               {1, 0, 0, 1},
                               {1, 1, 0, 1},
                               {0, 0, 1, 1}};
    int co[NB_P][NB_P], first, second, basket;
    for (first = 0; first < NB_P; first++) {
        for (second = 0; second < NB_P; second++) {
            co[first][second] = 0;
            for (basket = 0; basket < NB_B; basket++)   // عمود × عمود
                co[first][second] += baskets[basket][first] * baskets[basket][second];
            printf(" %d", co[first][second]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
4 3 1 2
3 4 2 1
1 2 3 1
2 1 1 3
```

المخطط الإنسيابي (Organigramme)



تمرين 5.24 أقرب رف فارغ

تجارة إلكترونية ★★ متوسط

المستودع شبكة رفوف `grid[5][6]` (1 مشغول، 0 فارغ). العامل في الموقع `(row, col)`. اكتب برنامجًا يجد **أقرب رف فارغ** بمسافة مانهاتن `|dr| + |dc|` (عدد الخطوات في الممرات).

خطوات الحل

1) نمر على كل الخانات الفارغة.
2) المسافة: القيمة المطلقة لفرق الأسطر + القيمة المطلقة لفرق الأعمدة.
3) نحتفظ بالأقرب.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int grid[5][6] = {{1, 1, 0, 1, 1, 1},
                      {1, 1, 1, 1, 1, 0},
                      {1, 1, 1, 1, 1, 1},
                      {0, 1, 1, 1, 1, 1},
                      {1, 1, 1, 1, 0, 1}};
    int myRow, myCol, row, col, dRow, dCol, dist;
    int best = -1, bestRow = -1, bestCol = -1;
    scanf("%d %d", &myRow, &myCol);
    for (row = 0; row < 5; row++)
        for (col = 0; col < 6; col++)
            if (grid[row][col] == 0) {
                dRow = row - myRow; if (dRow < 0) dRow = -dRow;
                dCol = col - myCol; if (dCol < 0) dCol = -dCol;
                dist = dRow + dCol;                 // مسافة مانهاتن
                if (best < 0 || dist < best) {
                    best = dist; bestRow = row; bestCol = col;
                }
            }
    printf("Nearest empty shelf: (%d, %d), %d steps\n", bestRow, bestCol, best);
    return 0;
}
```

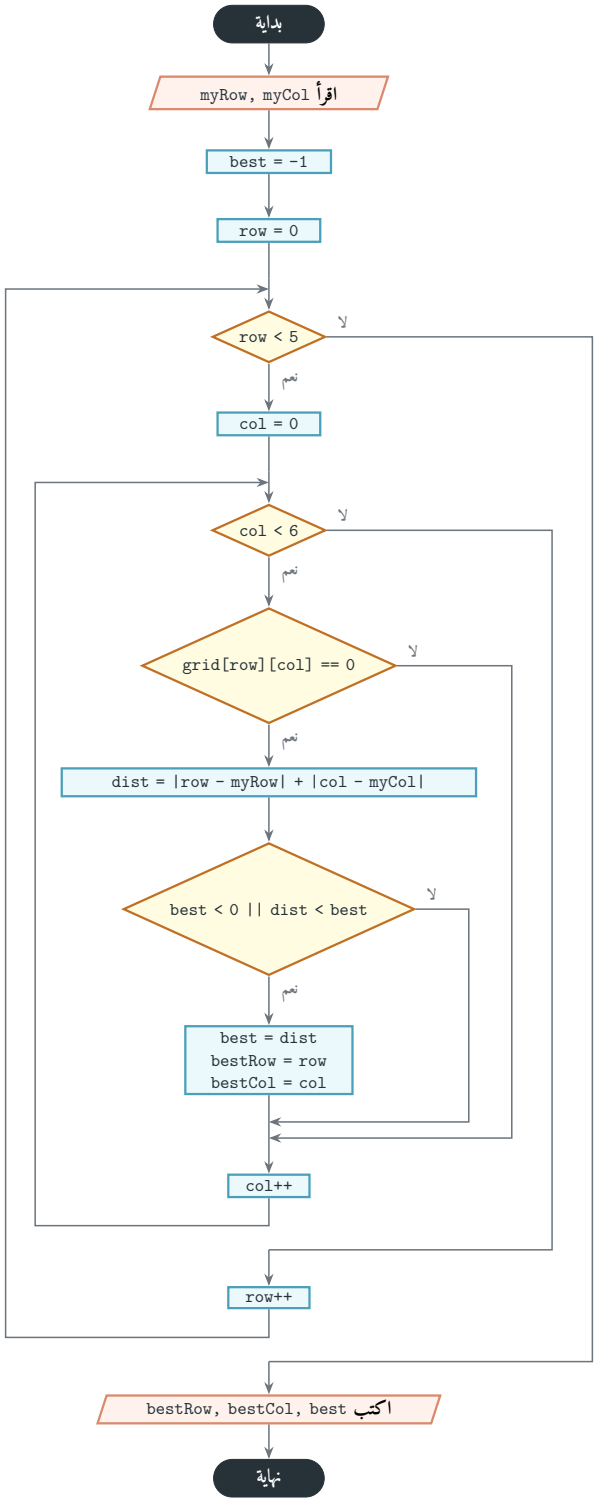
المدخلات

```
2 3
```

نتيجة التنفيذ

```
Nearest empty shelf: (0, 2), 3 steps
```

المخطط الإنسيابي (Organigramme)



تمرين 5.25 خريطة حرارية للطلبيات

علم البيانات ★★ متوسط

مصفوفة `orders[7][6]`: عدد الطلبيات لكل يوم من الأسبوع (أسطر) في 6 فترات من اليوم (أعمدة). اكتب برنامجًا يرسم **خريطة حرارية نصية**: كل خانة تُرسم برمز حسب شدتها . ثم - ثم + ثم #، بعد تحديد الأكبر.

خطوات الحل

1) نجد القيمة القصوى `peak`.
2) المستوى: `level = value * 4 / (peak + 1)` بين 0 و3.
3) `switch` على المستوى لاختيار الرمز.

البرنامج بلغة C

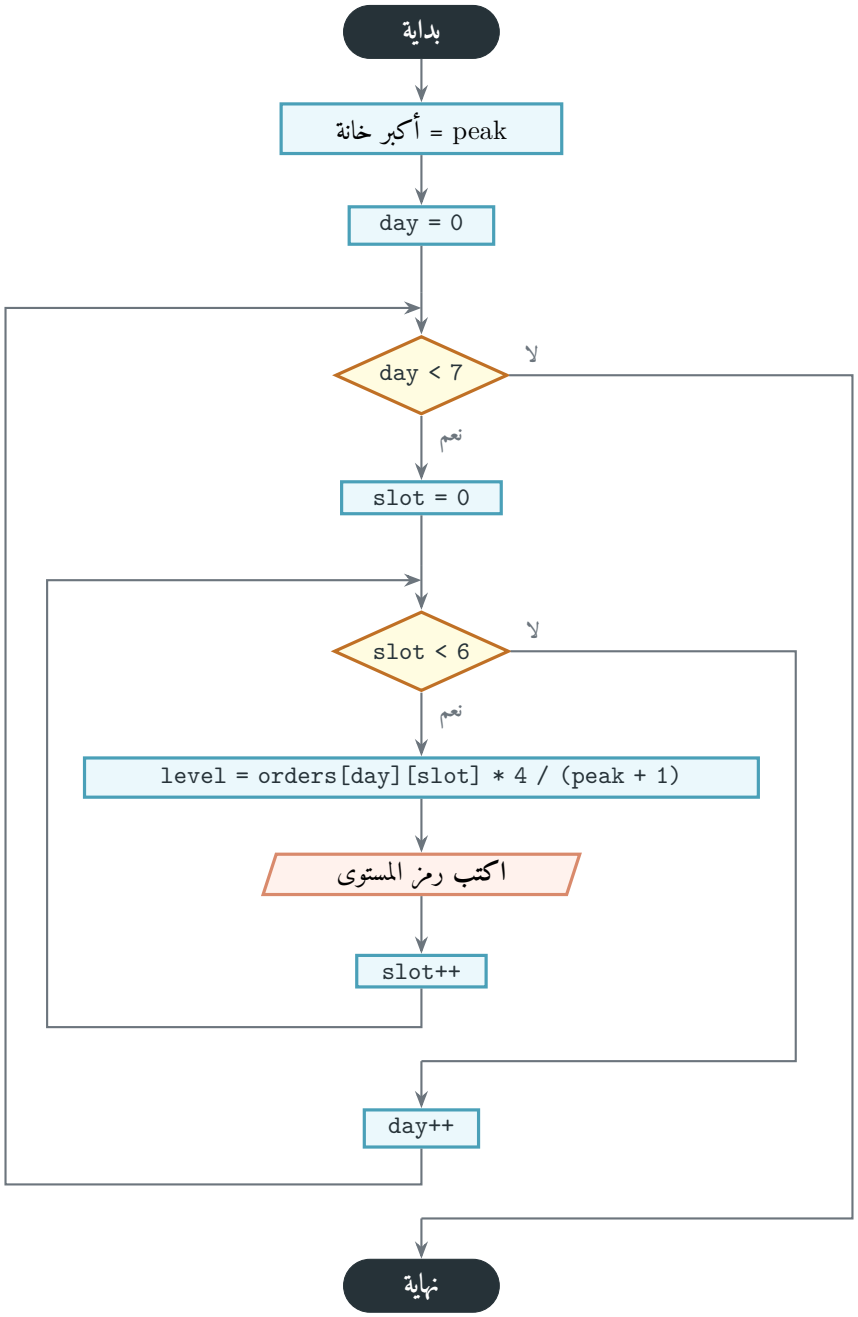
```c
#include <stdio.h>

int main(void) {
    int orders[7][6] = {{ 2,  5, 14, 10, 18, 25},
                        { 1,  6, 12,  9, 20, 22},
                        { 3,  4, 15, 11, 17, 28},
                        { 2,  7, 13, 12, 21, 30},
                        { 5,  9, 20, 16, 26, 38},
                        { 8, 15, 25, 22, 30, 41},
                        { 7, 12, 22, 18, 24, 29}};
    int day, slot, level, peak = 0;
    char symbol;
    for (day = 0; day < 7; day++)
        for (slot = 0; slot < 6; slot++)
            if (orders[day][slot] > peak)
                peak = orders[day][slot];
    printf("     0-4 4-8 8-12 12-16 16-20 20-24\n");
    for (day = 0; day < 7; day++) {
        printf("Day %d ", day + 1);
        for (slot = 0; slot < 6; slot++) {
            level = orders[day][slot] * 4 / (peak + 1);   // 0..3
            switch (level) {
                case 0:  symbol = '.'; break;
                case 1:  symbol = '-'; break;
                case 2:  symbol = '+'; break;
                default: symbol = '#';
            }
            printf("  %c  ", symbol);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
     0-4 4-8 8-12 12-16 16-20 20-24
Day 1   .    .    -    .    -    +
Day 2   .    .    -    .    -    +
Day 3   .    .    -    -    -    +
Day 4   .    .    -    -    +    +
Day 5   .    .    -    -    +    #
Day 6   .    -    +    +    +    #
Day 7   .    -    +    -    +    +
```

المخطط الإنسيابي (Organigramme)



تمرين 5.26 جدول محوري من السجلات

علم البيانات ★★ متوسط

لدينا سجلات بيع: لكل عملية (الصنف 0..2، الشهر 0..3، المبلغ). اكتب برنامجًا يبني **جدولًا محوريًا** `pivot[3][4]` يجمع المبالغ حسب الصنف والشهر، مع مجاميع الأسطر والأعمدة.

خطوات الحل

1) نهيئ المصفوفة بأصفار.
2) لكل سجل:

```
pivot[category[rec]][month[rec]] += amount[rec]
```

.

3) نعرض الجدول مع مجموع كل سطر، ثم سطر مجاميع الأعمدة.

البرنامج بلغة C

```c
#include <stdio.h>

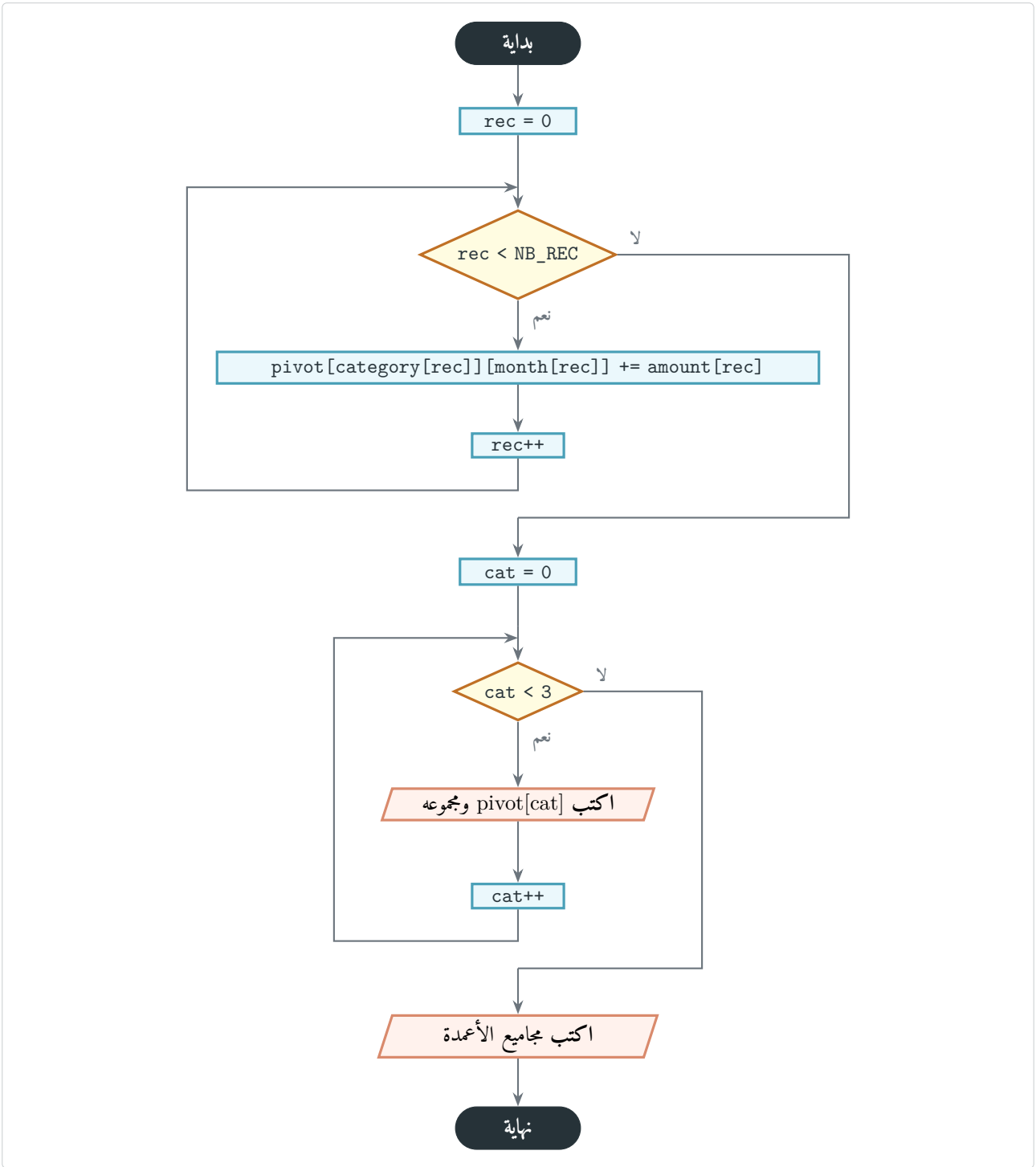
#define NB_REC 12

int main(void) {
    int category[NB_REC] = {0, 1, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2};
    int month[NB_REC]    = {0, 0, 1, 1, 2, 0, 3, 3, 0, 2, 3, 2};
    int amount[NB_REC]   = {500, 300, 450, 900, 120, 220, 610, 400, 350, 200, 180, 700};
    int pivot[3][4] = {{0}}, colTotal[4] = {0};
    int rec, cat, mon, rowTotal;
    for (rec = 0; rec < NB_REC; rec++)
        pivot[category[rec]][month[rec]] += amount[rec];   // التجميع
    printf("cat |  M1   M2   M3   M4 | total\n");
    for (cat = 0; cat < 3; cat++) {
        rowTotal = 0;
        printf("%3d |", cat);
        for (mon = 0; mon < 4; mon++) {
            printf(" %4d", pivot[cat][mon]);
            rowTotal += pivot[cat][mon];
            colTotal[mon] += pivot[cat][mon];
        }
        printf(" | %5d\n", rowTotal);
    }
    printf("sum |");
    for (mon = 0; mon < 4; mon++)
        printf(" %4d", colTotal[mon]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
cat |  M1   M2   M3   M4 | total
  0 |  500  450  200  610 |  1760
  1 |  520    0  120  180 |   820
  2 |  350  900  700  400 |  2350
sum | 1370 1350 1020 1190
```

المخطط الإنسيابي (Organigramme)



تمرين 5.27 مجاميع المستطيلات بالمجاميع التراكمية الثنائية

علم البيانات ★★ متوسط

للإجابة بسرعة عن "مجموع المبيعات للمتاجر من `r1` إلى `r2` في الأيام من `c1` إلى `c2`"، نبني مصفوفة **مجاميع تراكمية ثنائية** `prefix[row+1][col+1]` = مجموع المستطيل من `(0,0)` إلى `(row,col)`. كل استعلام يصبح 4 عمليات.

خطوات الحل

1) `prefix[r+1][c+1] = sales[r][c] + prefix[r][c+1] + prefix[r+1][c] - prefix[r][c]`
.

2) المستطيل:

```
prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]
```

.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int sales[3][4] = {{120, 95, 140, 180},
                       { 80, 60,  75,  90},
                       {200, 210, 190, 250}};
    int prefix[4][5] = {{0}};            // سطر وعمود إضافيان من الأصفار
    int row, col, r1, r2, c1, c2;
    for (row = 0; row < 3; row++)
        for (col = 0; col < 4; col++)
            // نطرح التداخل الذي جُمع مرتين
            prefix[row + 1][col + 1] = sales[row][col] + prefix[row][col + 1]
                                     + prefix[row + 1][col] - prefix[row][col];
    scanf("%d %d %d %d", &r1, &r2, &c1, &c2);
    printf("Stores %d-%d, days %d-%d: %d\n", r1, r2, c1 + 1, c2 + 1,
           prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1]
           - prefix[r2 + 1][c1] + prefix[r1][c1]);
    return 0;
}
```

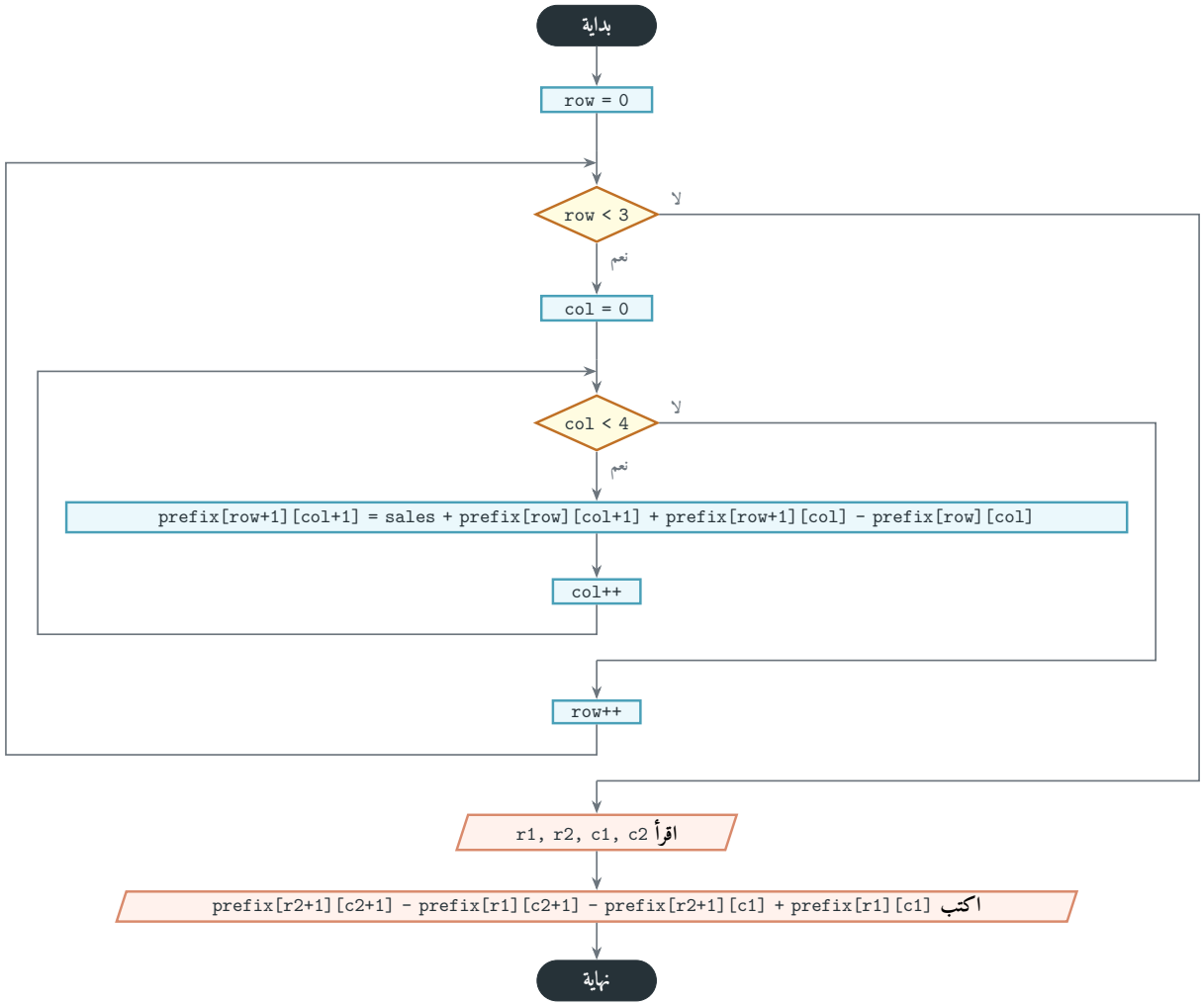
المدخلات

```
1 2 1 3
```

نتيجة التنفيذ

```
Stores 1-2, days 2-4: 875
```

المخطط الإنسيابي (Organigramme)



تمرين 5.28 تدوير صورة المنتج

تجارة إلكترونية

متوسط

صورة منتج صغيرة مخزنة كمصفوفة بكسلات `image[4][6]` (درجات رمادي). اكتب برنامجًا يديرها **90°** مع عقارب الساعة في مصفوفة جديدة `rotated[6][4]`.

خطوات الحل

1) البكسل `(row, col)` ينتقل إلى `(col, HEIGHT - 1 - row)`.
2) أبعاد الصورة تتبادل.

البرنامج بلغة C

```c
#include <stdio.h>

#define HEIGHT 4
#define WIDTH 6

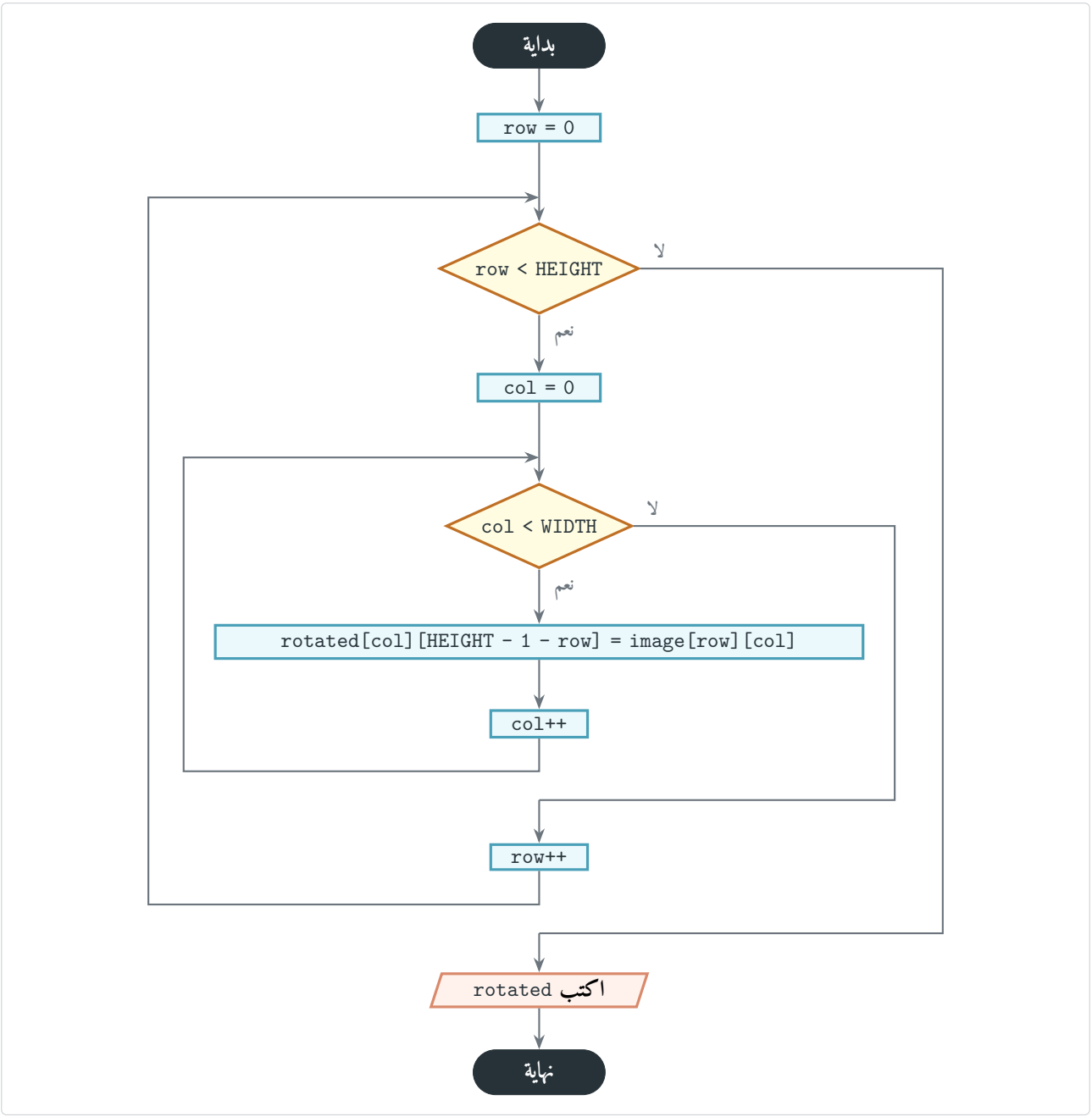
int main(void) {
    int image[HEIGHT][WIDTH] = {{ 1,  2,  3,  4,  5,  6},
                                { 7,  8,  9, 10, 11, 12},
```

```
                                {13, 14, 15, 16, 17, 18},
                                {19, 20, 21, 22, 23, 24}};
    int rotated[WIDTH][HEIGHT], row, col;
    for (row = 0; row < HEIGHT; row++)
        for (col = 0; col < WIDTH; col++)
            // السطر الأول يصبح العمود الأخير
            rotated[col][HEIGHT - 1 - row] = image[row][col];
    for (row = 0; row < WIDTH; row++) {
        for (col = 0; col < HEIGHT; col++)
            printf("%3d", rotated[row][col]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
 19 13  7  1
 20 14  8  2
 21 15  9  3
 22 16 10  4
 23 17 11  5
 24 18 12  6
```

المخطط الإنسيابي (Organigramme)



تمرين 5.29 تصغير صورة إلى مصغّرة

تجارة إلكترونية ★★ متوسط

لعرض مصغّرات المنتجات، اكتب برنامجًا يصغر صورة `image[6][8]` إلى النصف: كل بكسل في المصغرة متوسط مربع 2×2 من الصورة الأصلية.

خطوات الحل

1) المصغرة أبعادها 3×4.
2) `thumb[row][col]` = متوسط البكسلات `(2row, 2col)`، `(2row, 2col+1)`، `(2row+1, 2col)`، `(2row+1, 2col+1)`.

البرنامج بلغة C

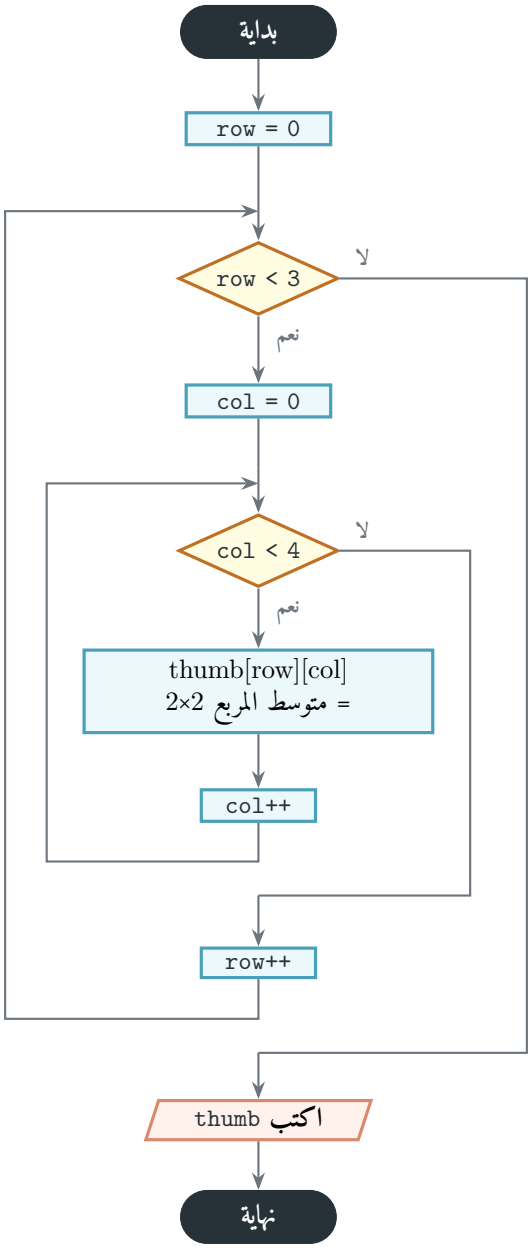
```c
#include <stdio.h>

int main(void) {
    int image[6][8] = {{ 10,  12, 200, 210,  50,  52, 90,  94},
                       { 14,  16, 205, 215,  54,  56, 92,  96},
                       {100, 104,  30,  34, 250, 246, 10,  12},
                       {102, 106,  32,  36, 248, 252, 14,  16},
                       { 60,  60,  60,  60, 120, 124, 180, 184},
                       { 60,  60,  60,  60, 128, 132, 188, 192}};
    int thumb[3][4], row, col;
    for (row = 0; row < 3; row++) {
        for (col = 0; col < 4; col++) {
            thumb[row][col] = (image[2 * row][2 * col] + image[2 * row][2 * col + 1]
                            + image[2 * row + 1][2 * col] + image[2 * row + 1][2 * col + 1]) / 4;
            printf("%4d", thumb[row][col]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
  13 207  53  93
 103  33 249  13
  60  60 126 186
```

المخطط الإنسيابي (Organigramme)



تمرين 5.30 تنعيم صورة بمرشح المتوسط

علم البيانات ★★ متوسط

اكتب برنامجًا يطبق **مرشح المتوسط 3×3** على صورة `image[5][5]` لإزالة الضجيج: كل بكسل داخلي يصبح متوسط جيرانه التسعة، بكسلات الحافة تبقى كما هي.

خطوات الحل

1) نعمل على مصفوفة جديدة `blurred` لكي لا نستعمل قيمًا معدلة.
2) لكل بكسل داخلي: حلقتان صغيرتان على `dRow` و`dCol` من -1 إلى 1.

البرنامج بلغة C

```c
#include <stdio.h>

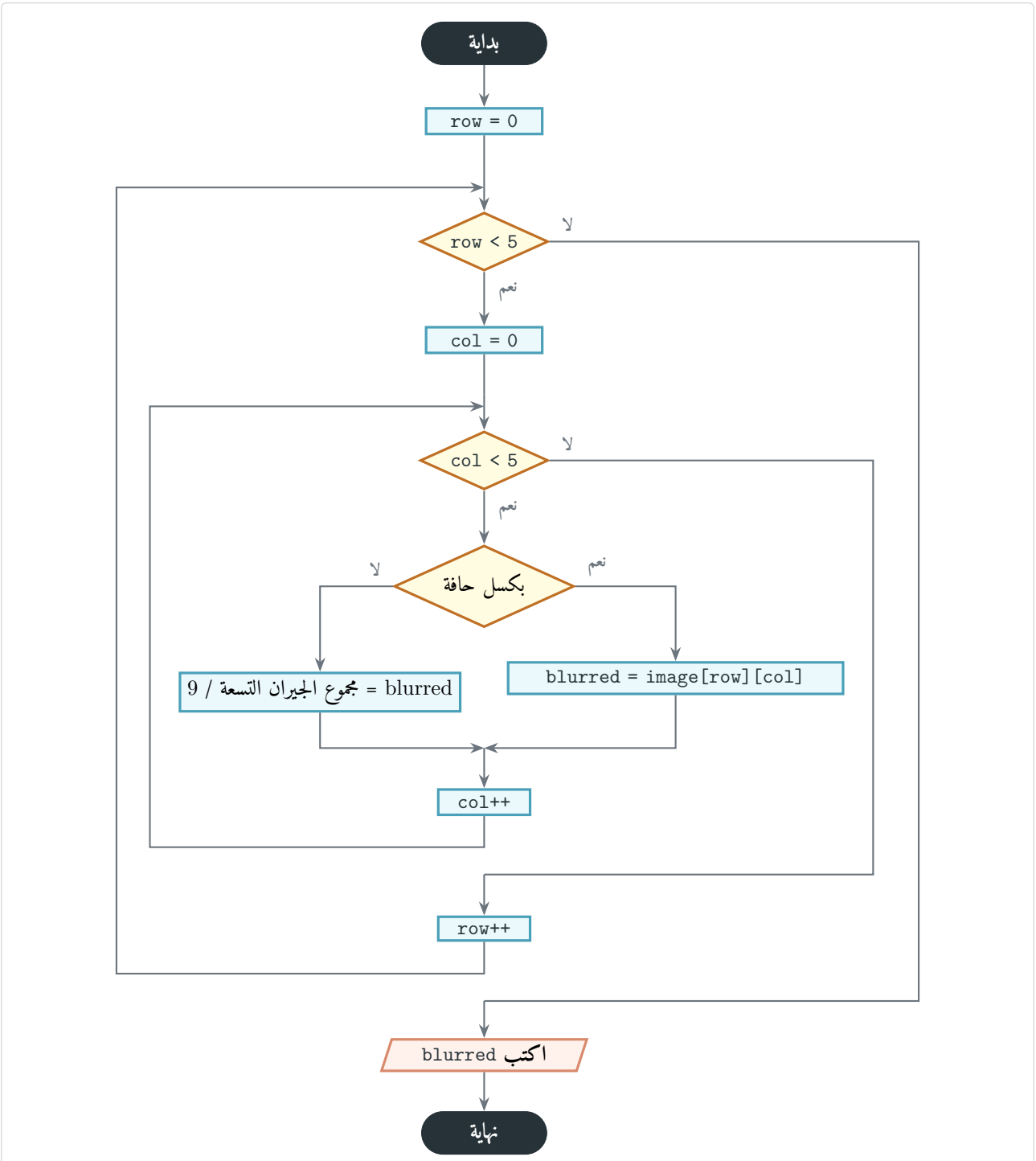
#define SIZE 5

int main(void) {
    int image[SIZE][SIZE] = {{10, 10,  10, 10, 10},
                             {10, 10, 255, 10, 10},
                             {10, 10,  10, 10, 10},
                             {10, 10,  10,  0, 10},
                             {10, 10,  10, 10, 10}};
    int blurred[SIZE][SIZE], row, col, dRow, dCol, sum;
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++) {
            if (row == 0 || col == 0 || row == SIZE - 1 || col == SIZE - 1) {
                blurred[row][col] = image[row][col];   // الحافة دون تغيير
            } else {
                sum = 0;
                for (dRow = -1; dRow <= 1; dRow++)     // الجوار 3×3
                    for (dCol = -1; dCol <= 1; dCol++)
                        sum += image[row + dRow][col + dCol];
                blurred[row][col] = sum / 9;
            }
            printf("%4d", blurred[row][col]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
  10  10  10  10  10
  10  37  37  37  10
  10  37  36  36  10
  10  10   8   8  10
  10  10  10  10  10
```

المخطط الإنسيابي (Organigramme)



تمرين 5.31 سلسلة ماركوف لحالات الزبائن

علم البيانات ★★ متوسط

للزبون 3 حالات: 0 جديد، 1 نشط، 2 خامل. مصفوفة الانتقال `transition[from][to]` تعطي احتمال الانتقال شهريًا. انطلاقًا من توزيع ابتدائي (1000 جديد)، اكتب برنامجًا يحسب **توزيع الزبائن** بعد كل شهر لمدة `months` أشهر.

خطوات الحل

1) `next[to] = Σ current[from] * transition[from][to]` (جداء شعاع في مصفوفة).
2) نسخ `next` إلى `current` بعد كل شهر.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_STATES 3

int main(void) {
    double transition[NB_STATES][NB_STATES] = {{0.0, 0.6, 0.4},   // جديد
                                               {0.0, 0.8, 0.2},   // نشط
                                               {0.0, 0.3, 0.7}};  // خامل
    double current[NB_STATES] = {1000, 0, 0}, next[NB_STATES];
    int months, month, from, to;
    scanf("%d", &months);
    for (month = 1; month <= months; month++) {
        for (to = 0; to < NB_STATES; to++) {
            next[to] = 0;
            for (from = 0; from < NB_STATES; from++)
                next[to] += current[from] * transition[from][to];
        }
        for (to = 0; to < NB_STATES; to++)
            current[to] = next[to];        // الشهر التالي ينطلق من هنا
        printf("Month %d: new %.0f, active %.1f, idle %.1f\n",
               month, current[0], current[1], current[2]);
    }
    return 0;
}
```

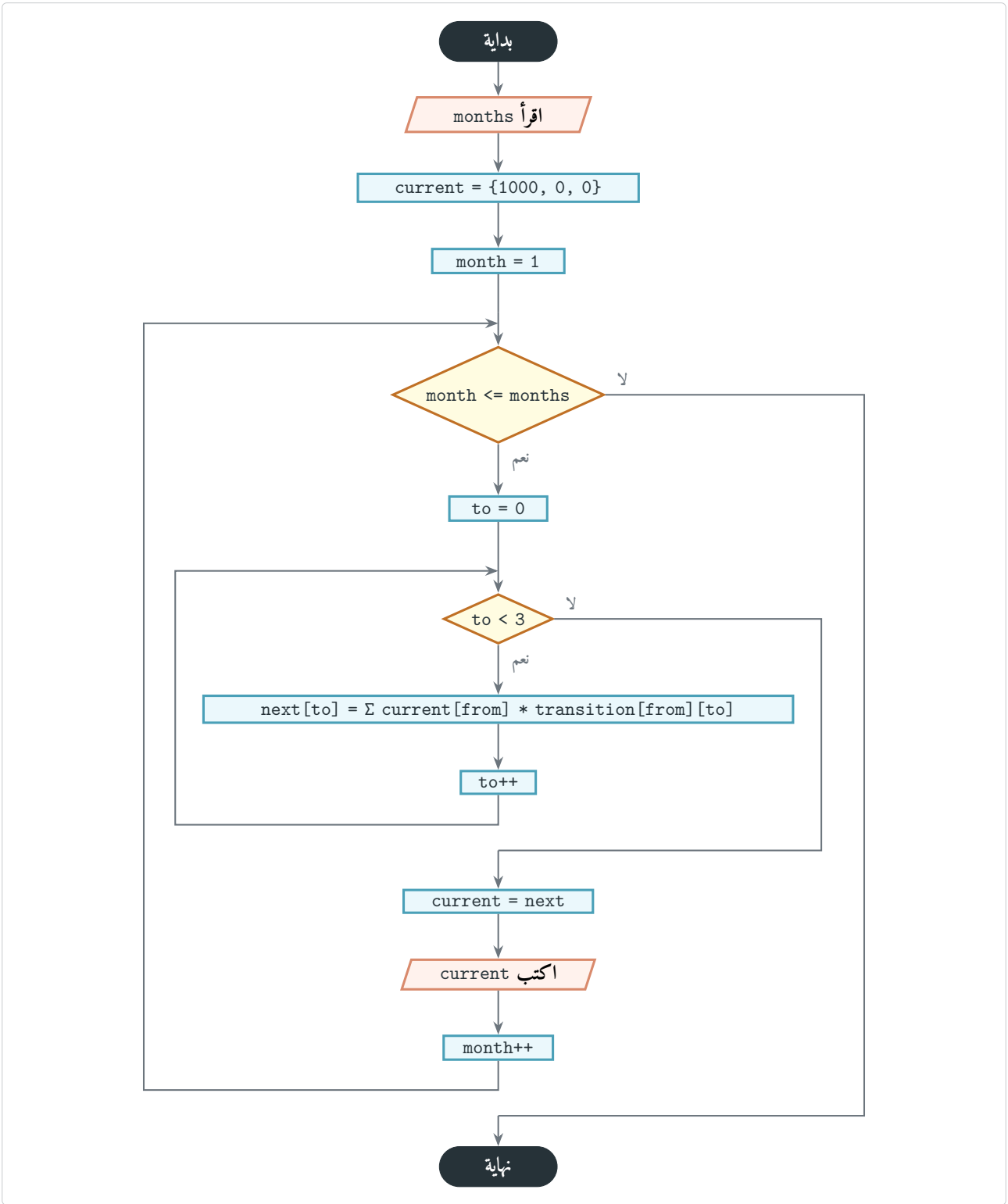
المدخلات

```
6
```

نتيجة التنفيذ

```
Month 1: new 0, active 600.0, idle 400.0
Month 2: new 0, active 600.0, idle 400.0
Month 3: new 0, active 600.0, idle 400.0
Month 4: new 0, active 600.0, idle 400.0
Month 5: new 0, active 600.0, idle 400.0
Month 6: new 0, active 600.0, idle 400.0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.32 جدول المسافات المثلثي

تجارة إلكترونية ★★ متوسط

جدول المسافات بين المدن متناظر، فتكفي مصفوفة مثلثية سفلية. اكتب برنامجًا يتحقق أن مصفوفة مقروءة مثلثية سفلية (كل الخانات فوق القطر أصفار)، ثم يستعملها لإيجاد المسافة بين مدينتين في أي ترتيب.

خطوات الحل

1) مثلثية سفلية: `matrix[row][col] == 0` لكل `col > row`.

2) المسافة بين a وb: نقرأ دائمًا `matrix[max][min]`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

int main(void) {
    int dist[NB][NB] = {{  0,   0,   0, 0},     // الجزائر
                        {430,   0,   0, 0},     // وهران
                        {390, 820,   0, 0},     // قسنطينة
                        {200, 600, 340, 0}};    // سطيف
    int row, col, lower = 1, cityA, cityB, tmp;
    for (row = 0; row < NB; row++)
        for (col = row + 1; col < NB; col++)    // فوق القطر
            if (dist[row][col] != 0)
                lower = 0;
    printf("Lower triangular: %s\n", lower ? "yes" : "no");
    scanf("%d %d", &cityA, &cityB);
    if (cityA < cityB) {                        // نقرأ دائمًا تحت القطر
        tmp = cityA; cityA = cityB; cityB = tmp;
    }
    printf("Distance = %d km\n", dist[cityA][cityB]);
    return 0;
}
```

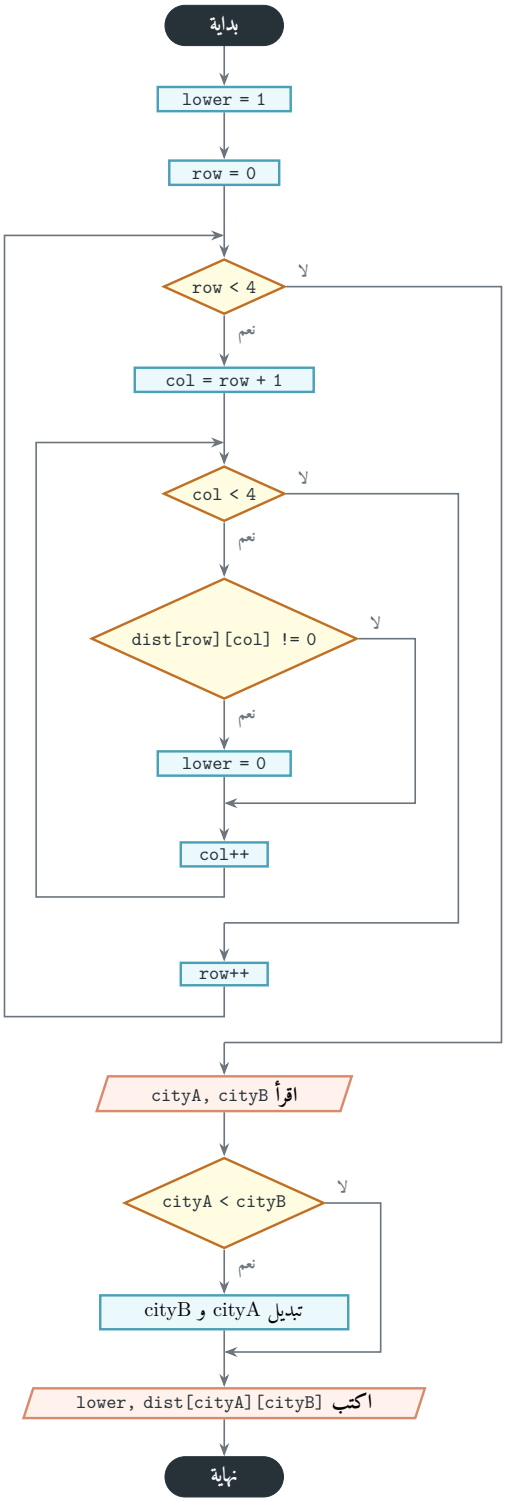
المدخلات

```
1 3
```

نتيجة التنفيذ

```
Lower triangular: yes
Distance = 600 km
```

المخطط الإنسيابي (Organigramme)



تمرين 5.33 المستودع الأرخص لكل زبون

تجارة إلكترونية ★★ متوسط

مصفوفة `shipCost[3][5]`: تكلفة الشحن من كل مستودع (سطر) إلى كل زبون (عمود). اكتب برنامجًا يختار لكل زبون **المستودع الأرخص**، ويحسب التكلفة الكلية وعدد الزبائن المخدومين من كل مستودع.

خطوات الحل

1) لكل عمود: بحث عن أصغر قيمة.
2) `served[bestDepot]++` و`total += shipCost[bestDepot][customer]`.

البرنامج بلغة C

```c
#include <stdio.h>

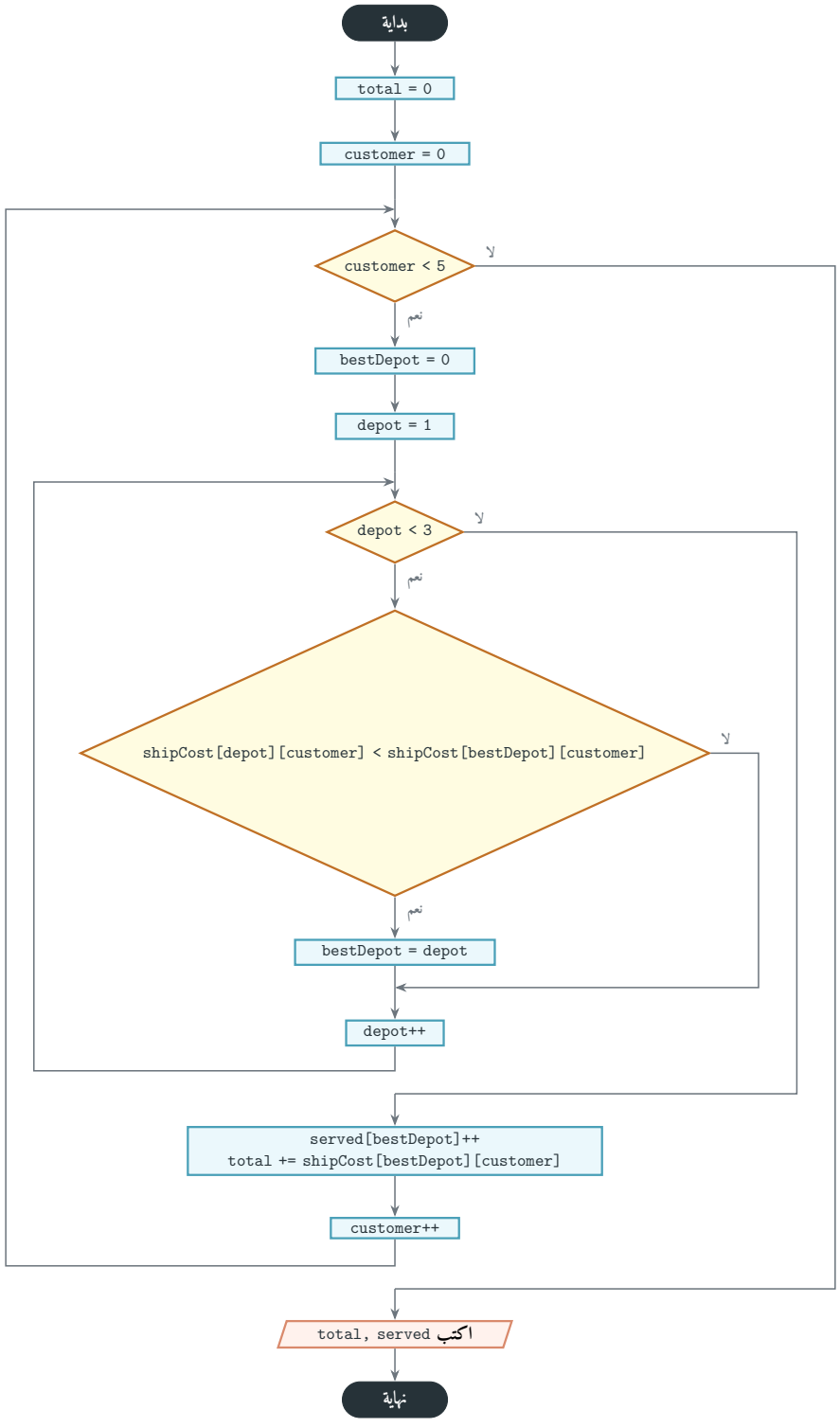
#define NB_DEPOTS 3
#define NB_CUST 5

int main(void) {
    int shipCost[NB_DEPOTS][NB_CUST] = {{400, 650, 300, 900, 500},
                                        {550, 300, 450, 600, 700},
                                        {700, 500, 350, 350, 450}};
    int served[NB_DEPOTS] = {0};
    int customer, depot, bestDepot, total = 0;
    for (customer = 0; customer < NB_CUST; customer++) {
        bestDepot = 0;
        for (depot = 1; depot < NB_DEPOTS; depot++)
            if (shipCost[depot][customer] < shipCost[bestDepot][customer])
                bestDepot = depot;
        served[bestDepot]++;
        total += shipCost[bestDepot][customer];
        printf("Customer %d <- depot %d (%d DA)\n", customer, bestDepot,
               shipCost[bestDepot][customer]);
    }
    printf("Total cost = %d DA\n", total);
    for (depot = 0; depot < NB_DEPOTS; depot++)
        printf("Depot %d serves %d customer(s)\n", depot, served[depot]);
    return 0;
}
```

نتيجة التنفيذ

```
Customer 0 <- depot 0 (400 DA)
Customer 1 <- depot 1 (300 DA)
Customer 2 <- depot 0 (300 DA)
Customer 3 <- depot 2 (350 DA)
Customer 4 <- depot 2 (450 DA)
Total cost = 1800 DA
Depot 0 serves 2 customer(s)
Depot 1 serves 1 customer(s)
Depot 2 serves 2 customer(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.34 كثافة مصفوفة متفرقة

علم البيانات ★★ متوسط

مصفوفة زبائن × منتجات أغلب خاناتها أصفار (**متفرقة**). لتوفير الذاكرة نخزن فقط الخانات غير الصفرية بصيغة **الإحداثيات** `(row, col, value)` في ثلاثة جداول. اكتب برنامجًا يحسب الكثافة وينشئ هذا التمثيل.

خطوات الحل

1) نمر على المصفوفة، كل خانة غير صفرية تضاف إلى الجداول الثلاثة.
2) الكثافة = عدد الخانات غير الصفرية / عدد الخانات.

البرنامج بلغة C

```c
#include <stdio.h>

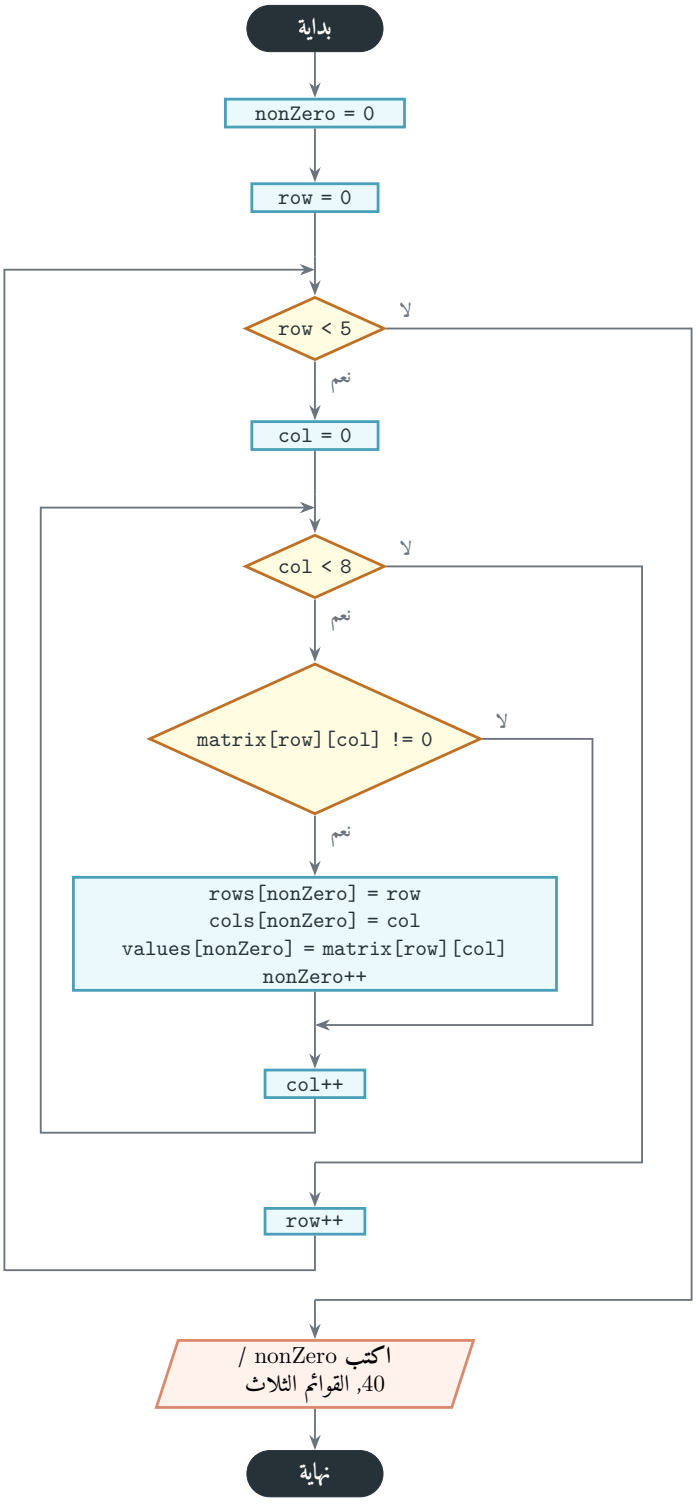
#define NB_ROWS 5
#define NB_COLS 8

int main(void) {
    int matrix[NB_ROWS][NB_COLS] = {{0, 0, 3, 0, 0, 0, 0, 1},
                                    {0, 0, 0, 0, 0, 0, 0, 0},
                                    {2, 0, 0, 0, 5, 0, 0, 0},
                                    {0, 0, 0, 0, 0, 0, 4, 0},
                                    {0, 1, 0, 0, 0, 0, 0, 0}};
    int rows[NB_ROWS * NB_COLS], cols[NB_ROWS * NB_COLS], values[NB_ROWS * NB_COLS];
    int row, col, nonZero = 0, index;
    for (row = 0; row < NB_ROWS; row++)
        for (col = 0; col < NB_COLS; col++)
            if (matrix[row][col] != 0) {      // نحفظ الإحداثيات والقيمة فقط
                rows[nonZero] = row;
                cols[nonZero] = col;
                values[nonZero] = matrix[row][col];
                nonZero++;
            }
    printf("Density = %d/%d = %.1f %%\n", nonZero, NB_ROWS * NB_COLS,
           nonZero * 100.0 / (NB_ROWS * NB_COLS));
    for (index = 0; index < nonZero; index++)
        printf("(%d,%d)=%d ", rows[index], cols[index], values[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Density = 6/40 = 15.0 %
(0,2)=3 (0,7)=1 (2,0)=2 (2,4)=5 (3,6)=4 (4,1)=1
```

المخطط الإنسيابي (Organigramme)



تمرين 5.35 عدد مسارات الروبوت في المستودع

تجارة إلكترونية ★★ متوسط

روبوت المستودع يتحرك من الزاوية العليا اليسرى إلى السفلى اليمنى، **يمينًا أو أسفل فقط**، وبعض الخانات محجوبة (1). اكتب برنامجًا يحسب **عدد المسارات** الممكنة بالبرمجة الديناميكية:

```
paths[r][c] = paths[r-1][c] + paths[r][c-1]
```

.

خطوات الحل

1) خانة محجوبة: 0 مسار.
2) خانة البداية: مسار واحد إن لم تكن محجوبة.
3) الخانات الأخرى: مجموع الخانة فوقها والخانة على يسارها (إن وجدتا).

البرنامج بلغة C

```c
#include <stdio.h>

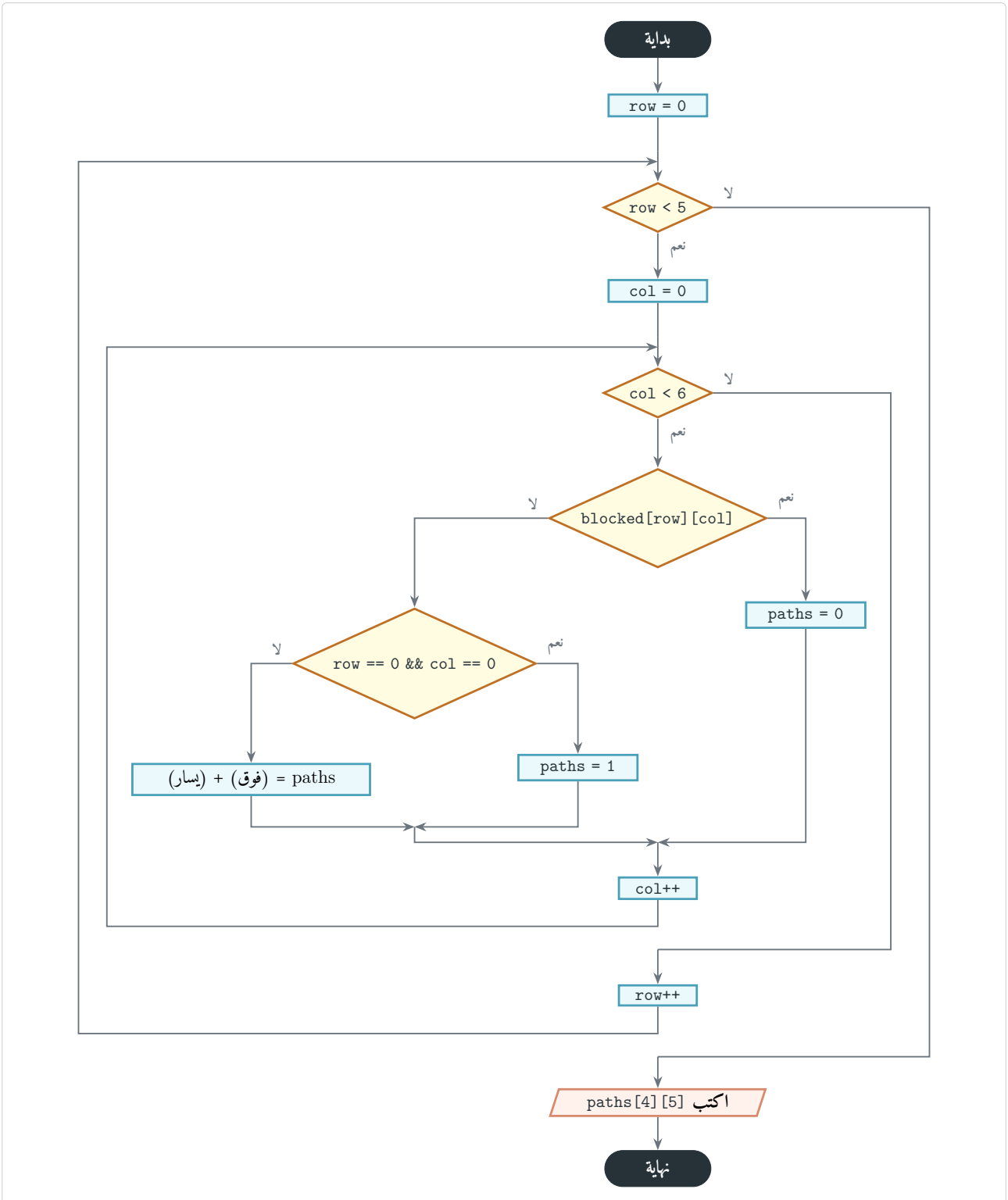
#define NB_ROWS 5
#define NB_COLS 6

int main(void) {
    int blocked[NB_ROWS][NB_COLS] = {{0, 0, 0, 0, 0, 0},
                                     {0, 1, 0, 0, 1, 0},
                                     {0, 0, 0, 1, 0, 0},
                                     {1, 0, 0, 0, 0, 0},
                                     {0, 0, 1, 0, 0, 0}};
    long paths[NB_ROWS][NB_COLS];
    int row, col;
    for (row = 0; row < NB_ROWS; row++) {
        for (col = 0; col < NB_COLS; col++) {
            if (blocked[row][col])
                paths[row][col] = 0;              // لا يمر منها أحد
            else if (row == 0 && col == 0)
                paths[row][col] = 1;              // نقطة الانطلاق
            else
                paths[row][col] = (row > 0 ? paths[row - 1][col] : 0)
                                + (col > 0 ? paths[row][col - 1] : 0);
            printf("%4ld", paths[row][col]);
        }
        printf("\n");
    }
    printf("Paths to the exit: %ld\n", paths[NB_ROWS - 1][NB_COLS - 1]);
    return 0;
}
```

نتيجة التنفيذ

```
   1   1   1   1   1   1
   1   0   1   2   0   1
   1   1   2   0   0   1
   0   1   3   3   3   4
   0   1   0   3   6  10
Paths to the exit: 10
```

المخطط الإنسيابي (Organigramme)



تمرين 5.36 أرخص مسار للتوصيل في المدينة

تجارة إلكترونية ★★ متوسط

المدينة شبكة أحياء `cost[4][5]` (زمن عبور كل حي بالدقائق). سائق التوصيل يتحرك **يميناً أو أسفل**. اكتب برنامجًا يحسب **أقل زمن** من الزاوية العليا اليسرى إلى السفلى اليمنى، ويعيد بناء المسار.

خطوات الحل

1) `best[r][c] = cost[r][c] + min(best[r-1][c], best[r][c-1])`

.

2) إعادة البناء: من النهاية نعود إلى الخانة (فوق أو يسار) ذات القيمة الأصغر.

البرنامج بلغة C

```c
#include <stdio.h>

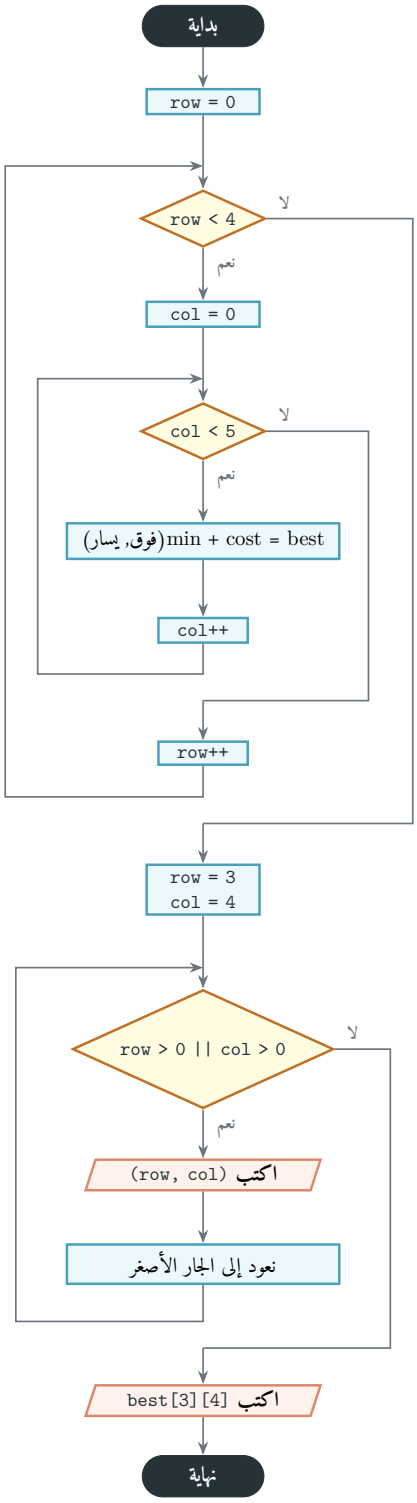
#define NB_ROWS 4
#define NB_COLS 5

int main(void) {
    int cost[NB_ROWS][NB_COLS] = {{3, 7, 2, 8, 1},
                                  {4, 1, 9, 2, 6},
                                  {8, 2, 1, 5, 3},
                                  {6, 9, 4, 1, 2}};
    int best[NB_ROWS][NB_COLS], row, col, up, left;
    for (row = 0; row < NB_ROWS; row++)
        for (col = 0; col < NB_COLS; col++) {
            if (row == 0 && col == 0) { best[row][col] = cost[row][col]; continue; }
            up = (row > 0) ? best[row - 1][col] : -1;     // غير موجود :-1
            left = (col > 0) ? best[row][col - 1] : -1;
            if (up < 0 || (left >= 0 && left < up))
                best[row][col] = cost[row][col] + left;
            else
                best[row][col] = cost[row][col] + up;
        }
    printf("Minimum time = %d min\nPath (reversed):", best[NB_ROWS - 1][NB_COLS - 1]);
    row = NB_ROWS - 1; col = NB_COLS - 1;
    while (row > 0 || col > 0) {          // نعود من النهاية إلى البداية
        printf(" (%d,%d)", row, col);
        if (row == 0) col--;
        else if (col == 0) row--;
        else if (best[row - 1][col] <= best[row][col - 1]) row--;
        else col--;
    }
    printf(" (0,0)\n");
    return 0;
}
```

نتيجة التنفيذ

```
Minimum time = 18 min
Path (reversed): (3,4) (3,3) (3,2) (2,2) (2,1) (1,1) (1,0) (0,0)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.37 أسعار الوحدة من ثلاث فواتير (قاعدة كرامر)

تجارة إلكترونية ★★ متوسط

لدينا ثلاث فواتير، كل واحدة تحتوي كميات من نفس ثلاثة منتجات ومبلغًا إجماليًا، لكن الأسعار مفقودة. اكتب برنامجًا يحل جملة المعادلات $A \times prices = totals$ بـ **قاعدة كرامر**: المحدد بطريقة ساروس.

خطوات الحل

1) `det(A)` بصيغة ساروس للمصفوفة 3×3.
2) لكل مجهول `unknown`: نعوض العمود `unknown` بشعاع المبالغ ونحسب المحدد.
3) `price[unknown] = detReplaced / det`.

البرنامج بلغة C

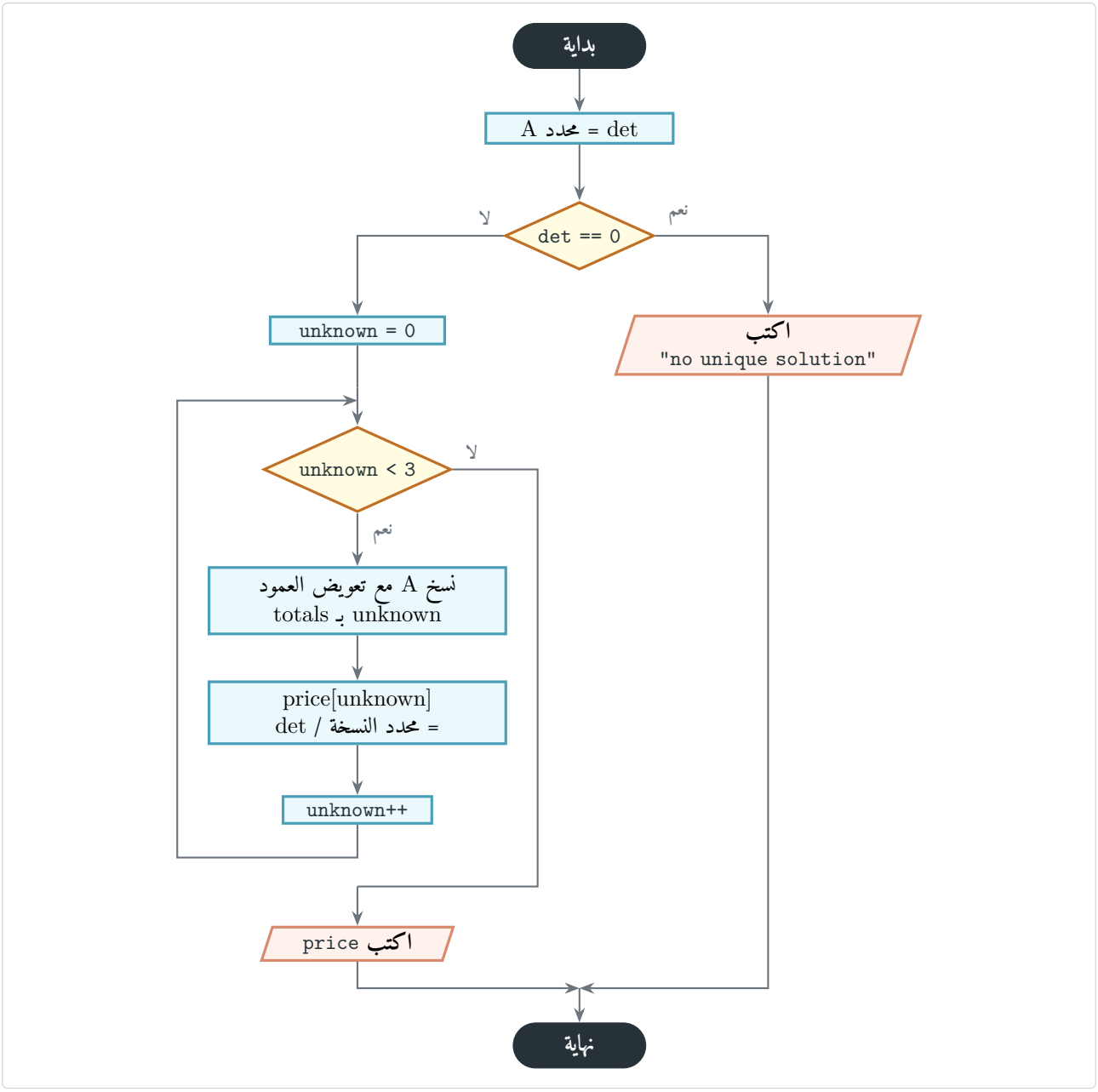
```c
#include <stdio.h>

int main(void) {
    double qty[3][3] = {{2, 1, 3},       // الفاتورة 1: الكميات
                        {1, 4, 1},
                        {3, 2, 2}};
    double totals[3] = {5100, 4300, 6200};
    double work[3][3], det, detReplaced, price[3];
    int unknown, row, col;
    // قاعدة ساروس: مجموع الأقطار ناقص مجموع الأقطار المعاكسة
    det = qty[0][0] * qty[1][1] * qty[2][2] + qty[0][1] * qty[1][2] * qty[2][0]
        + qty[0][2] * qty[1][0] * qty[2][1] - qty[0][2] * qty[1][1] * qty[2][0]
        - qty[0][0] * qty[1][2] * qty[2][1] - qty[0][1] * qty[1][0] * qty[2][2];
    if (det == 0) {
        printf("No unique solution\n");
        return 0;
    }
    for (unknown = 0; unknown < 3; unknown++) {
        for (row = 0; row < 3; row++)
            for (col = 0; col < 3; col++)
                work[row][col] = (col == unknown) ? totals[row] : qty[row][col];
        detReplaced = work[0][0] * work[1][1] * work[2][2] + work[0][1] * work[1][2] * work[2][0]
                    + work[0][2] * work[1][0] * work[2][1] - work[0][2] * work[1][1] * work[2][0]
                    - work[0][0] * work[1][2] * work[2][1] - work[0][1] * work[1][0] * work[2][2];
        price[unknown] = detReplaced / det;
        printf("Product %d: %.2f DA\n", unknown, price[unknown]);
    }
    printf("det = %.0f\n", det);
    return 0;
}
```

نتيجة التنفيذ

```
Product 0: 1200.00 DA
Product 1: 600.00 DA
Product 2: 700.00 DA
det = -17
```

المخطط الإنسيابي (Organigramme)



تمرين 5.38 ترتيب المتاجر حسب مجموع مبيعاتها

علم البيانات ★★ متوسط

اكتب برنامجًا يرتب أسطر مصفوفة المبيعات (متاجر × أيام) تنازليًا حسب مجموع كل سطر، بتبديل الأسطر كاملة، ثم يعرض الترتيب.

خطوات الحل

1) نحسب `total[store]` لكل سطر.
2) ترتيب بالانتقاء على `total`، عند التبديل نبدل أيضًا السطرين كاملين وأرقام المتاجر.

البرنامج بلغة C

```c
#include <stdio.h>

```

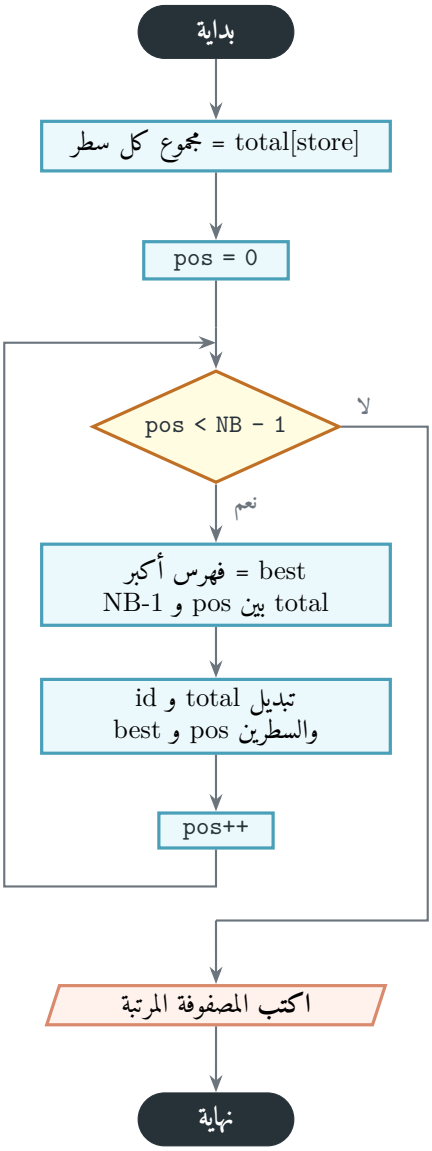
```c
#define NB 4
#define NB_DAYS 3

int main(void) {
    int sales[NB][NB_DAYS] = {{80, 60, 75}, {200, 210, 190}, {120, 95, 140}, {10, 300, 20}};
    int id[NB] = {0, 1, 2, 3}, total[NB];
    int store, day, pos, scan, best, tmp;
    for (store = 0; store < NB; store++) {
        total[store] = 0;
        for (day = 0; day < NB_DAYS; day++)
            total[store] += sales[store][day];
    }
    for (pos = 0; pos < NB - 1; pos++) {          // ترتيب بالانتقاء
        best = pos;
        for (scan = pos + 1; scan < NB; scan++)
            if (total[scan] > total[best])
                best = scan;
        tmp = total[pos]; total[pos] = total[best]; total[best] = tmp;
        tmp = id[pos]; id[pos] = id[best]; id[best] = tmp;
        for (day = 0; day < NB_DAYS; day++) {     // تبديل السطرين كاملين
            tmp = sales[pos][day];
            sales[pos][day] = sales[best][day];
            sales[best][day] = tmp;
        }
    }
    for (store = 0; store < NB; store++) {
        printf("#%d store %d:", store + 1, id[store]);
        for (day = 0; day < NB_DAYS; day++)
            printf(" %3d", sales[store][day]);
        printf(" = %d\n", total[store]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
#1 store 1: 200 210 190 = 600
#2 store 2: 120  95 140 = 355
#3 store 3:  10 300  20 = 330
#4 store 0:  80  60  75 = 215
```

المخطط الإنسيابي (Organigramme)



تمرين 5.39 حجز فترات توصيل متتالية

تجارة إلكترونية ★★ متوسط

مصفوفة `slots[5][8]` تمثل فترات التوصيل لـ 5 أيام (8 فترات يومياً): 0 حرة و1 محجوزة. زبون يحتاج `need` فترات **متتالية** في نفس اليوم. اكتب برنامجًا يجد **أول** يوم وفترة بداية ممكنين ويحجزهما.

خطوات الحل

1) لكل يوم نعد الفترات الحرة المتتالية `run`، نعيده إلى 0 عند فترة محجوزة.
2) عندما يبلغ `run == need` البداية: `slot - need + 1`.
3) نحجز الفترات بوضع 1.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int slots[5][8] = {{1, 1, 0, 0, 1, 0, 1, 1},
                       {0, 1, 1, 0, 0, 1, 0, 0},
                       {1, 0, 0, 0, 0, 1, 1, 0},
                       {0, 0, 0, 1, 1, 1, 1, 1},
                       {1, 1, 1, 1, 1, 1, 1, 1}};
    int need, day, slot, run, found = 0, start = 0, bookDay = 0, pos;
    scanf("%d", &need);
    for (day = 0; day < 5 && !found; day++) {
        run = 0;                            // طول السلسلة الحرة الحالية
        for (slot = 0; slot < 8 && !found; slot++) {
            if (slots[day][slot] == 0) run++;
            else run = 0;
            if (run == need) {
                found = 1;
                start = slot - need + 1;
                bookDay = day;
            }
        }
    }
    if (!found) {
        printf("No %d consecutive free slots\n", need);
    } else {
        for (pos = start; pos < start + need; pos++)
            slots[bookDay][pos] = 1;        // الحجز
        printf("Booked day %d, slots %d to %d:", bookDay + 1, start, start + need - 1);
        for (pos = 0; pos < 8; pos++)
            printf(" %d", slots[bookDay][pos]);
        printf("\n");
    }
    return 0;
}
```

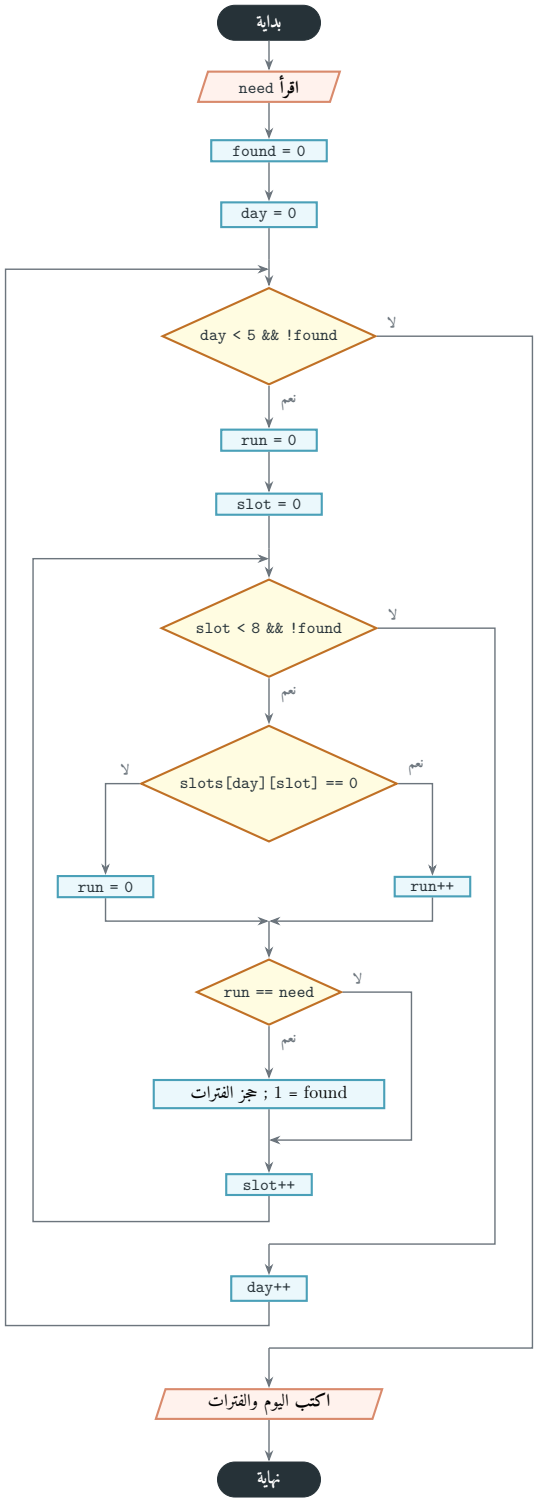
المدخلات

```
3
```

نتيجة التنفيذ

```
Booked day 3, slots 1 to 3: 1 1 1 1 0 1 1 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.40 قوة مصفوفة الإنتقال

علم البيانات ★★ متوسط

احتمال انتقال زبون بين 3 صفحات من الموقع في خطوة واحدة معطى بالمصفوفة `step`. اكتب برنامجًا يحسب **مصفوفة الانتقال بعد خطوتين** `step × step`، ويتحقق أن مجموع كل سطر يبقى 1.

خطوات الحل

1) `twoSteps[from][to] = Σ step[from][mid] * step[mid][to]` نمر بكل صفحة وسيطة.

2) مجموع كل سطر يجب أن يساوي 1 (توزيع احتمالي).

البرنامج بلغة C

```c
#include <stdio.h>

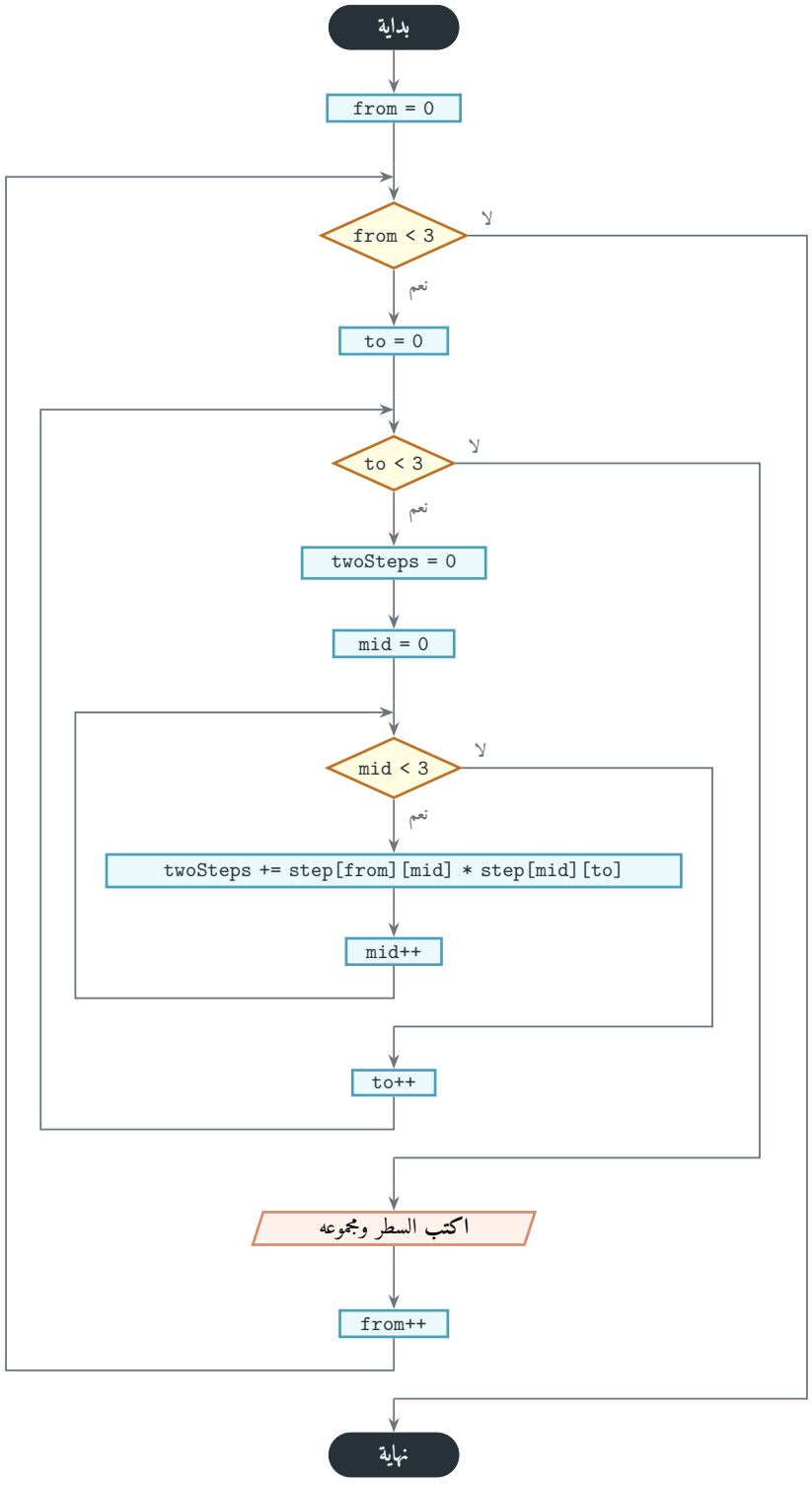
#define NB 3

int main(void) {
    double step[NB][NB] = {{0.2, 0.5, 0.3},       // الرئيسية
                           {0.1, 0.6, 0.3},       // المنتج
                           {0.4, 0.2, 0.4}};      // السلة
    double twoSteps[NB][NB], rowSum;
    int from, to, mid;
    for (from = 0; from < NB; from++) {
        rowSum = 0;
        for (to = 0; to < NB; to++) {
            twoSteps[from][to] = 0;
            for (mid = 0; mid < NB; mid++)        // كل الطرق عبر صفحة وسيطة
                twoSteps[from][to] += step[from][mid] * step[mid][to];
            rowSum += twoSteps[from][to];
            printf(" %.3f", twoSteps[from][to]);
        }
        printf("  (sum %.2f)\n", rowSum);
    }
    return 0;
}
```

نتيجة التنفيذ

```
0.210 0.460 0.330  (sum 1.00)
0.200 0.470 0.330  (sum 1.00)
0.260 0.400 0.340  (sum 1.00)
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 5.41 حل جملة معادلات بطريقة غاوس

علم البيانات ★★★ صعب

نموذج تسعير يقود إلى جملة من 4 معادلات خطية بـ 4 مجاهيل `A x = b`. اكتب برنامجًا يحلها بطريقة **الحذف لغاوس مع الارتكاز الجزئي** (اختيار أكبر محور بالقيمة المطلقة لتقليل أخطاء التقريب) ثم **التعويض التراجعي**.

خطوات الحل

1) نضع `b` كعمود إضافي في المصفوفة الموسعة `aug[4][5]`.
2) لكل عمود `pivotCol`: نختار السطر ذا أكبر `|aug[row][pivotCol]|` ونبدله مع سطر المحور.
3) نحذف المجهول من الأسطر السفلى: `aug[row] -= factor * aug[pivotCol]`.
4) التعويض التراجعي من آخر سطر إلى الأول.

البرنامج بلغة C

```c
#include <stdio.h>

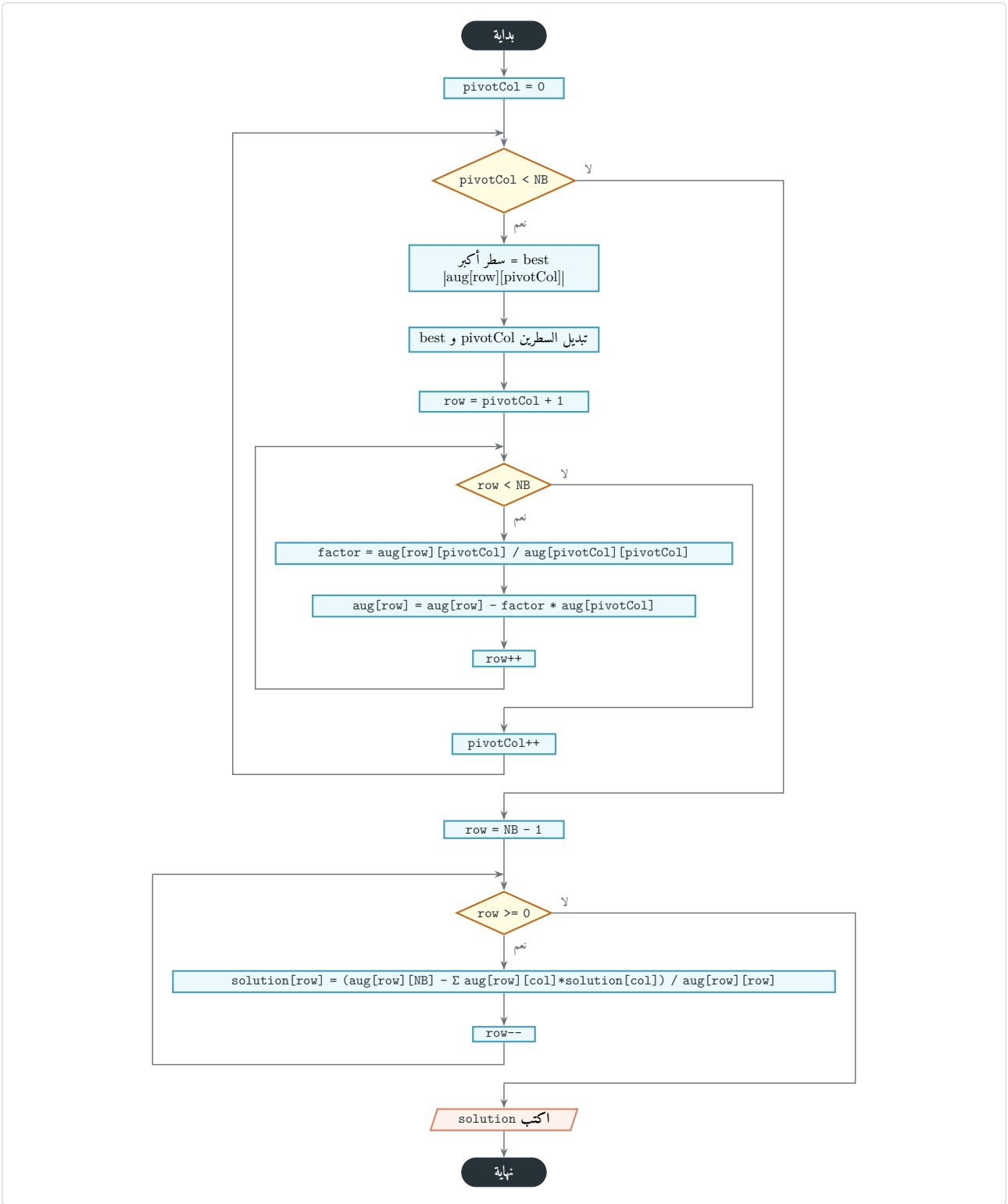
#define NB 4

int main(void) {
    double aug[NB][NB + 1] = {{ 2,  1, -1,  3,  18},     // المصفوفة الموسعة [A | b]
                              { 4,  1,  2, -1,  14},
                              {-2,  5,  1,  1,  20},
                              { 1, -1,  3,  2,  21}};
    double solution[NB], factor, tmp, bestVal, value, sum;
    int pivotCol, row, col, best;
    for (pivotCol = 0; pivotCol < NB; pivotCol++) {
        best = pivotCol;                          // الارتكاز الجزئي
        bestVal = aug[pivotCol][pivotCol] < 0 ? -aug[pivotCol][pivotCol] : aug[pivotCol][pivotCol];
        for (row = pivotCol + 1; row < NB; row++) {
            value = aug[row][pivotCol] < 0 ? -aug[row][pivotCol] : aug[row][pivotCol];
            if (value > bestVal) { bestVal = value; best = row; }
        }
        for (col = 0; col <= NB; col++) {         // تبديل السطرين
            tmp = aug[pivotCol][col]; aug[pivotCol][col] = aug[best][col]; aug[best][col] = tmp;
        }
        for (row = pivotCol + 1; row < NB; row++) {   // الحذف تحت المحور
            factor = aug[row][pivotCol] / aug[pivotCol][pivotCol];
            for (col = pivotCol; col <= NB; col++)
                aug[row][col] -= factor * aug[pivotCol][col];
        }
    }
    for (row = NB - 1; row >= 0; row--) {         // التعويض التراجعي
        sum = aug[row][NB];
        for (col = row + 1; col < NB; col++)
            sum -= aug[row][col] * solution[col];
        solution[row] = sum / aug[row][row];
    }
    for (row = 0; row < NB; row++)
        printf("x%d = %.4f\n", row + 1, solution[row]);
    return 0;
}
```

نتيجة التنفيذ

```
x1 = 2.0000
x2 = 3.0000
x3 = 4.0000
x4 = 5.0000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.42 مقلوب مصفوفة بطريقة غاوس-جوردان

علم البيانات صعب ★★★

اكتب برنامجًا يحسب **مقلوب** مصفوفة 3×3 بطريقة **غاوس-جوردان**: نلصق مصفوفة الوحدة على يمين المصفوفة، ونحول الجزء الأيسر إلى الوحدة؛ الجزء الأيمن يصبح المقلوب. تحقق بحساب $A \times A^{-1}$.

خطوات الحل

1) `work[3][6]`: الأعمدة 0..2 هي `A` والأعمدة 3..5 هي الوحدة.
2) لكل محور: نقسم سطره على المحور ليصبح 1، ثم نصفر العمود في **كل** الأسطر الأخرى (فوق وتحت).
3) إذا كان المحور 0 نبحث عن سطر نبدله معه؛ إذا لم نجد فالمصفوفة غير قابلة للقلب.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 3

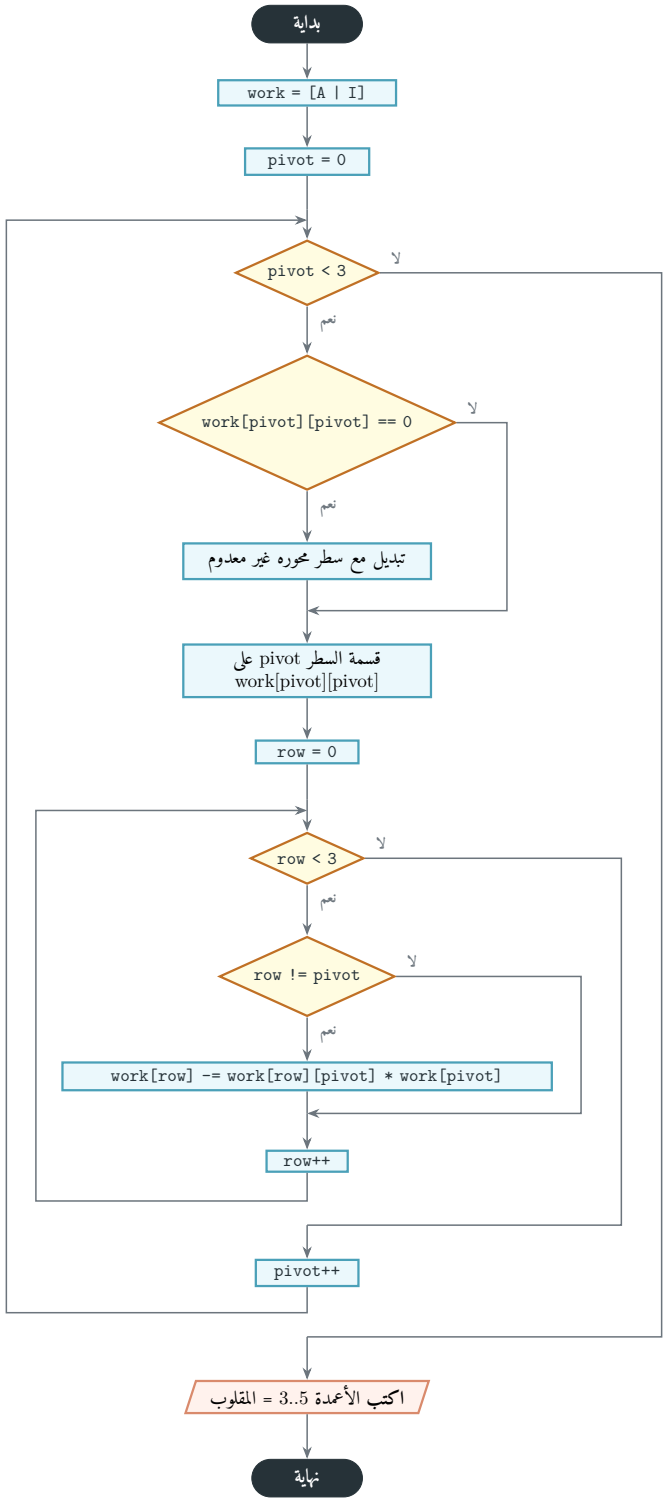
int main(void) {
    double matrix[NB][NB] = {{2, 1, 1}, {1, 3, 2}, {1, 0, 0}};
    double work[NB][2 * NB], factor, tmp, check;
    int pivot, row, col, swapRow, inner;
    for (row = 0; row < NB; row++)
        for (col = 0; col < 2 * NB; col++)
            work[row][col] = (col < NB) ? matrix[row][col] : (col - NB == row);  // [A | I]
    for (pivot = 0; pivot < NB; pivot++) {
        if (work[pivot][pivot] == 0) {            // محور معدوم: نبحث عن سطر آخر
            for (swapRow = pivot + 1; swapRow < NB && work[swapRow][pivot] == 0; swapRow++)
                ;
            if (swapRow == NB) { printf("Matrix is not invertible\n"); return 0; }
            for (col = 0; col < 2 * NB; col++) {
                tmp = work[pivot][col]; work[pivot][col] = work[swapRow][col]; work[swapRow][col] = tmp;
            }
        }
        factor = work[pivot][pivot];
        for (col = 0; col < 2 * NB; col++)
            work[pivot][col] /= factor;           // المحور يصبح 1
        for (row = 0; row < NB; row++)
            if (row != pivot) {                   // نصفر العمود في باقي الأسطر
                factor = work[row][pivot];
                for (col = 0; col < 2 * NB; col++)
                    work[row][col] -= factor * work[pivot][col];
            }
    }
    printf("Inverse:\n");
    for (row = 0; row < NB; row++) {
        for (col = NB; col < 2 * NB; col++)
            printf(" %7.3f", work[row][col]);
        printf("\n");
    }
    printf("Check A x inv (diagonal):");
    for (row = 0; row < NB; row++) {
        check = 0;
        for (inner = 0; inner < NB; inner++)
            check += matrix[row][inner] * work[inner][NB + row];
        printf(" %.3f", check);
    }
    printf("\n");
    return 0;
```

```
46 }
```

نتيجة التنفيذ

```
Inverse:
   0.000   0.000   1.000
  -2.000   1.000   3.000
   3.000  -1.000  -5.000
Check A x inv (diagonal): 1.000 1.000 1.000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.43 ترتيب صفحات المنتجات بخوارزمية PageRank

علم البيانات ★★★ صعب

صفحات 5 منتجات مرتبطة بروابط "منتجات مشابهة": `links[from][to] = 1`. اكتب برنامجًا يحسب **PageRank** لكل صفحة بطريقة **القوة المتكررة**:

```
rank_new[to] = (1 - d)/N + d × Σ rank[from] / outDegree[from]
```

مع `d = 0.85`، حتى يصبح التغير الكلي أقل من `1e-6`.

خطوات الحل

1) `outDegree[from]` عدد الروابط الخارجة من كل صفحة.
2) نبدأ بـ `rank = 1/N` لكل صفحة.
3) في كل تكرار: كل صفحة توزع رتبتها بالتساوي على الصفحات التي تشير إليها.
4) نتوقف عندما يصبح `Σ |rank_new - rank|` صغيرًا.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 5
#define DAMPING 0.85

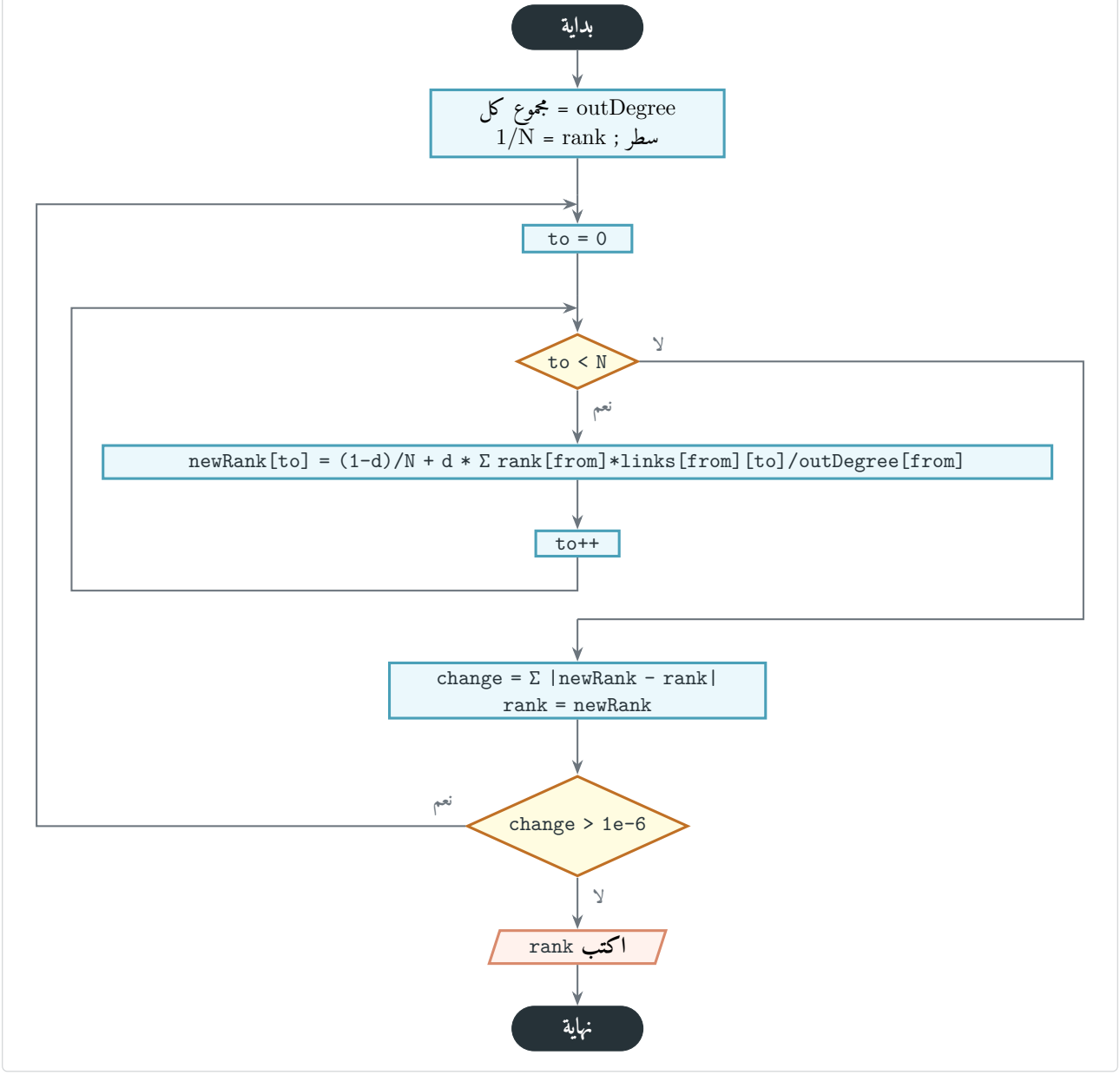
int main(void) {
    int links[NB][NB] = {{0, 1, 1, 0, 0},
                         {0, 0, 1, 0, 0},
                         {1, 0, 0, 1, 0},
                         {0, 0, 1, 0, 1},
                         {1, 0, 1, 0, 0}};
    double rank[NB], newRank[NB], change, gap;
    int outDegree[NB], from, to, iteration = 0;
    for (from = 0; from < NB; from++) {
        outDegree[from] = 0;
        for (to = 0; to < NB; to++)
            outDegree[from] += links[from][to];
        rank[from] = 1.0 / NB;                    // توزيع ابتدائي متساوٍ
    }
    do {
        change = 0;
        for (to = 0; to < NB; to++) {
            newRank[to] = (1 - DAMPING) / NB;     // القفز العشوائي
            for (from = 0; from < NB; from++)
                if (links[from][to])
                    newRank[to] += DAMPING * rank[from] / outDegree[from];
        }
        for (to = 0; to < NB; to++) {
            gap = newRank[to] - rank[to];
            change += (gap < 0) ? -gap : gap;
            rank[to] = newRank[to];
        }
        iteration++;
    } while (change > 1e-6);
    printf("Converged after %d iterations\n", iteration);
    for (to = 0; to < NB; to++)
        printf("Page %d: %.4f\n", to, rank[to]);
    return 0;
}
```

نتيجة التنفيذ

```
Converged after 25 iterations
Page 0: 0.2273
Page 1: 0.1266
Page 2: 0.3571
```

```
Page 3: 0.1818
Page 4: 0.1072
```

المخطط الإنسيابي (Organigramme)



تمرين 5.44 أقصر المسافات بين كل المستودعات (فلويد-وارشال) تجارة إلكترونية صعب

مصفوفة `dist[5][5]` تعطي طول الطريق المباشر بين المستودعات (`INF` إذا لم يوجد طريق مباشر). اكتب برنامجًا يحسب **أقصر مسافة بين كل زوج** بخوارزمية **فلويد-وارشال**: لكل مستودع وسيط `via`، نتحقق هل المرور عبره أقصر.

خطوات الحل

1) ثلاث حلقات متداخلة: الوسيط `via` في الخارج.

2) `if (dist[from][via] + dist[via][to] < dist[from][to])`

نحدّث.

3) تجنب جمع `INF` مع قيمة.

البرنامج بلغة C

```c
#include <stdio.h>

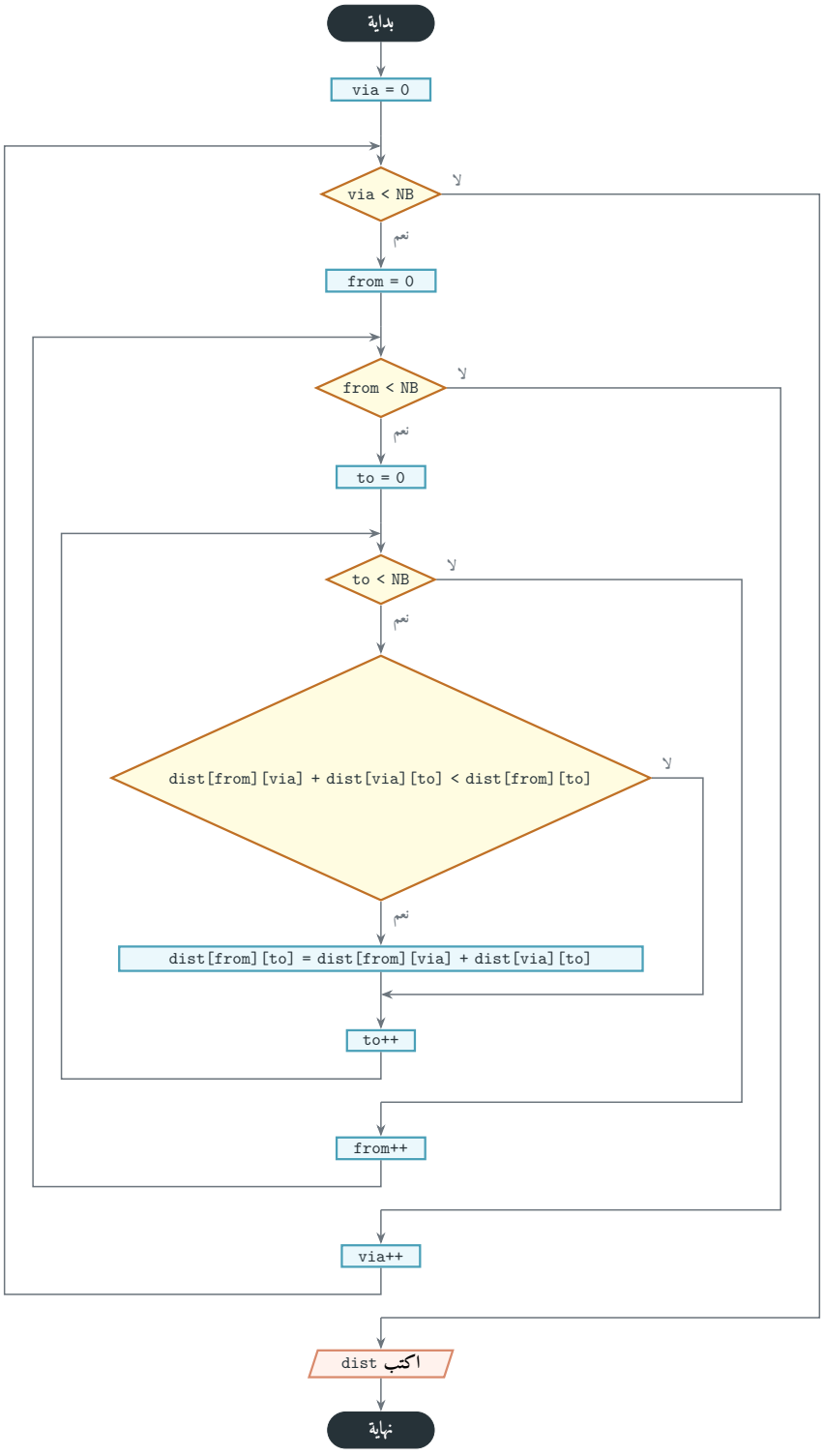
#define NB 5
#define INF 1000000

int main(void) {
    int dist[NB][NB] = {{  0, 120, INF, 300, INF},
                        {120,   0,  90, INF, 400},
                        {INF,  90,   0,  60, 150},
                        {300, INF,  60,   0, 110},
                        {INF, 400, 150, 110,   0}};
    int via, from, to;
    for (via = 0; via < NB; via++)                // المستودع الوسيط
        for (from = 0; from < NB; from++)
            for (to = 0; to < NB; to++)
                if (dist[from][via] < INF && dist[via][to] < INF
                    && dist[from][via] + dist[via][to] < dist[from][to])
                    dist[from][to] = dist[from][via] + dist[via][to];   // طريق أقصر
    for (from = 0; from < NB; from++) {
        for (to = 0; to < NB; to++)
            printf("%5d", dist[from][to]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
    0  120  210  270  360
  120    0   90  150  240
  210   90    0   60  150
  270  150   60    0  110
  360  240  150  110    0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.45 أقصر طريق للتوصيل (ديكسترا)

تجارة إلكترونية

صعب ★★★

اكتب برنامجًا يحسب أقصر طريق من المستودع (العقدة 0) إلى كل حي في المدينة بخوارزمية **ديكسترا** على مصفوفة المجاورة (0 = لا طريق)، ويعيد بناء المسار إلى حي مقروء باستعمال جدول `previous`.

خطوات الحل

1) `best[node] = INF` إلا المصدر 0، و`done[node] = 0`.
2) في كل دورة: نختار العقدة غير المعالجة ذات أصغر `best`، ونعلمها معالجة.
3) نحدّث جيرانها: إذا كان `best[node] + weight < best[next]`.
4) المسار: نتبع `previous` من الوجهة إلى المصدر.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 6
#define INF 1000000

int main(void) {
    int road[NB][NB] = {{0, 7, 9, 0, 0, 14},
                        {7, 0, 10, 15, 0, 0},
                        {9, 10, 0, 11, 0, 2},
                        {0, 15, 11, 0, 6, 0},
                        {0, 0, 0, 6, 0, 9},
                        {14, 0, 2, 0, 9, 0}};
    int best[NB], done[NB], previous[NB], path[NB];
    int node, next, round, target, length = 0;
    for (node = 0; node < NB; node++) {
        best[node] = INF; done[node] = 0; previous[node] = -1;
    }
    best[0] = 0;                                    // المستودع
    for (round = 0; round < NB; round++) {
        node = -1;                                  // أقرب عقدة غير معالجة
        for (next = 0; next < NB; next++)
            if (!done[next] && (node < 0 || best[next] < best[node]))
                node = next;
        done[node] = 1;                             // مسافتها نهائية الآن
        for (next = 0; next < NB; next++)
            if (road[node][next] && best[node] + road[node][next] < best[next]) {
                best[next] = best[node] + road[node][next];
                previous[next] = node;
            }
    }
    scanf("%d", &target);
    for (node = target; node != -1; node = previous[node])
        path[length++] = node;                      // المسار معكوسًا
    printf("Distance to %d = %d min\nPath:", target, best[target]);
    while (length > 0)
        printf(" %d", path[--length]);
    printf("\n");
    return 0;
}
```

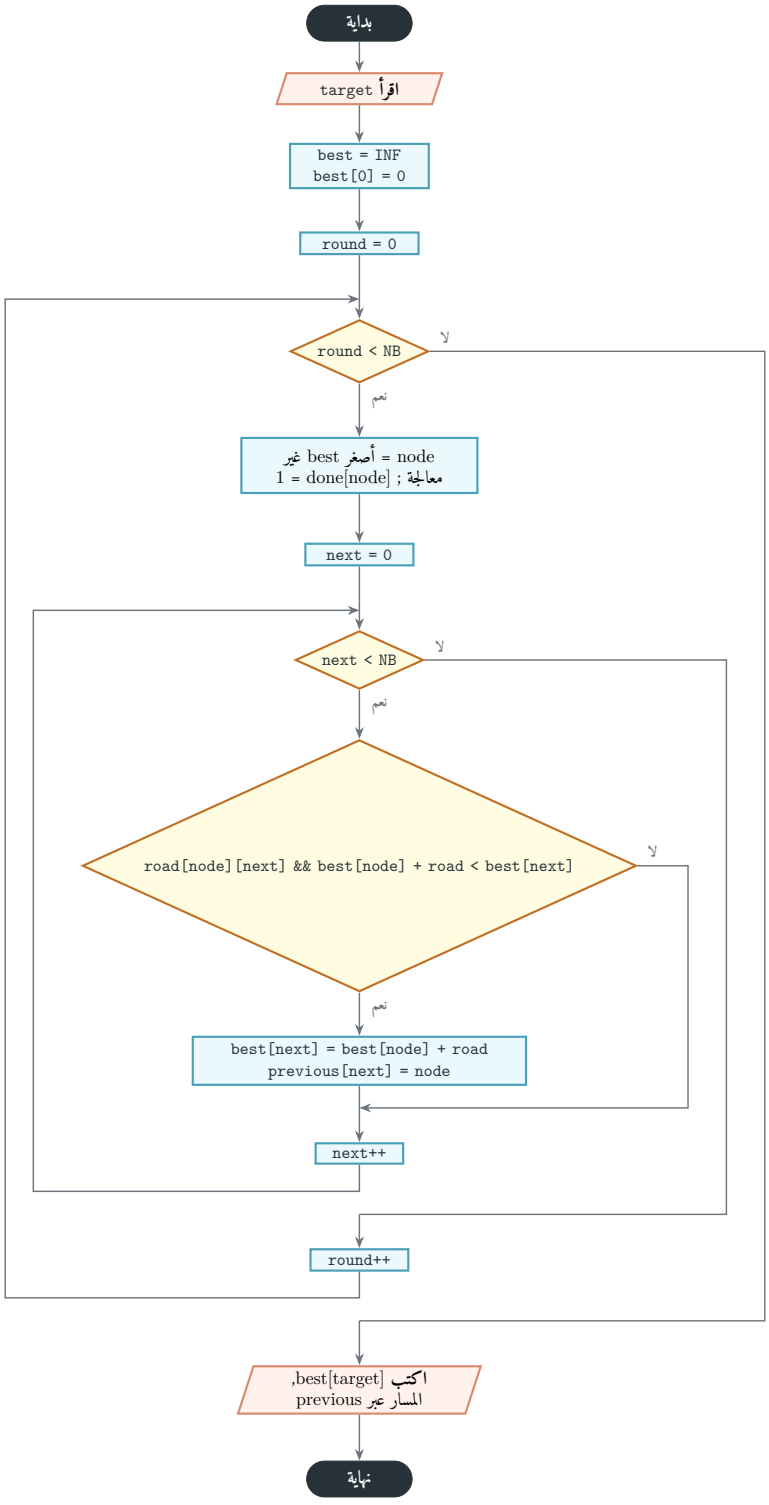
المدخلات

```
5
```

نتيجة التنفيذ

```
Distance to 5 = 11 min
Path: 0 2 5
```

المخطط الإنسيابي (Organigramme)



تمرين 5.46 الإنحدار الخطي المتعدد بالمعادلات الطبيعية

علم البيانات ★★★ صعب

نريد التنبؤ بالمبيعات `y` من ميزانية الإعلان `x1` وعدد التخفيضات `x2`: `y = b0 + b1 x1 + b2 x2`. اكتب برنامجًا يبني المعادلات الطبيعية `(X X) = X y` (مصفوفة 3×3) ثم يحلها بالحذف لغاوس، ويحسب معامل التحديد R^2.

خطوات الحل

1) مصفوفة التصميم X: عمود من 1 ثم x1 ثم x2.
2) xtx[a][b] = Σ X[row][a] * X[row][b] و xty[a] = Σ X[row][a] * sales[row].
3) حل الجملة 3×3 بالحذف ثم التعويض.
4) $R^2 = 1 - SSres / SStot$.

البرنامج بلغة C

```c
#include <stdio.h>

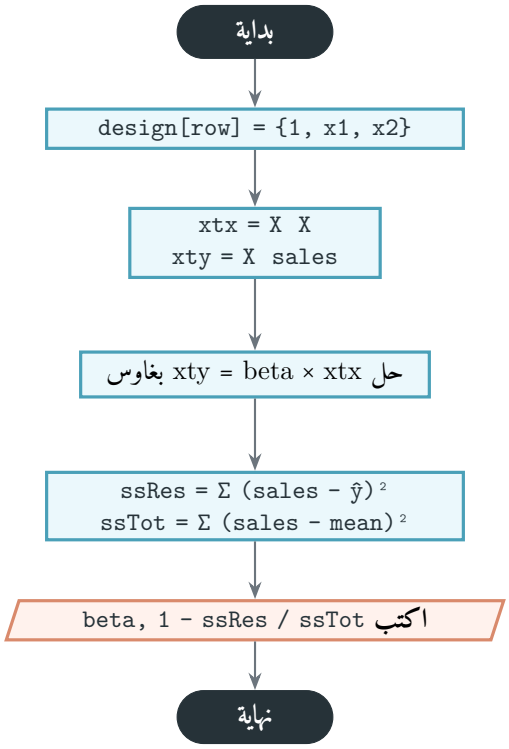
#define NB 8
#define NB_COEF 3

int main(void) {
    double x1[NB] = {1, 2, 3, 4, 5, 6, 7, 8};              // الإعلان
    double x2[NB] = {0, 1, 0, 2, 1, 3, 2, 3};              // التخفيضات
    double sales[NB]  = {14, 22, 21, 35, 33, 47, 44, 52};     // المبيعات
    double design[NB][NB_COEF], aug[NB_COEF][NB_COEF + 1], beta[NB_COEF];
    double factor, sum, mean = 0, ssRes = 0, ssTot = 0, predicted;
    int row, colA, colB, pivot;
    for (row = 0; row < NB; row++) {
        design[row][0] = 1; design[row][1] = x1[row]; design[row][2] = x2[row];
        mean += sales[row] / NB;
    }
    for (colA = 0; colA < NB_COEF; colA++) {      // المعادلات الطبيعية
        for (colB = 0; colB < NB_COEF; colB++) {
            aug[colA][colB] = 0;
            for (row = 0; row < NB; row++)
                aug[colA][colB] += design[row][colA] * design[row][colB];
        }
        aug[colA][NB_COEF] = 0;
        for (row = 0; row < NB; row++)
            aug[colA][NB_COEF] += design[row][colA] * sales[row];
    }
    for (pivot = 0; pivot < NB_COEF; pivot++)     // الحذف (XtX موجبة: لا ارتكاز)
        for (row = pivot + 1; row < NB_COEF; row++) {
            factor = aug[row][pivot] / aug[pivot][pivot];
            for (colB = pivot; colB <= NB_COEF; colB++)
                aug[row][colB] -= factor * aug[pivot][colB];
        }
    for (row = NB_COEF - 1; row >= 0; row--) {    // التعويض التراجعي
        sum = aug[row][NB_COEF];
        for (colB = row + 1; colB < NB_COEF; colB++)
            sum -= aug[row][colB] * beta[colB];
        beta[row] = sum / aug[row][row];
    }
    for (row = 0; row < NB; row++) {
        predicted = beta[0] + beta[1] * x1[row] + beta[2] * x2[row];
        ssRes += (sales[row] - predicted) * (sales[row] - predicted);
        ssTot += (sales[row] - mean) * (sales[row] - mean);
    }
    printf("sales = %.3f + %.3f*ads + %.3f*promos\n", beta[0], beta[1], beta[2]);
    printf("R^2 = %.4f\n", 1 - ssRes / ssTot);
    return 0;
}
```

نتيجة التنفيذ

```
sales = 10.989 + 3.237*ads + 5.298*promos
R^2 = 0.9982
```

المخطط الإنسيابي (Organigramme)



تمرين 5.47 تجميع الزبائن في بعدين

علم البيانات صعب

مصفوفة points[12][2]: لكل زبون (التكرار الشهري، متوسط السلة). اكتب برنامجًا يطبق **K-Means** بمجموعتين في بعدين: المراكز مصفوفة centers[2][2] تبدأ بأول زبونين، والمسافة مربع إقليدي. نعرض المراكز النهائية وحجم كل مجموعة ومجموع المربعات داخل المجموعات.

خطوات الحل

1) إسناد كل زبون لأقرب مركز في بعدين.
2) إعادة الحساب: متوسط كل إحداثية لأعضاء المجموعة.
3) التوقف عندما لا يتغير أي إسناد.
4) inertia = Σ مربع المسافة إلى المركز.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12
#define NB_K 2

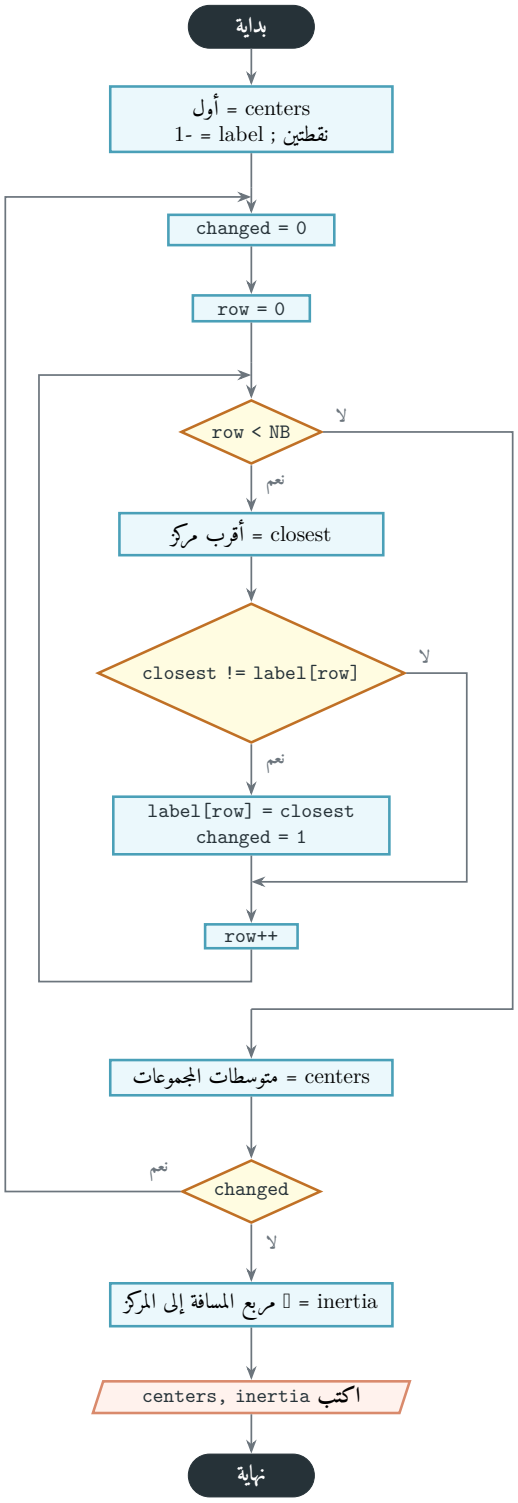
int main(void) {
    double points[NB][2] = {{1, 2.0}, {2, 1.5}, {1, 1.0}, {3, 2.5}, {2, 2.2}, {9, 8.0},
                            {8, 7.5}, {10, 9.0}, {9, 7.0}, {7, 8.5}, {3, 1.8}, {8, 9.2}};
    double centers[NB_K][2], sum[NB_K][2], dx, dy, dist, bestDist, inertia = 0;
    int label[NB], count[NB_K], row, cluster, closest, changed, axis;
    for (cluster = 0; cluster < NB_K; cluster++)
        for (axis = 0; axis < 2; axis++)
```

```c
            centers[cluster][axis] = points[cluster][axis];    // أول نقطتين
    for (row = 0; row < NB; row++)
        label[row] = -1;
    do {
        changed = 0;
        for (row = 0; row < NB; row++) {
            closest = 0; bestDist = -1;
            for (cluster = 0; cluster < NB_K; cluster++) {
                dx = points[row][0] - centers[cluster][0];
                dy = points[row][1] - centers[cluster][1];
                dist = dx * dx + dy * dy;
                if (bestDist < 0 || dist < bestDist) { bestDist = dist; closest = cluster; }
            }
            if (closest != label[row]) { label[row] = closest; changed = 1; }
        }
        for (cluster = 0; cluster < NB_K; cluster++) {
            sum[cluster][0] = sum[cluster][1] = 0; count[cluster] = 0;
        }
        for (row = 0; row < NB; row++) {          // مجموع الإحداثيات لكل مجموعة
            sum[label[row]][0] += points[row][0];
            sum[label[row]][1] += points[row][1];
            count[label[row]]++;
        }
        for (cluster = 0; cluster < NB_K; cluster++)
            if (count[cluster] > 0)
                for (axis = 0; axis < 2; axis++)
                    centers[cluster][axis] = sum[cluster][axis] / count[cluster];
    } while (changed);
    for (row = 0; row < NB; row++) {
        dx = points[row][0] - centers[label[row]][0];
        dy = points[row][1] - centers[label[row]][1];
        inertia += dx * dx + dy * dy;
    }
    for (cluster = 0; cluster < NB_K; cluster++)
        printf("Cluster %d: center (%.2f, %.2f), %d customers\n", cluster,
               centers[cluster][0], centers[cluster][1], count[cluster]);
    printf("Inertia = %.3f\n", inertia);
    return 0;
}
```

نتيجة التنفيذ

```
Cluster 0: center (2.00, 1.83), 6 customers
Cluster 1: center (8.50, 8.20), 6 customers
Inertia = 14.613
```

المخطط الإنسيابي (Organigramme)



تمرين 5.48 التنبؤ بتقييم مفقود (الترشيح التعاوني)

علم البيانات صعب ★★★

في مصفوفة التقييمات (زبائن × منتجات) القيمة 0 تعني "لم يقيّم". نريد التنبؤ بتقييم الزبون `target` للمنتج `item` بطريقة **الترشيح التعاوني**: نحسب **تشابه جيب التمام** بينه وبين كل زبون آخر قيّم المنتج (على المنتجات المشتركة فقط)، ثم المتوسط الموزون بالتشابه.

خطوات الحل

1) لكل زبون آخر `other` قيّم `item`: نحسب `dot`، و`normA`، و`normB` على المنتجات التي قيّمها الاثنان.
2) `similarity = dot / (normA * normB)` (الجذر بطريقة نيوتن).
3) `prediction = Σ similarity × rating[other][item] / Σ similarity`

.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_USERS 5
#define NB_ITEMS 5

int main(void) {
    int ratings[NB_USERS][NB_ITEMS] = {{5, 4, 0, 1, 0},
                                       {4, 5, 4, 1, 1},
                                       {1, 1, 2, 5, 4},
                                       {5, 5, 5, 0, 1},
                                       {2, 0, 1, 4, 5}};
    int target, item, other, col, step;
    double dot, normA, normB, product, root, similarity;
    double weightedSum = 0, simSum = 0;
    scanf("%d %d", &target, &item);
    for (other = 0; other < NB_USERS; other++) {
        if (other == target || ratings[other][item] == 0)
            continue;                           // لم يقيّم المنتج المطلوب
        dot = normA = normB = 0;
        for (col = 0; col < NB_ITEMS; col++)
            if (ratings[target][col] > 0 && ratings[other][col] > 0) {   // مشترك
                dot += ratings[target][col] * ratings[other][col];
                normA += ratings[target][col] * ratings[target][col];
                normB += ratings[other][col] * ratings[other][col];
            }
        if (dot == 0)
            continue;
        product = normA * normB;
        root = product;
        for (step = 0; step < 60; step++)       // الجذر بطريقة نيوتن
            root = (root + product / root) / 2;
        similarity = dot / root;
        printf("User %d: similarity %.3f, rating %d\n", other, similarity, ratings[other][item]);
        weightedSum += similarity * ratings[other][item];
        simSum += similarity;
    }
    if (simSum > 0)
        printf("Predicted rating of user %d for item %d = %.2f\n", target, item, weightedSum / simSum);
    else
        printf("Not enough data\n");
    return 0;
}
```

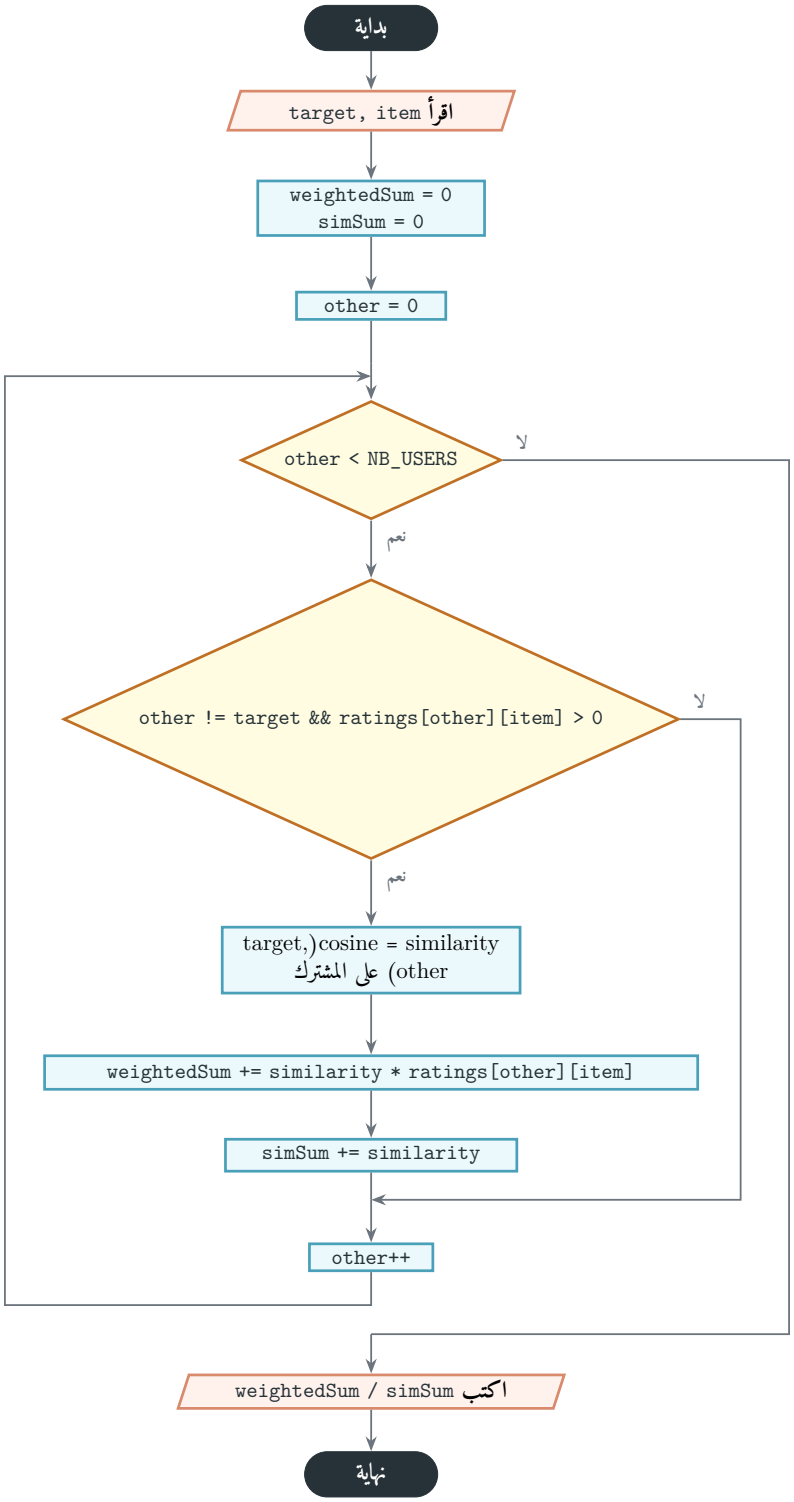
المدخلات

```
0 2
```

نتيجة التنفيذ

```
User 1: similarity 0.976, rating 4
User 2: similarity 0.416, rating 2
User 3: similarity 0.994, rating 5
User 4: similarity 0.614, rating 1
Predicted rating of user 0 for item 2 = 3.44
```

المخطط الإنسيابي (Organigramme)



تمرين 5.49 مناطق التوصيل المتصلة (ملء المناطق)

تجارة إلكترونية صعب

خريطة المدينة شبكة zone[6][8]: 1 حي مخدوم و0 غير مخدوم. نريد عد **المناطق المتصلة** (أحياء مخدومة متجاورة أفقيًا أو عموديًا) وحجم كل منطقة، لتعيين سائق لكل منطقة. اكتب البرنامج بخوارزمية **ملء المنطقة** باستعمال **مكدس يدوي** (دون عودية).

خطوات الحل

1) `region[row][col]` رقم المنطقة لكل خانة (0 = لم تُزر).
2) لكل خانة مخدومة لم تُزر: نبدأ منطقة جديدة وندفعها في المكدس.
3) ما دام المكدس غير فارغ: نسحب خانة ونعلّم جيرانها الأربعة المخدومين غير المزارين وندفعهم.

البرنامج بلغة C

```c
#include <stdio.h>

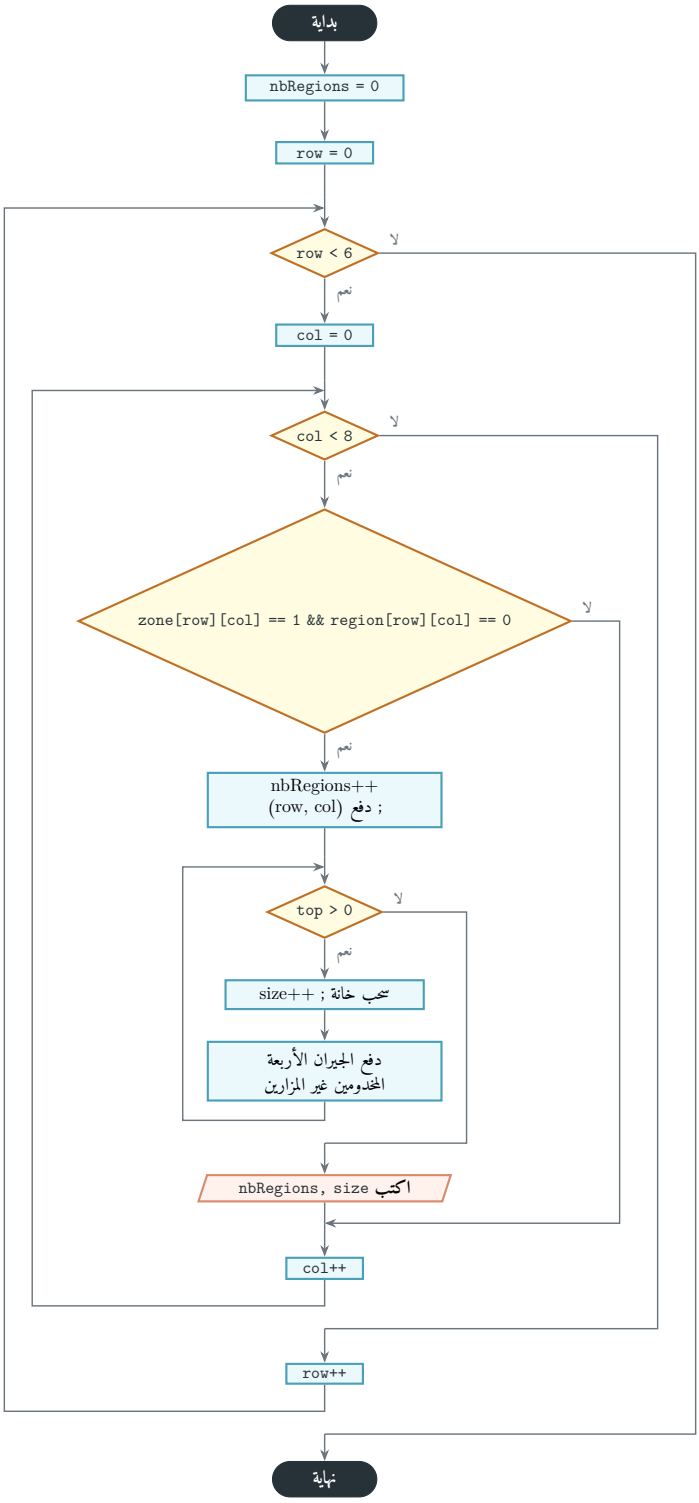
#define NB_ROWS 6
#define NB_COLS 8

int main(void) {
    int zone[NB_ROWS][NB_COLS] = {{1, 1, 0, 0, 0, 1, 1, 0},
                                  {1, 0, 0, 1, 0, 0, 1, 0},
                                  {0, 0, 1, 1, 1, 0, 0, 0},
                                  {1, 0, 0, 1, 0, 0, 1, 1},
                                  {1, 1, 0, 0, 0, 0, 1, 0},
                                  {0, 1, 0, 1, 1, 0, 0, 0}};
    int region[NB_ROWS][NB_COLS] = {{0}};
    int stackRow[NB_ROWS * NB_COLS], stackCol[NB_ROWS * NB_COLS];
    int moveRow[4] = {-1, 1, 0, 0}, moveCol[4] = {0, 0, -1, 1};
    int row, col, top, curRow, curCol, nextRow, nextCol, dir, size, nbRegions = 0;
    for (row = 0; row < NB_ROWS; row++)
        for (col = 0; col < NB_COLS; col++) {
            if (zone[row][col] != 1 || region[row][col] != 0)
                continue;
            nbRegions++;                          // منطقة جديدة
            region[row][col] = nbRegions;
            top = 0; size = 0;
            stackRow[top] = row; stackCol[top] = col; top++;
            while (top > 0) {
                top--;
                curRow = stackRow[top]; curCol = stackCol[top];
                size++;
                for (dir = 0; dir < 4; dir++) {   // الجيران الأربعة
                    nextRow = curRow + moveRow[dir];
                    nextCol = curCol + moveCol[dir];
                    if (nextRow >= 0 && nextRow < NB_ROWS && nextCol >= 0 && nextCol < NB_COLS
                        && zone[nextRow][nextCol] == 1 && region[nextRow][nextCol] == 0) {
                        region[nextRow][nextCol] = nbRegions;   // نعلّم قبل الدفع
                        stackRow[top] = nextRow; stackCol[top] = nextCol; top++;
                    }
                }
            }
            printf("Region %d: %d block(s)\n", nbRegions, size);
        }
    for (row = 0; row < NB_ROWS; row++) {
        for (col = 0; col < NB_COLS; col++)
            printf("%2d", region[row][col]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Region 1: 3 block(s)
Region 2: 3 block(s)
Region 3: 5 block(s)
Region 4: 4 block(s)
Region 5: 3 block(s)
Region 6: 2 block(s)
 1 1 0 0 0 2 2 0
 1 0 0 3 0 0 2 0
 0 0 3 3 3 0 0 0
 4 0 0 3 0 0 5 5
 4 4 0 0 0 0 5 0
 0 4 0 6 6 0 0 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.50 أطول مسار تصفح مشترك

علم البيانات ★★★ صعب

مسارا تصفح لزبونين ممثلان بجدولي أرقام صفحات. اكتب برنامجًا يجد **أطول متتالية مشتركة** (LCS) بينهما بالبرمجة الديناميكية: `table[i][j]` طول LCS لأول `i` صفحة من الأول وأول `j` من الثاني، ثم يعيد بناء المتتالية.

خطوات الحل

1) إذا تساوت الصفحتان:

```
table[posA][posB] = table[posA-1][posB-1] + 1
```

.

2) وإلا: الأكبر بين `table[posA-1][posB]` و`table[posA][posB-1]`.

3) إعادة البناء من الخانة الأخيرة نحو `[0][0]`.

البرنامج بلغة C

```c
#include <stdio.h>

#define LEN_A 8
#define LEN_B 7

int main(void) {
    int pagesA[LEN_A] = {1, 4, 2, 7, 3, 9, 5, 6};      // الصفحة 1 = الرئيسية
    int pagesB[LEN_B] = {1, 2, 4, 7, 9, 6, 5};
    int table[LEN_A + 1][LEN_B + 1], common[LEN_B];
    int posA, posB, length;
    for (posA = 0; posA <= LEN_A; posA++)
        for (posB = 0; posB <= LEN_B; posB++) {
            if (posA == 0 || posB == 0)
                table[posA][posB] = 0;                  // متتالية فارغة
            else if (pagesA[posA - 1] == pagesB[posB - 1])
                table[posA][posB] = table[posA - 1][posB - 1] + 1;
            else if (table[posA - 1][posB] >= table[posA][posB - 1])
                table[posA][posB] = table[posA - 1][posB];
            else
                table[posA][posB] = table[posA][posB - 1];
        }
    length = table[LEN_A][LEN_B];
    posA = LEN_A; posB = LEN_B;
    while (posA > 0 && posB > 0) {                      // إعادة البناء
        if (pagesA[posA - 1] == pagesB[posB - 1]) {
            common[table[posA][posB] - 1] = pagesA[posA - 1];
            posA--; posB--;
        } else if (table[posA - 1][posB] >= table[posA][posB - 1]) {
            posA--;
        } else {
            posB--;
        }
    }
    printf("LCS length = %d\nCommon path:", length);
    for (posA = 0; posA < length; posA++)
        printf(" %d", common[posA]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
LCS length = 5
Common path: 1 4 7 9 5
```

المخطط الإنسيابي (Organigramme)

